

ĐẠI HỌC BÁCH KHOA HÀ NỘI
KHOA TOÁN-TIN

—————o0o—————



ĐỒ ÁN TỐT NGHIỆP

XÂY DỰNG GIAO THỨC NHẮN TIN AN TOÀN TÍCH HỢP GIẢI
PHÁP MẬT MÃ HẠU LƯỢNG TỬ

Sinh viên thực hiện: **LÊ HOÀNG HIỆP**
Email: hiiep.lh227175@sis.hust.edu.vn
Mã sinh viên: 20227175
Chuyên ngành: Hệ thống thông tin quản lý

Giảng viên hướng dẫn: **TS. TRẦN ANH TÚ**

Chữ ký của GVHD

Hà Nội, 06/2026

NHẬN XÉT CỦA GIẢNG VIÊN HƯỚNG DẪN

1. Mục tiêu và nội dung của đồ án

- (a) Tìm hiểu tổng quan về mã hóa đầu cuối trong ứng dụng nhắn tin, mật mã hậu lượng tử và thuật toán Kyber đã được NIST chuẩn hóa.
- (b) Thiết kế và xây dựng ứng dụng nhắn tin an toàn tích hợp giải pháp mật mã hậu lượng tử nhằm bảo đảm tính bảo mật, toàn vẹn và khả năng kháng tấn công lượng tử.

2. Kết quả đạt được

3. Ý thức làm việc của sinh viên

Hà Nội, ngày tháng năm 2026

Giảng viên hướng dẫn

TS. Trần Anh Tú

Phiếu báo cáo tiến độ đồ án

Hình 1: Phiếu báo cáo tiến độ đồ án

Mục lục

Danh sách hình vẽ	9
Chương 1 Kiến thức cơ sở	10
1.1 Tổng quan về mã hóa trong ứng dụng nhắn tin	10
1.1.1 Mô hình đe dọa trong giao tiếp mạng	10
1.1.2 Các cấp độ bảo vệ dữ liệu	10
1.1.3 Yêu cầu đối với một hệ thống mã hóa đầu cuối	11
1.2 Mật mã đối xứng và mã hóa có xác thực	11
1.2.1 Thuật toán AES và chế độ GCM	11
1.2.2 Hàm dẫn xuất khóa từ mật khẩu	11
1.3 Mật mã khóa công khai truyền thống và mối đe dọa từ máy tính lượng tử .	12
1.3.1 Nguyên lý mã hóa khóa công khai	12
1.3.2 Thuật toán Shor và hệ quả	12
1.4 Mật mã hậu lượng tử	13
1.4.1 Các họ thuật toán kháng lượng tử	13
1.4.2 Cơ sở toán học của mật mã lattice	13
1.4.3 Chương trình chuẩn hóa của NIST	14
1.4.4 Cơ chế Key Encapsulation	14
1.5 Thuật toán Kyber và chuẩn ML-KEM	15
1.5.1 Không gian làm việc và các tham số	15
1.5.2 Phân phối nhiễu tâm nhị thức	16
1.5.3 Sinh khóa	16
1.5.4 Encapsulate	17
1.5.5 Decapsulate	18
1.5.6 Phân tích tính đúng đắn	18
1.5.7 Biến đổi Fujisaki-Okamoto và bảo mật IND-CCA2	19
1.5.8 Vì sao Kyber phù hợp với hệ thống nhắn tin	19
1.5.9 Các biến thể và mức an toàn	19
1.6 Forward secrecy, Double Ratchet và MLS	20
1.6.1 Forward secrecy và post-compromise security	20
1.6.2 Double Ratchet trong hội thoại hai người	20
1.6.3 Messaging Layer Security cho hội thoại nhóm	22
1.7 Giao thức TLS và vai trò trong kiến trúc bảo mật	24
1.8 Các công nghệ triển khai	24
1.8.1 Ngôn ngữ Python và PyQt5	24
1.8.2 Ngôn ngữ C và thư viện hệ thống	24
1.8.3 Thư viện Open Quantum Safe	25
1.8.4 Thư viện cryptography và các thao tác AES-GCM	25
1.8.5 Cơ sở dữ liệu MySQL	25
1.8.6 Docker và Docker Compose	25

Chương 2 Phân tích và thiết kế hệ thống	26
2.1 Khảo sát thực tế	26
2.1.1 Thực trạng các giao thức nhắn tin mã hóa đầu cuối	26
2.1.2 Các cơ chế bảo mật được sử dụng trong hệ thống nhắn tin hiện nay	26
2.1.3 Những thách thức trong thiết kế giao thức nhắn tin thế hệ mới . .	27
2.1.4 Xu hướng tích hợp mật mã hậu lượng tử vào hệ thống nhắn tin . .	27
2.2 Phân tích yêu cầu hệ thống	27
2.2.1 Mục tiêu chức năng và phi chức năng	27
2.2.2 Đối tượng sử dụng và kịch bản sử dụng chính	28
2.2.3 Danh sách Use Case mức hệ thống	29
2.2.4 Sơ đồ Usecase tổng quan	30
2.2.5 Đặc tả chức năng	31
2.3 Thiết kế hệ thống	39
2.3.1 Thiết kế cơ sở dữ liệu	39
2.3.2 Mô tả thực thể	41
2.3.3 Thiết kế cấu trúc đối tượng	44
2.3.4 Thiết kế luồng xử lý	48
2.4 Thiết kế giao thức bảo mật	50
2.4.1 Thiết kế mã hóa đầu cuối	51
2.5 Thiết kế giao thức truyền tin	58
2.5.1 Các loại gói chính	58
2.5.2 Định dạng body mã hóa	58
2.5.3 Metadata và quyền riêng tư	59
2.6 Thiết kế client	59
2.6.1 Tổ chức mã nguồn	59
2.6.2 Vai trò của App	60
2.6.3 Giao diện chat	60
2.7 Thiết kế server	60
2.7.1 Kiến trúc phân tầng	60
2.7.2 Xác thực, rate limit và đổi mật khẩu	61
2.7.3 Lưu và chuyển tiếp message	61
2.7.4 Group và MLS validation	61
2.7.5 Vận hành và giám sát	61
Chương 3 Cài đặt và đánh giá thực nghiệm	62
3.1 Môi trường cài đặt	62
3.2 Cài đặt và khởi động hệ thống	62
3.3 Kết quả giao diện	63
3.3.1 Đăng nhập và danh sách hội thoại	63
3.3.2 DM, identity và thao tác tin nhắn	65
3.3.3 Group chat và quản lý thành viên	67
3.3.4 Backup, profile và trạng thái cuối	69
3.4 Kiểm thử chức năng	70
3.4.1 Kiểm thử thuộc tính an toàn	71
3.5 Kiểm toán mã nguồn server	72
3.5.1 Kiểm thử group MLS	73
3.6 Quan sát vận hành	74
3.7 Đánh giá bảo mật	75
3.7.1 Nội dung tin nhắn	75
3.7.2 Kháng lượng tử	76
3.7.3 Forward secrecy và post-compromise security	76

3.7.4	Xác thực danh tính	76
3.7.5	Metadata	76
3.7.6	Local plaintext cache	77
3.8	Đánh giá kiến trúc phần mềm	77
3.8.1	Client	77
3.8.2	Server	77
3.8.3	Cơ sở dữ liệu	78
3.9	Các nguy cơ còn lại	78
3.10	Hướng phát triển	78
3.10.1	Multi-device	78
3.10.2	Nâng cấp bảo vệ metadata	78
3.10.3	Cải thiện UI và nền tảng	79
Kết luận		80
Tài liệu tham khảo		80

Danh sách hình vẽ

1	Phiếu báo cáo tiến độ đồ án	2
2.1	Sơ đồ use case tổng quan của SecChat	30
2.2	Sơ đồ hoạt động: Đăng nhập và công bố khóa (UC02)	33
2.3	Sơ đồ hoạt động: Mở DM và gửi tin (UC05)	35
2.4	Sơ đồ hoạt động: Quản lý thành viên và vai trò (UC10)	37
2.5	Sơ đồ hoạt động: Sao lưu và khôi phục E2EE (UC13)	39
2.6	ERD phân hệ lõi: người dùng, hội thoại và tin nhắn	40
2.7	ERD phân hệ khóa và định danh (PQXDH, MLS KeyPackage, identity log)	40
2.8	ERD phân hệ nhóm MLS (group state, handshake, system event, pending op)	41
2.9	ERD phân hệ tiện ích tin nhắn và thông báo (reaction, pin, notification)	41
2.10	Sơ đồ class diagram các lớp chính của hệ thống	45
2.11	Sơ đồ tuần tự: đăng nhập và công bố khóa	49
2.12	Sơ đồ tuần tự: gửi tin nhắn DM E2EE	49
2.13	Sơ đồ tuần tự: thay đổi thành viên group bằng MLS	50
2.14	Sơ đồ tuần tự: tải lịch sử, read cursor và system event	50
2.15	Vòng đời các khóa chính trong SecChat	51
2.16	Luồng identity transparency giữa client, chat server và auditor	53
2.17	Sơ đồ tuần tự: bắt tay PQXDH hybrid (X25519 + ML-KEM-1024/Kyber)	54
2.18	Luồng Double Ratchet cho tin nhắn DM	55
2.19	Sơ đồ tuần tự: vòng đời khóa group MLS X-Wing (KeyPackage → Commit/Welcome → epoch → application message)	56
2.20	Chuỗi MLS Commit dùng để kiểm tra thay đổi membership	56
2.21	Transcript checkpoint giúp phát hiện group history không nhất quán	57
2.22	Quan hệ giữa plaintext cache cục bộ và backup E2EE	58
2.23	Data flow của một tin nhắn đã mã hóa đầu cuối	59
2.24	Ranh giới giữa UI controller và crypto engine	60
3.1	Mô hình container khi chạy SecChat	62
3.2	Giao diện server khi hệ thống đang chạy	63
3.3	Màn hình đăng nhập trong môi trường VNC	64
3.4	Danh sách bạn bè, DM và group sau khi đăng nhập	64
3.5	Gửi nhận DM hai chiều với nội dung được giải mã ở client	65
3.6	Màn hình kiểm tra safety code trong DM	66
3.7	Popover thao tác tin nhắn	66
3.8	Gửi file trong hội thoại	67
3.9	Màn hình group chat với trạng thái MLS/PQ	68
3.10	Quản lý thành viên, role và system event trong group	68
3.11	Profile, privacy, avatar và backup E2EE	69
3.12	Các tùy chọn privacy và cache	70

3.13	Kết quả chạy bộ kiểm thử tự động: 19/19 file PASS, 0 fail	71
3.14	Chi tiết test-case con: domain (75), build server (4), an toàn (16), metrics (17) đều PASS.	71
3.15	Bảng chứng server không lưu plaintext: truy vấn bảng messages, mọi body là ciphertext prefix S3DR.	72
3.16	Kiểm toán mã C trong container: cppcheck 36/36, ASan/UBSan 75/75, libFuzzer 30k×2 0 lỗi, 0 crash.	73
3.17	Selftest MLS bridge (OpenMLS): ciphersuite X-Wing hybrid; validator chấp nhận state hợp lệ, từ chối commit giả mạo.	74
3.18	Ảnh GUI metric của server.	75

Lời mở đầu

Trong thập niên vừa qua, các ứng dụng nhắn tin đã trở thành phương tiện giao tiếp chủ yếu của hàng tỷ người dùng trên thế giới. Cùng với sự phát triển mạnh mẽ đó, mức độ nhạy cảm của dữ liệu được trao đổi qua các ứng dụng này cũng ngày một lớn, từ thông tin cá nhân, giao dịch tài chính cho đến các tài liệu nội bộ của tổ chức. Nhu cầu bảo mật thông tin trong quá trình truyền tải trở thành một yêu cầu bắt buộc, và mã hóa đầu cuối đã dần trở thành tiêu chuẩn trong thiết kế các nền tảng nhắn tin hiện đại như Signal, WhatsApp hay iMessage.

Tuy nhiên, phần lớn các hệ mã hóa khóa công khai đang được sử dụng hiện nay đều dựa trên các bài toán khó truyền thống như phân tích số nguyên lớn trong RSA hay bài toán logarit rời rạc trên đường cong elliptic trong ECDH. Sự xuất hiện của máy tính lượng tử cùng thuật toán Shor đã đặt ra nguy cơ rằng toàn bộ các giao thức trao đổi khóa dựa trên nền tảng này sẽ không còn an toàn trong tương lai không xa. Mối đe dọa không chỉ giới hạn ở thời điểm máy tính lượng tử thực sự xuất hiện, mà còn hiện hữu ngay lúc này qua mô hình tấn công “harvest now, decrypt later”, trong đó kẻ tấn công thu thập trước các bản mã và chờ đến khi công nghệ đủ mạnh để giải mã.

Nhằm giải quyết thách thức đó, Viện Tiêu chuẩn và Công nghệ Quốc gia Hoa Kỳ (NIST) đã tổ chức chương trình chuẩn hóa mật mã hậu lượng tử kéo dài nhiều năm, và đến năm 2024 đã công bố các chuẩn chính thức bao gồm ML-KEM, được phát triển dựa trên thuật toán Kyber. Đây là thuật toán trao đổi khóa dạng Key Encapsulation Mechanism, xây dựng trên bài toán Module Learning With Errors trong mật mã lattice, được thiết kế để kháng lại cả tấn công cổ điển lẫn tấn công bằng máy tính lượng tử.

Xuất phát từ bối cảnh thực tế cùng mong muốn tìm hiểu sâu hơn về mật mã học hiện đại và các công nghệ bảo mật mạng, em đã chọn đề tài “Xây dựng giao thức nhắn tin an toàn tích hợp giải pháp mật mã hậu lượng tử” để làm đồ án tốt nghiệp. Đề tài hướng tới việc khảo sát nguyên lý của các thuật toán mật mã hậu lượng tử, thiết kế một kiến trúc nhắn tin đầu cuối có bắt tay khóa hybrid hậu lượng tử, và hiện thực hóa kiến trúc này thành một ứng dụng desktop có các chức năng chính của một nền tảng nhắn tin thực tế.

Nội dung chính của đồ án bao gồm 3 chương ngoài phần mở đầu và kết luận, cụ thể:

- **Chương 1: Kiến thức cơ sở:** Chương này trình bày các kiến thức nền tảng về mã hóa đầu cuối trong ứng dụng nhắn tin, mật mã đối xứng hiện đại, mối đe dọa của máy tính lượng tử đối với mật mã khóa công khai truyền thống, nguyên lý của mật mã hậu lượng tử và thuật toán Kyber, cùng các công nghệ được sử dụng để triển khai ứng dụng.
- **Chương 2: Phân tích và thiết kế hệ thống:** Chương này tập trung khảo sát các ứng dụng nhắn tin bảo mật hiện hành, phân tích yêu cầu, đề xuất kiến trúc tổng thể và trình bày thiết kế các thành phần cốt lõi của hệ thống, bao gồm cơ chế E2EE, bắt tay khóa kiểu PQXDH, Double Ratchet, group theo epoch, identity transparency, cơ sở dữ liệu, client và server.
- **Chương 3: Cài đặt và đánh giá hệ thống:** Chương này trình bày cách triển

khai hệ thống bằng Docker, mô tả các luồng sử dụng chính, quan sát dữ liệu lưu trên server, đánh giá bảo mật, kiến trúc phần mềm, khả năng sử dụng, hiệu năng, giới hạn còn lại và các hướng phát triển tiếp theo.

Cuối cùng, em xin chân thành cảm ơn các thầy cô trong khoa Toán-Tin đã tận tình giảng dạy và trang bị cho em những kiến thức quý báu trong suốt quá trình học tập. Đặc biệt, em xin gửi lời cảm ơn sâu sắc đến thầy Trần Anh Tú đã luôn quan tâm, hướng dẫn và tạo điều kiện giúp em hoàn thành đồ án này.

Chương 1

Kiến thức cơ sở

1.1 Tổng quan về mã hóa trong ứng dụng nhắn tin

1.1.1 Mô hình đe dọa trong giao tiếp mạng

Khi một tin nhắn được gửi đi giữa hai người dùng, dữ liệu phải đi qua nhiều thành phần trung gian trước khi đến được đích. Trên đường truyền từ thiết bị của người gửi đến máy chủ của nhà cung cấp dịch vụ, gói tin có thể bị các điểm trung gian như bộ định tuyến, nhà cung cấp dịch vụ mạng hoặc các thiết bị phân tích lưu lượng theo dõi. Ngay cả khi dữ liệu đến được máy chủ, nó vẫn có nguy cơ bị chính nhà cung cấp dịch vụ đọc, lưu trữ hoặc chuyển giao cho bên thứ ba. Đây là thực tế đã được ghi nhận trong nhiều vụ việc lộ dữ liệu người dùng từ các nền tảng nhắn tin phổ biến trong những năm gần đây.

Mô hình đe dọa đối với một ứng dụng nhắn tin vì vậy cần bao hàm nhiều loại đối tượng tấn công khác nhau. Kẻ tấn công thụ động có thể là nhà cung cấp dịch vụ mạng hoặc chính phủ, thực hiện nghe lén trên đường truyền để thu thập metadata và nội dung tin nhắn. Kẻ tấn công chủ động có thể thực hiện tấn công xen giữa, mạo danh một trong hai đầu để đánh lừa đối phương. Một mối đe dọa đặc biệt quan trọng nhưng thường bị xem nhẹ chính là máy chủ của nhà cung cấp dịch vụ, nơi có khả năng truy cập toàn bộ dữ liệu nếu không áp dụng mã hóa đầu cuối.

1.1.2 Các cấp độ bảo vệ dữ liệu

Trong thực tế, các ứng dụng nhắn tin hiện nay áp dụng nhiều cấp độ bảo vệ khác nhau với mức độ hiệu quả không đồng đều. Cấp độ cơ bản nhất là mã hóa tầng vận chuyển, thường được hiện thực hóa qua TLS. Cơ chế này chỉ bảo vệ dữ liệu trên đường truyền từ thiết bị tới máy chủ, đồng nghĩa với việc máy chủ vẫn có khả năng đọc được toàn bộ nội dung tin nhắn trước khi chuyển tiếp.

Cấp độ cao hơn là mã hóa phía máy chủ, trong đó dữ liệu được lưu trữ dưới dạng mã hóa nhưng chính máy chủ nắm giữ khóa giải mã. Phương án này bảo vệ tốt trước các rủi ro về rò rỉ dữ liệu do xâm phạm phần cứng hoặc bản sao lưu, nhưng vẫn không ngăn được việc nhân viên nội bộ hoặc cơ quan chức năng truy cập nội dung khi có yêu cầu.

Cấp độ bảo vệ cao nhất và cũng là tiêu chuẩn trong các hệ thống nhắn tin hiện đại chính là mã hóa đầu cuối. Trong mô hình này, tin nhắn được mã hóa ngay trên thiết bị của người gửi bằng khóa mà chỉ người nhận có thể giải mã. Máy chủ chỉ đóng vai trò trung chuyển các bản mã mà không có bất kỳ khả năng đọc nội dung nào. Đây là mô hình được Signal tiên phong và sau đó được WhatsApp, iMessage và nhiều nền tảng khác áp dụng.

1.1.3 Yêu cầu đối với một hệ thống mã hóa đầu cuối

Một hệ thống mã hóa đầu cuối an toàn cần đáp ứng đồng thời nhiều thuộc tính bảo mật. Tính bí mật đảm bảo rằng chỉ người nhận hợp lệ mới có thể đọc được nội dung tin nhắn. Tính toàn vẹn bảo đảm rằng nội dung không bị sửa đổi trong quá trình truyền tải, đồng thời cho phép người nhận phát hiện mọi thay đổi dù nhỏ nhất. Tính xác thực cho phép người nhận xác minh chắc chắn rằng tin nhắn xuất phát từ người gửi được tuyên bố. Ngoài ra, các hệ thống tiên tiến còn hướng tới thuộc tính forward secrecy, nghĩa là ngay cả khi khóa dài hạn bị lộ trong tương lai, các phiên tin nhắn trong quá khứ vẫn không thể bị giải mã.

Để hiện thực hóa các yêu cầu trên, hệ thống cần kết hợp giữa mã hóa đối xứng cho nội dung tin nhắn với một cơ chế trao đổi khóa an toàn giữa hai đầu. Chính cơ chế trao đổi khóa là nơi mà các thuật toán mật mã khóa công khai truyền thống đang đối mặt với nguy cơ trực tiếp từ máy tính lượng tử, đòi hỏi phải được thay thế bằng những giải pháp kháng lượng tử.

1.2 Mật mã đối xứng và mã hóa có xác thực

1.2.1 Thuật toán AES và chế độ GCM

AES là một thuật toán mã hóa khối được Viện Tiêu chuẩn và Công nghệ Quốc gia Hoa Kỳ chuẩn hóa từ năm 2001 và hiện vẫn là thuật toán mã hóa đối xứng phổ biến nhất trên thế giới. AES hoạt động trên các khối dữ liệu 128-bit với độ dài khóa 128, 192 hoặc 256 bit, trong đó biến thể AES-256 cung cấp mức an toàn cao nhất. Thuật toán bao gồm nhiều vòng lặp với các thao tác thay thế, dịch hàng, trộn cột và cộng khóa vòng, tạo ra một hoán vị phụ thuộc khóa có tính kháng tấn công cao.

AES ở dạng nguyên bản chỉ định nghĩa phép mã hóa trên một khối 128-bit, do đó trong thực tế cần kết hợp với một chế độ vận hành để xử lý các thông điệp có độ dài bất kỳ. Trong các ứng dụng hiện đại, chế độ GCM là lựa chọn được khuyến nghị bởi NIST nhờ khả năng cung cấp đồng thời mã hóa và xác thực trong một thao tác duy nhất. Cấu trúc này được gọi chung là AEAD, viết tắt của Authenticated Encryption with Associated Data.

Khi vận hành AES-GCM, ngoài khóa và bản rõ, thuật toán còn cần một giá trị nonce 96-bit. Nonce phải là duy nhất đối với mỗi lần mã hóa dưới cùng một khóa, nhưng không cần bí mật và có thể được truyền công khai cùng với bản mã. Đầu ra của thuật toán bao gồm bản mã có độ dài bằng bản rõ và một thẻ xác thực 128-bit. Phía giải mã sẽ kiểm tra thẻ này trước khi trả về bản rõ, bất kỳ sự thay đổi nào trong bản mã hoặc nonce đều dẫn đến việc thẻ không hợp lệ và quá trình giải mã sẽ thất bại. Đây là một tính chất rất quan trọng giúp hệ thống tự động phát hiện tấn công sửa đổi dữ liệu mà không cần cơ chế xác thực riêng.

1.2.2 Hàm dẫn xuất khóa từ mật khẩu

Trong các ứng dụng nhắn tin sử dụng tài khoản, mật khẩu là thông tin duy nhất người dùng có thể ghi nhớ và sử dụng để thiết lập truy cập. Tuy nhiên, mật khẩu thường có entropy thấp và không thể được sử dụng trực tiếp làm khóa mã hóa, vì vậy cần một hàm dẫn xuất khóa để chuyển đổi mật khẩu thành khóa có kích thước và entropy phù hợp.

PBKDF2 là hàm dẫn xuất khóa được chuẩn hóa trong RFC 8018 và được sử dụng rộng rãi trong nhiều hệ thống bảo mật. PBKDF2 nhận đầu vào gồm mật khẩu, một giá

trị salt, số vòng lặp và hàm băm cơ sở, sau đó trả về khóa dẫn xuất có độ dài mong muốn. Salt đóng vai trò quan trọng trong việc ngăn chặn tấn công sử dụng bảng băm tiền tính toán, còn số vòng lặp cao làm tăng chi phí tính toán của mỗi lần thử mật khẩu, khiến các tấn công brute-force trở nên tốn kém. Khuyến nghị mới nhất của NIST năm 2023 đề xuất sử dụng ít nhất 600.000 vòng khi kết hợp với hàm băm SHA-256, nhằm cân bằng giữa an toàn và trải nghiệm người dùng.

Một đặc điểm quan trọng của PBKDF2 là tính xác định. Khi đầu vào gồm mật khẩu, salt, số vòng và hàm băm không đổi, đầu ra luôn là cùng một khóa. Tính chất này cho phép ứng dụng dẫn xuất lại khóa chủ mỗi khi người dùng đăng nhập, mà không cần lưu khóa này ở bất cứ đâu.

1.3 Mật mã khóa công khai truyền thống và mối đe dọa từ máy tính lượng tử

1.3.1 Nguyên lý mã hóa khóa công khai

Khác với mã hóa đối xứng trong đó cả hai bên dùng chung một khóa bí mật, mật mã khóa công khai sử dụng một cặp khóa gồm khóa công khai và khóa bí mật. Khóa công khai có thể được phát hành rộng rãi và cho phép bất kỳ ai mã hóa thông điệp hoặc xác minh chữ ký số, trong khi khóa bí mật chỉ được người sở hữu giữ lại và dùng để giải mã hoặc ký. Cơ chế này giải quyết được bài toán trao đổi khóa trong môi trường mở, vốn là trở ngại lớn nhất của mã hóa đối xứng.

An toàn của các hệ mã hóa khóa công khai dựa trên các bài toán được cho là khó với máy tính cổ điển. Hệ RSA dựa trên độ khó của bài toán phân tích một số nguyên lớn thành tích các số nguyên tố. Các hệ mã đường cong elliptic như ECDH và ECDSA dựa trên bài toán logarit rời rạc trên nhóm điểm của đường cong elliptic, với độ khó tương đương một số bit gấp khoảng năm lần so với RSA ở cùng mức an toàn, cho phép khóa nhỏ gọn hơn đáng kể.

1.3.2 Thuật toán Shor và hệ quả

Năm 1994, Peter Shor công bố một thuật toán lượng tử có khả năng phân tích số nguyên và tính logarit rời rạc trong thời gian đa thức. Đối với máy tính cổ điển, cả hai bài toán này đều đòi hỏi thời gian dưới dạng hàm mũ so với kích thước đầu vào. Điều đó có nghĩa là một máy tính lượng tử đủ lớn, đủ ổn định và chạy được thuật toán Shor sẽ có thể bẻ khóa RSA và ECDH trong thời gian khả thi, phá vỡ toàn bộ nền tảng mật mã khóa công khai đang được sử dụng.

Mặc dù các máy tính lượng tử hiện tại chưa đủ quy mô để thực thi Shor trên khóa có độ dài thực tế, xu hướng phát triển phần cứng lượng tử đang diễn ra rất nhanh, với các thông báo mới về số qubit và độ ổn định xuất hiện liên tục trong những năm gần đây. Chính vì vậy, cộng đồng nghiên cứu đã sớm đặt ra yêu cầu phải chuẩn bị các thuật toán thay thế trước khi máy tính lượng tử trở thành hiện thực.

Ngoài ra, có một lớp tấn công cần được tính đến ngay lập tức dù máy tính lượng tử chưa tồn tại, đó là mô hình tấn công “harvest now, decrypt later”. Trong mô hình này, kẻ tấn công thu thập và lưu trữ dữ liệu mã hóa hiện tại, chờ đến khi máy tính lượng tử đủ mạnh xuất hiện để tiến hành giải mã. Những dữ liệu có giá trị lâu dài như hồ sơ y tế, thông tin tình báo hay tài liệu pháp lý đặc biệt dễ trở thành mục tiêu. Đó là lý do các tiêu chuẩn mật mã hậu lượng tử cần được triển khai càng sớm càng tốt, ngay trong giai đoạn chuyển tiếp.

1.4 Mật mã hậu lượng tử

1.4.1 Các họ thuật toán kháng lượng tử

Mật mã hậu lượng tử là tập hợp các thuật toán được thiết kế để có thể chạy trên phần cứng cổ điển hiện có nhưng vẫn an toàn trước các tấn công từ cả máy tính cổ điển và máy tính lượng tử. Các thuật toán này được xây dựng trên các bài toán khó mà chưa có thuật toán lượng tử nào được biết đến có khả năng giải trong thời gian đa thức.

Cộng đồng nghiên cứu đã đề xuất nhiều họ thuật toán khác nhau. Họ lattice-based dựa trên độ khó của các bài toán liên quan đến lattice như Learning With Errors hay Shortest Vector Problem, nổi bật với tính linh hoạt và hiệu năng tốt. Họ code-based dựa trên độ khó của giải mã mã tuyến tính ngẫu nhiên, có lịch sử lâu đời từ hệ McEliece năm 1978 nhưng có nhược điểm là khóa công khai rất lớn. Họ hash-based xây dựng chữ ký số trên nền các hàm băm đã được chứng minh an toàn, ví dụ như SPHINCS+, có ưu điểm là độ tin cậy về mặt lý thuyết rất cao nhưng chữ ký lại tương đối lớn. Họ multivariate và họ isogeny đang được nghiên cứu nhưng còn nhiều thách thức về mặt bảo mật và hiệu năng.

1.4.2 Cơ sở toán học của mật mã lattice

Trước khi đi vào thuật toán Kyber, cần nhắc lại một số khái niệm nền tảng về lattice, vốn là đối tượng toán học trung tâm cho gần như toàn bộ các đề xuất hậu lượng tử được NIST đánh giá cao nhất.

Định nghĩa lattice. Cho n vector $\mathbf{b}_1, \mathbf{b}_2, \dots, \mathbf{b}_n$ độc lập tuyến tính trong $\mathbb{R}^m$. Lattice $\mathcal{L}$ sinh bởi các vector này được định nghĩa là tập hợp tất cả các tổ hợp tuyến tính với hệ số nguyên:

$$\mathcal{L}(\mathbf{b}_1, \dots, \mathbf{b}_n) = \left\{ \sum_{i=1}^n x_i \mathbf{b}_i \mid x_i \in \mathbb{Z} \right\}.$$

Tập $\{\mathbf{b}_1, \dots, \mathbf{b}_n\}$ được gọi là một cơ sở của lattice, còn n là bậc của lattice. Về mặt hình học, lattice là một mạng các điểm phân bố đều trong không gian, và cùng một lattice có thể được biểu diễn bằng vô số cơ sở khác nhau.

Các bài toán khó trên lattice. Hai bài toán nền tảng được sử dụng làm cơ sở cho an toàn của mật mã lattice gồm:

- **Shortest Vector Problem (SVP):** cho một cơ sở của lattice $\mathcal{L}$, tìm vector khác không $\mathbf{v} \in \mathcal{L}$ có độ dài Euclid $\|\mathbf{v}\|$ nhỏ nhất.
- **Closest Vector Problem (CVP):** cho một cơ sở của lattice $\mathcal{L}$ và một điểm $\mathbf{t} \in \mathbb{R}^m$ không thuộc lattice, tìm vector $\mathbf{v} \in \mathcal{L}$ sao cho $\|\mathbf{v} - \mathbf{t}\|$ nhỏ nhất.

Cả hai bài toán đều đã được chứng minh là NP-hard trong trường hợp chính xác và vẫn rất khó ngay cả với các phiên bản xấp xỉ. Điều đặc biệt quan trọng là cho đến nay, không có thuật toán lượng tử nào được biết giải SVP hoặc CVP trong thời gian đa thức, khác hoàn toàn với bài toán phân tích số nguyên hay logarit rời rạc mà thuật toán Shor đã giải được.

Bài toán Learning With Errors (LWE). Bài toán LWE do Oded Regev đề xuất năm 2005 là cầu nối giữa các bài toán lattice dạng hình học nêu trên và mật mã ứng dụng. Cho n, q là các số nguyên dương và χ là một phân phối xác suất trên $\mathbb{Z}_q$ trả về các giá trị nhỏ. Với vector bí mật $\mathbf{s} \in \mathbb{Z}_q^n$, phân phối LWE sinh các mẫu có dạng

$$(\mathbf{a}, b) = (\mathbf{a}, \langle \mathbf{a}, \mathbf{s} \rangle + e \bmod q),$$

trong đó $\mathbf{a} \in \mathbb{Z}_q^n$ được chọn đều ngẫu nhiên và $e \leftarrow \chi$ là nhiễu nhỏ. Bài toán *search-LWE* yêu cầu khôi phục $\mathbf{s}$ khi biết nhiều mẫu $(\mathbf{a}_i, b_i)$, còn bài toán *decision-LWE* yêu cầu phân biệt phân phối các mẫu LWE với phân phối đều. Khi viết dưới dạng ma trận với m mẫu, ta có

$$\mathbf{b} = A \cdot \mathbf{s} + \mathbf{e} \bmod q,$$

với $A \in \mathbb{Z}_q^{m \times n}$ ngẫu nhiên và $\mathbf{e} \in \mathbb{Z}_q^m$ là vector nhiễu có các phần tử nhỏ. Nhờ công trình của Regev, bài toán decision-LWE có mức độ khó được chứng minh bằng reduction tới SVP trong trường hợp xấu nhất, nghĩa là nếu LWE giải được trong thời gian trung bình thì SVP cũng giải được trong thời gian tồi nhất. Đây là tính chất rất mạnh mà rất ít giả thiết mật mã khác có được.

Bài toán Module-LWE. Nếu thay không gian làm việc từ $\mathbb{Z}_q$ sang vành đa thức $R_q = \mathbb{Z}_q[X]/(X^n + 1)$ với n là một lũy thừa của 2, ta thu được biến thể Module-LWE. Các phần tử của R_q là các đa thức có bậc nhỏ hơn n với hệ số trong $\mathbb{Z}_q$, và phép nhân trong R_q được thực hiện modulo đa thức $X^n + 1$. Bài toán Module-LWE cho một ma trận $\mathbf{A} \in R_q^{k \times k}$ và vector $\mathbf{b} = \mathbf{A} \cdot \mathbf{s} + \mathbf{e} \in R_q^k$ yêu cầu khôi phục $\mathbf{s} \in R_q^k$. Về bản chất Module-LWE là LWE đóng gói lại theo khối: thay vì một số nguyên bí mật ta có một đa thức bí mật có n hệ số, và các phép nhân được thực hiện trên đa thức chứ không phải số nguyên riêng lẻ.

Ưu điểm lớn của Module-LWE là giảm đáng kể kích thước khóa so với LWE thuần, vì một phần tử của R_q đã mang thông tin tương đương n phần tử của $\mathbb{Z}_q$, trong khi vẫn giữ được độ khó tương đương nhờ cấu trúc đại số phong phú của vành đa thức. Đồng thời, các phép nhân đa thức có thể được tăng tốc đáng kể bằng biến đổi NTT (Number Theoretic Transform), giúp Module-LWE đạt được hiệu năng gần với các hệ mã hóa đối xứng.

1.4.3 Chương trình chuẩn hóa của NIST

Nhận thức được tầm quan trọng của việc chuẩn hóa mật mã hậu lượng tử, Viện Tiêu chuẩn và Công nghệ Quốc gia Hoa Kỳ đã khởi động chương trình tuyển chọn thuật toán từ năm 2016. Chương trình trải qua ba vòng đánh giá công khai với sự tham gia của hàng trăm nhà nghiên cứu trên toàn thế giới, mỗi thuật toán đều được xem xét về mặt toán học, hiệu năng cũng như khả năng cài đặt an toàn.

Năm 2022, NIST thông báo kết thúc vòng tuyển chọn chính và công bố các thuật toán được chọn để chuẩn hóa. Trong đó, Kyber được chọn làm thuật toán trao đổi khóa chuẩn cho mục đích mã hóa thông tin công cộng, Dilithium được chọn cho chữ ký số tổng quát, Falcon cho các trường hợp cần chữ ký ngắn, và SPHINCS+ được duy trì như một phương án dự phòng dựa trên hàm băm. Đến năm 2024, NIST chính thức công bố các tiêu chuẩn FIPS 203, 204 và 205, trong đó Kyber được chuẩn hóa dưới tên gọi ML-KEM.

1.4.4 Cơ chế Key Encapsulation

Một điểm khác biệt quan trọng của Kyber và nhiều thuật toán hậu lượng tử khác so với RSA là chúng thường được thiết kế dưới dạng Key Encapsulation Mechanism thay vì mã hóa bất đối xứng trực tiếp. Một KEM là bộ ba thuật toán (KeyGen, Encaps, Decaps) trong đó:

- $\text{KeyGen}() \rightarrow (pk, sk)$: sinh cặp khóa công khai và bí mật.
- $\text{Encaps}(pk) \rightarrow (c, K)$: người gửi cung cấp khóa công khai của người nhận, nhận về một ciphertext c và một shared secret K .

- $\text{Decaps}(sk, c) \rightarrow K$: người nhận dùng khóa bí mật của mình và ciphertext vừa nhận để khôi phục cùng shared secret K .

Shared secret K sau đó được dùng làm khóa cho một thuật toán đối xứng như AES-GCM để mã hóa dữ liệu thực tế. Ưu điểm của cấu trúc KEM là đơn giản, có chứng minh bảo mật chặt chẽ theo mô hình IND-CCA2 và phù hợp với đặc điểm của nhiều bài toán lattice, nơi mà việc mã hóa trực tiếp một thông điệp tùy ý thường gặp nhiều hạn chế kỹ thuật.

1.5 Thuật toán Kyber và chuẩn ML-KEM

1.5.1 Không gian làm việc và các tham số

Kyber hoạt động trên vành đa thức

$$R_q = \mathbb{Z}_q[X]/(X^n + 1),$$

với các tham số cố định $n = 256$ và $q = 3329$. Mỗi phần tử của R_q là một đa thức dạng $a_0 + a_1X + a_2X^2 + \dots + a_{255}X^{255}$ trong đó các hệ số $a_i \in \mathbb{Z}_q = \{0, 1, \dots, 3328\}$. Phép cộng trong R_q đơn giản là cộng hệ số theo modulo q , còn phép nhân được thực hiện bằng cách nhân đa thức thông thường rồi rút gọn modulo $X^n + 1$. Do $X^n \equiv -1 \pmod{X^n + 1}$, phép rút gọn này chỉ đơn thuần là hoán vị dấu các hệ số bậc cao, khiến việc cài đặt rất hiệu quả.

Tham số $q = 3329$ là một số nguyên tố đặc biệt thỏa mãn $q \equiv 1 \pmod{2n}$, cho phép vành R_q tồn tại căn bậc $2n$ của đơn vị. Nhờ đó, phép nhân đa thức có thể được tăng tốc bằng biến đổi NTT, đưa độ phức tạp từ $O(n^2)$ xuống $O(n \log n)$, tương tự như vai trò của biến đổi Fourier rời rạc trong xử lý tín hiệu.

Các phiên bản Kyber khác nhau chỉ thay đổi tham số k , là số chiều của module làm việc. Cụ thể ma trận công khai $\mathbf{A}$ thuộc $R_q^{k \times k}$, còn các vector bí mật $\mathbf{s}$, nhiễu $\mathbf{e}$ và vector mã hóa thuộc R_q^k . Ba biến thể được sử dụng là:

- Kyber-512 với $k = 2$, tương ứng mức an toàn NIST Level 1 (tương đương AES-128).
- Kyber-768 với $k = 3$, tương ứng mức an toàn NIST Level 3 (tương đương AES-192).
- Kyber-1024 với $k = 4$, tương ứng mức an toàn NIST Level 5 (tương đương AES-256).

Trong đề án này, Kyber-768 được dùng làm ví dụ toán học vì đây là biến thể cân bằng và dễ trình bày.

Bảng sau tóm tắt đầu vào, đầu ra và vai trò của ba thao tác chính trong Kyber/ML-KEM. Đây cũng là giao diện mà ứng dụng dùng khi gọi thư viện liboqs, thay vì tự thao tác trực tiếp với đa thức bên trong thuật toán.

Thao tác	Đầu vào	Đầu ra	Vai trò trong hệ thống
KeyGen	Nguồn ngẫu nhiên an toàn và bộ tham số thuật toán	Khóa công khai pk và khóa bí mật sk	Người nhận công bố pk trong prekey bundle, còn sk chỉ lưu cục bộ trên client.
Encaps	Khóa công khai pk của người nhận	Ciphertext KEM c và shared secret K	Người gửi tạo một secret chung mà chỉ người giữ sk tương ứng mới khôi phục được.
Decaps	Khóa bí mật sk và ciphertext KEM c	Shared secret K hoặc một secret giả ngẫu nhiên nếu ciphertext không hợp lệ	Người nhận khôi phục secret chung để đưa vào HKDF và thiết lập khóa phiên.

Bảng 1.1: Đầu vào, đầu ra và vai trò của các thao tác Kyber/ML-KEM

1.5.2 Phân phối nhiễu tâm nhị thức

Các vector bí mật và nhiễu trong Kyber được lấy mẫu từ phân phối tâm nhị thức B_η , với η là tham số nhỏ. Một mẫu $x \leftarrow B_\eta$ được sinh bằng cách lấy 2η bit ngẫu nhiên độc lập $b_1, \dots, b_\eta, b'_1, \dots, b'_\eta$ và tính:

$$x = \sum_{i=1}^{\eta} (b_i - b'_i).$$

Giá trị x nằm trong khoảng $[-\eta, \eta]$ với phân phối đối xứng quanh 0. Với Kyber-768, tham số $\eta_1 = 2$ được dùng cho $\mathbf{s}$ và $\mathbf{e}$ trong KeyGen, còn $\eta_2 = 2$ được dùng cho nhiễu trong Encaps. Khi mở rộng phân phối từ một số nguyên sang một đa thức thuộc R_q , mỗi hệ số của đa thức được lấy mẫu độc lập từ B_η , cho nên một “đa thức nhỏ” trong R_q là một đa thức mà toàn bộ $n = 256$ hệ số đều có giá trị tuyệt đối nhỏ.

Việc chọn phân phối tâm nhị thức thay vì phân phối Gaussian rời rạc xuất phát từ nhu cầu cài đặt: B_η chỉ cần các phép xor và đếm bit, hoàn toàn tránh được các phép tính dấu phẩy động vốn nhạy cảm với tấn công kênh kề. Về mặt bảo mật, B_η cho độ khó Module-LWE gần như tương đương với Gaussian có phương sai tương ứng.

1.5.3 Sinh khóa

Thuật toán Kyber.KeyGen không nhận thông điệp của người dùng. Đầu vào thực tế của nó là nguồn ngẫu nhiên an toàn và bộ tham số như Kyber-768 hoặc Kyber-1024. Đầu ra là cặp khóa (pk, sk) , trong đó pk được gửi lên server để người khác bắt tay, còn sk phải được giữ bí mật ở client.

Các bước cốt lõi của quá trình sinh khóa như sau:

1. Sinh một seed $\rho \in \{0, 1\}^{256}$ và một seed $\sigma \in \{0, 1\}^{256}$ độc lập, ngẫu nhiên.
2. Mở rộng ρ thành ma trận ngẫu nhiên $\mathbf{A} \in R_q^{k \times k}$ bằng hàm XOF(ρ, i, j) (cụ thể là SHAKE-128), đảm bảo rằng từ cùng ρ ta luôn dựng lại được cùng $\mathbf{A}$.

3. Lấy mẫu vector bí mật $\mathbf{s} \in R_q^k$ với mỗi thành phần đa thức có các hệ số theo phân phối B_{η_1} , sử dụng seed σ kết hợp bộ đếm.

4. Lấy mẫu vector nhiễu $\mathbf{e} \in R_q^k$ cũng theo B_{η_1} với seed σ và bộ đếm tiếp theo.

5. Tính

$$\mathbf{t} = \mathbf{A} \cdot \mathbf{s} + \mathbf{e} \in R_q^k.$$

6. Khóa công khai là $pk = (\mathbf{t}, \rho)$. Chỉ ρ được lưu thay vì toàn bộ $\mathbf{A}$ giúp tiết kiệm không gian đáng kể.

7. Khóa bí mật là $sk = \mathbf{s}$ (trên thực tế, phiên bản IND-CCA2 gắn thêm pk , hash của pk và một giá trị ngẫu nhiên z vào sk để phục vụ biến đổi Fujisaki-Okamoto trình bày ở mục sau).

Về mặt toán học, $(\mathbf{A}, \mathbf{t})$ chính là một thể hiện của bài toán Module-LWE: từ $\mathbf{A}$ ngẫu nhiên và $\mathbf{t} = \mathbf{A} \cdot \mathbf{s} + \mathbf{e}$, việc tìm $\mathbf{s}$ tương đương giải Module-LWE, được giả định là khó ngay cả với máy tính lượng tử.

1.5.4 Encapsulate

Thuật toán Kyber.Encaps là thao tác phía người gửi. Đầu vào là khóa công khai $pk = (\mathbf{t}, \rho)$ của người nhận. Đầu ra gồm ciphertext KEM c để gửi qua server và shared secret K dài 256 bit để đưa vào quá trình dẫn xuất khóa.

Các bước cốt lõi gồm:

1. Sinh thông điệp ngẫu nhiên $m \in \{0, 1\}^{256}$.
2. Từ ρ dựng lại ma trận $\mathbf{A}$ giống hệt bên sinh khóa.
3. Lấy mẫu ba đại lượng ngẫu nhiên phụ thuộc m và pk :
 - Vector $\mathbf{r} \in R_q^k$ với các hệ số theo B_{η_1} .
 - Vector nhiễu $\mathbf{e}_1 \in R_q^k$ theo B_{η_2} .
 - Nhiễu scalar $e_2 \in R_q$ theo B_{η_2} .

Việc lấy mẫu là xác định theo m thông qua hàm băm, nên cùng một m luôn sinh ra cùng $(\mathbf{r}, \mathbf{e}_1, e_2)$. Đây là chi tiết quan trọng cho biến đổi Fujisaki-Okamoto.

4. Tính hai thành phần của ciphertext:

$$\mathbf{u} = \mathbf{A}^\top \cdot \mathbf{r} + \mathbf{e}_1 \in R_q^k,$$

$$v = \mathbf{t}^\top \cdot \mathbf{r} + e_2 + \left\lceil \frac{q}{2} \right\rceil \cdot m \in R_q,$$

trong đó $\lceil q/2 \rceil$ là phần nguyên gần nhất của $q/2$ (với $q = 3329$ thì $\lceil q/2 \rceil = 1665$), và $\lceil q/2 \rceil \cdot m$ có nghĩa là “gắn” thông điệp m vào hệ số của đa thức bằng cách ánh xạ bit 0 thành hệ số 0 và bit 1 thành hệ số 1665.

5. Nén ciphertext: cả $\mathbf{u}$ và v đều được nén bằng cách cắt bớt các bit thấp nhất trong mỗi hệ số, giảm kích thước truyền đi. Ký hiệu $c = (\text{Compress}(\mathbf{u}), \text{Compress}(v))$.
6. Dẫn xuất shared secret: $K = \text{KDF}(m \parallel \text{Hash}(c))$, với KDF là SHAKE-256. Đây chính là khóa cuối cùng mà AES-GCM sẽ dùng.

Trực giác toán học là như sau. Nếu bỏ qua nhiễu, ta có $\mathbf{t} = \mathbf{A} \cdot \mathbf{s}$ và $v \approx \mathbf{t}^\top \mathbf{r} + \lceil q/2 \rceil m = \mathbf{s}^\top \mathbf{A}^\top \mathbf{r} + \lceil q/2 \rceil m$. Phía giải mã biết $\mathbf{s}$, có thể tính $\mathbf{s}^\top \mathbf{u} \approx \mathbf{s}^\top \mathbf{A}^\top \mathbf{r}$, rồi lấy $v - \mathbf{s}^\top \mathbf{u}$ sẽ gần bằng $\lceil q/2 \rceil m$. Các nhiễu nhỏ bị tích lũy trong quá trình nhưng vẫn đủ nhỏ để phục hồi được m bằng cách làm tròn.

1.5.5 Decapsulate

Kyber.Decaps là thao tác phía người nhận. Đầu vào là khóa bí mật sk đang lưu cục bộ và ciphertext KEM $c = (\mathbf{u}, v)$ nhận từ người gửi. Đầu ra là shared secret K giống với phía Encaps nếu ciphertext hợp lệ. Nếu ciphertext bị sửa đổi hoặc không hợp lệ, triển khai chuẩn không trả lỗi trực tiếp mà tạo một secret giả ngẫu nhiên, giúp tránh để lộ thông tin qua phản hồi giải mã.

Các bước gồm:

1. Giải nén $\mathbf{u}$ và v về dạng đa thức.
2. Tính

$$w = v - \mathbf{s}^\top \cdot \mathbf{u} \in R_q.$$

3. Khôi phục thông điệp m' bằng cách làm tròn từng hệ số của w : hệ số gần 0 cho bit 0, hệ số gần $\lceil q/2 \rceil$ cho bit 1. Cụ thể, với mỗi hệ số $w_i \in \{0, \dots, q-1\}$ ta đặt

$$m'_i = \left\lfloor \frac{2 \cdot w_i}{q} \right\rfloor \bmod 2.$$

4. Chạy lại toàn bộ Encaps với m' để thu được c' và so sánh với ciphertext đầu vào. Nếu $c' = c$, trả về

$$K = \text{KDF}(m' \parallel \text{Hash}(c)).$$

Nếu không, trả về

$$K = \text{KDF}(z \parallel \text{Hash}(c)),$$

với z là giá trị ngẫu nhiên bí mật được gắn vào sk lúc sinh khóa. Kiểm tra so sánh này chính là cốt lõi của biến đổi Fujisaki-Okamoto, bảo đảm IND-CCA2.

1.5.6 Phân tích tính đúng đắn

Để thấy vì sao quá trình giải mã khôi phục lại được m , xét phép tính ở bước 2 của Decaps khi bỏ qua phần nén để đơn giản:

$$w = v - \mathbf{s}^\top \mathbf{u} = (\mathbf{t}^\top \mathbf{r} + e_2 + \lceil q/2 \rceil m) - \mathbf{s}^\top (\mathbf{A}^\top \mathbf{r} + \mathbf{e}_1).$$

Thay $\mathbf{t}^\top = (\mathbf{A}\mathbf{s} + \mathbf{e})^\top = \mathbf{s}^\top \mathbf{A}^\top + \mathbf{e}^\top$ ta được:

$$w = \mathbf{s}^\top \mathbf{A}^\top \mathbf{r} + \mathbf{e}^\top \mathbf{r} + e_2 + \lceil q/2 \rceil m - \mathbf{s}^\top \mathbf{A}^\top \mathbf{r} - \mathbf{s}^\top \mathbf{e}_1.$$

Rút gọn thành

$$w = \lceil q/2 \rceil \cdot m + \underbrace{\mathbf{e}^\top \mathbf{r} - \mathbf{s}^\top \mathbf{e}_1 + e_2}_{\text{sai số tổng hợp } \delta}.$$

Như vậy w bằng $\lceil q/2 \rceil m$ cộng một sai số δ . Do $\mathbf{e}, \mathbf{r}, \mathbf{s}, \mathbf{e}_1, e_2$ đều có hệ số nhỏ (theo phân phối B_η với $\eta \leq 2$), kỳ vọng và phương sai của từng hệ số của δ đều được giới hạn. Các tham số n, q, η_1, η_2 và sơ đồ nén được chọn sao cho với xác suất rất cao (cụ thể,

$\Pr[\text{giải mã sai}] < 2^{-164}$ với Kyber-768), mỗi hệ số của δ có độ lớn nhỏ hơn $q/4$. Khi đó, phép làm tròn ở bước 3 của Decaps khôi phục chính xác từng bit của m :

$$m'_i = \left\lfloor \frac{2w_i}{q} \right\rfloor = \left\lfloor \frac{2(\lceil q/2 \rceil m_i + \delta_i)}{q} \right\rfloor = m_i \bmod 2.$$

Đây chính là lý do Kyber vừa đảm bảo tính đúng đắn gần như hoàn hảo, vừa giữ được mức nhiễu đủ cho Module-LWE trở thành khó.

1.5.7 Biến đổi Fujisaki-Okamoto và bảo mật IND-CCA2

Kyber áp dụng biến đổi Fujisaki-Okamoto để nâng lên mức bảo mật IND-CCA2 - an toàn ngay cả khi kẻ tấn công có quyền truy cập tới một oracle giải mã.

Ý tưởng của biến đổi này là thay vì mã hóa ngẫu nhiên, các giá trị $(\mathbf{r}, \mathbf{e}_1, e_2)$ được dẫn xuất xác định từ chính thông điệp m và khóa công khai pk thông qua hàm băm:

$$(\mathbf{r}, \mathbf{e}_1, e_2) = G(m, pk),$$

với G là hàm băm SHAKE-256. Nhờ đó, người giải mã sau khi khôi phục m' có thể tự tính lại ciphertext c' một cách xác định và so sánh với c . Nếu $c' \neq c$, kết luận rằng ciphertext đã bị sửa đổi và trả về một khóa giả ngẫu nhiên (được dẫn xuất từ giá trị bí mật z) thay vì báo lỗi. Cơ chế “giả ngẫu nhiên thay vì báo lỗi” này được gọi là implicit rejection, giúp tránh tấn công kênh kẻ dạng time-based khi kẻ tấn công cố gắng phân biệt giữa ciphertext hợp lệ và không hợp lệ.

Kết quả là bộ ba (KeyGen, Encaps, Decaps) đầy đủ của Kyber đạt mức an toàn IND-CCA2 trong Random Oracle Model lượng tử, được chứng minh reduction tới độ khó Module-LWE. Đây chính là mức bảo mật mạnh nhất cần thiết cho một KEM được sử dụng thực tế, trong đó kẻ tấn công có thể vừa có khóa công khai, vừa chủ động gửi ciphertext bất kỳ để quan sát phản hồi.

1.5.8 Vì sao Kyber phù hợp với hệ thống nhắn tin

Đối với một ứng dụng nhắn tin, Kyber có ba điểm mạnh chính. Thứ nhất, thuật toán đã được NIST chuẩn hóa dưới tên ML-KEM trong FIPS 203, nên phù hợp hơn cho một hệ thống nghiêm túc so với việc tự chọn một đề xuất chưa ổn định. Thứ hai, đầu ra của Kyber là một shared secret ngắn, dễ đưa vào HKDF để kết hợp với X25519, Double Ratchet hoặc cơ chế MLS group. Thứ ba, thao tác encapsulate và decapsulate đủ nhanh để chạy trên client desktop mà không tạo độ trễ rõ rệt cho người dùng.

Kyber không dùng để mã hóa trực tiếp nội dung tin nhắn. Ciphertext KEM chỉ có nhiệm vụ vận chuyển một secret dùng cho bắt tay. Sau khi secret được tạo, nội dung tin nhắn vẫn nên được mã hóa bằng thuật toán đối xứng như AES-GCM. Cách tách vai trò này giúp hệ thống đơn giản hơn: KEM giải quyết bài toán thiết lập secret ban đầu, còn AEAD giải quyết bài toán mã hóa và xác thực từng message.

1.5.9 Các biến thể và mức an toàn

Kyber được cung cấp ở ba biến thể tương ứng với ba mức an toàn khác nhau. Kyber-512 với $k = 2$ cung cấp mức an toàn tương đương AES-128, phù hợp cho các ứng dụng hạn chế về tài nguyên. Kyber-768 với $k = 3$ cung cấp mức an toàn tương đương AES-192, là lựa chọn cân bằng giữa an toàn và hiệu năng, và được khuyến nghị cho đa số ứng dụng

thực tế. Kyber-1024 với $k = 4$ đạt mức an toàn tương đương AES-256 và phù hợp cho các ứng dụng có yêu cầu bảo mật cao nhất.

Về kích thước, Kyber-768 có khóa công khai 1184 byte, khóa bí mật 2400 byte, ciphertext 1088 byte và shared secret 32 byte. Các giá trị này lớn hơn đáng kể so với ECDH trên đường cong X25519 với khóa chỉ 32 byte, nhưng vẫn trong giới hạn có thể chấp nhận đối với một ứng dụng desktop hiện đại. Bên lại, thời gian thực thi của Kyber ở các thao tác keygen, encapsulate và decapsulate đều rất nhanh, thường chỉ trong khoảng vài trăm micro giây trên một CPU phổ thông.

1.6 Forward secrecy, Double Ratchet và MLS

1.6.1 Forward secrecy và post-compromise security

Forward secrecy là thuộc tính bảo đảm rằng việc lộ một khóa dài hạn ở thời điểm sau không tự động làm lộ các tin nhắn đã gửi trước đó. Trong nhắn tin, yêu cầu này quan trọng vì transcript có thể bị lưu lại rất lâu bởi mạng, server hoặc bản sao lưu. Nếu hệ thống dùng một khóa hội thoại cố định cho toàn bộ lịch sử, chỉ cần khóa đó bị lộ thì toàn bộ transcript cũ đều nguy hiểm.

Post-compromise security là thuộc tính mạnh hơn ở chiều ngược lại. Nếu attacker từng lấy được một phần trạng thái tạm thời trên thiết bị nhưng sau đó không còn kiểm soát thiết bị nữa, giao thức cần có khả năng tự phục hồi sau khi hai bên tiếp tục trao đổi secret mới. Hai thuộc tính này không đến từ một thuật toán đơn lẻ, mà đến từ cách tổ chức vòng đời khóa: khóa message chỉ dùng một lần, chain key được cập nhật liên tục, và các secret mới được đưa vào state theo thời gian.

1.6.2 Double Ratchet trong hội thoại hai người

Double Ratchet là cơ chế đạt forward secrecy ở mức từng tin nhắn cho hội thoại hai người, được phổ biến qua giao thức Signal. Sau bước bắt tay ban đầu, hai phía cùng có một *root key* bí mật chung. Toàn bộ phần còn lại của giao thức xoay quanh việc liên tục biến đổi root key này thành các khóa con dùng một lần, theo hai “bánh cóc” (ratchet) lồng nhau: một bánh cóc đối xứng chạy theo từng tin, và một bánh cóc Diffie-Hellman chạy theo từng lượt đổi chiều. Sở dĩ gọi là bánh cóc vì giống cơ cấu cơ khí chỉ quay được một chiều: trạng thái khóa chỉ tiến tới, không thể quay ngược lại để khôi phục các giá trị cũ.

Trạng thái phiên. Ở mỗi phía, mỗi cuộc hội thoại được mô tả bằng một *trạng thái phiên* (st) - là toàn bộ vật liệu khóa và bộ đếm cần để mã hóa tin kế tiếp và giải mã tin nhận về:

$$\text{st} = (RK, CK_s, CK_r, (pk_{\text{self}}, sk_{\text{self}}), pk_{\text{remote}}, N_s, N_r, PN, MK_{\text{skip}}).$$

Ý nghĩa và vai trò của từng thành phần:

- *RK* - *root key*, bí mật gốc của phiên (ban đầu do bắt tay PQXDH gieo). Nó chỉ thay đổi khi có bánh cóc DH nên đóng vai trò trục ổn định giữa các lượt đổi chiều, và là nguồn dẫn ra mọi chain key.
- *CK_s*, *CK_r* - hai *chain key* cho chiều gửi và chiều nhận, mỗi tin tiến một bước qua KDF_CK. Vì hai phía cùng một *RK* nhưng trao vai nên chain gửi của bên này đúng bằng chain nhận của bên kia.

- $(pk_{\text{self}}, sk_{\text{self}})$ - cặp khóa X25519 ratchet hiện hành của mình; khóa công khai pk_{self} được đính vào header mỗi tin gửi đi.
- pk_{remote} - khóa ratchet công khai mới nhất của đối phương. Khi header một tin đến mang khóa khác giá trị này, đó là tín hiệu để thực hiện một bước DH ratchet.
- N_s, N_r - số tin đã gửi trong chain gửi hiện tại, và số thứ tự tin mong đợi tiếp theo ở chain nhận.
- PN - (*previous N*) độ dài chain gửi trước lần xoay DH ratchet gần nhất. Nhờ PN đặt trong header, phía nhận biết phải sinh và lưu nốt bao nhiêu khóa của chain cũ trước khi chuyển sang chain mới, để các tin cũ đến muộn vẫn giải mã được.
- MK_{skip} - bảng message key của các tin bị bỏ qua (đến muộn hoặc lệch thứ tự), có giới hạn kích thước để chống bị làm cạn bộ nhớ.

Mọi khóa trong st đều dài 32 byte (root key, chain key, message key và khóa X25519). Cài đặt thực tế còn bổ sung vài trường để ràng buộc metadata (khóa mã hóa header, định danh phiên và hội thoại, số thứ tự định danh) cùng cơ chế chống nhận lặp, nhưng không làm thay đổi bản chất hai bánh cóc.

Bánh cóc đối xứng (symmetric-key ratchet). Từ root key, mỗi phía dẫn xuất hai *chain key*: một cho chiều gửi và một cho chiều nhận. Mỗi khi cần xử lý một tin, chain key được đưa qua một hàm dẫn xuất một chiều KDF_CK để tách ra đồng thời một *message key* dùng một lần và một chain key kế tiếp:

$$MK_i = \text{HMAC}(CK_i, 0x01), \quad CK_{i+1} = \text{HMAC}(CK_i, 0x02).$$

Message key MK_i chỉ dùng để mã hóa hoặc giải mã đúng một tin rồi bị xóa; chain key tiến từ CK_i sang CK_{i+1} và giá trị cũ cũng bị xóa ngay. Vì HMAC là một chiều, biết CK_{i+1} hay MK_i không cho phép suy ngược ra CK_i hoặc bất kỳ message key nào trước đó. Đây chính là nguồn gốc của forward secrecy ở mức tin nhắn: việc lộ trạng thái hiện tại không làm lộ các tin đã gửi trước.

Bánh cóc Diffie-Hellman (DH ratchet). Riêng bánh cóc đối xứng thì không tự phục hồi: nếu attacker lấy được chain key hiện tại, hẳn có thể tính mọi message key về sau trong cùng một chiều. Để vá điểm này, mỗi phía giữ thêm một cặp khóa DH tạm thời và đính kèm khóa công khai DH hiện hành của mình vào header mỗi tin. Khi một phía nhận được một khóa DH công khai mới của đối phương, nó thực hiện một bước DH ratchet: tính một secret DH mới rồi trộn vào root key để sinh ra root key và chain key mới qua hàm KDF_RK:

$$RK' \parallel CK' = \text{HKDF}(\text{salt} = RK, \text{IKM} = \text{DH}(sk_{\text{self}}, pk_{\text{remote}}), \text{info}, 64).$$

Đầu ra 64 byte được cắt làm đôi: 32 byte đầu là root key mới RK' , 32 byte sau là chain key mới CK' cho chiều vừa thiết lập; *info* là một nhãn cố định gắn với ứng dụng. Bên trong, HKDF gồm bước Extract (dùng RK làm salt để trộn DH secret thành một khóa giả ngẫu nhiên) rồi bước Expand (kéo dài thành 64 byte có cấu trúc). Sau đó phía này tự sinh một cặp khóa DH mới cho riêng mình và gửi kèm khóa công khai trong tin tiếp theo, để đối phương cũng thực hiện bước ratchet tương ứng. Nhờ liên tục bơm secret DH tươi vào root key, giao thức đạt post-compromise security: chỉ cần hai bên còn trao đổi được một lượt khóa DH mới mà attacker không quan sát, trạng thái sẽ “lạnh lại” và các tin sau đó an toàn trở lại.

Thuật toán gửi và nhận một tin. Khi *gửi*, phía gửi tiến chain key gửi để lấy message key, gắn header rồi mã hóa bằng AEAD với header làm dữ liệu xác thực kèm theo (AAD):

1. $(CK_s, MK) \leftarrow \text{KDF_CK}(CK_s)$ và tăng N_s ;
2. dựng header $H = (pk_{\text{self}}, PN, N_s)$;
3. $c = \text{AEAD.Enc}(MK; \text{plaintext}; \text{AAD} = H)$, rồi xóa ngay MK .

Khi nhận một tin có header $H' = (pk', PN', N')$: nếu pk' khác pk_{remote} đang lưu thì trước hết sinh và cắt các message key còn thiếu của chain cũ tới chỉ số PN' , rồi thực hiện một *bước DH ratchet* (tính RK, CK_r mới theo công thức trên, sinh cặp khóa DH mới và đặt lại $N_s = N_r = 0$). Tiếp đó phía nhận tiến chain nhận tới chỉ số N' , lưu các message key trung gian vào MK_{skip} , lấy ra MK rồi giải mã và kiểm tra tag AEAD trên cả header lẫn ciphertext. Việc đưa header vào AAD ràng buộc mỗi ciphertext với đúng khóa DH và chỉ số của nó, nên kẻ tấn công không thể trao header để đánh lừa bước ratchet.

Tin lịch thứ tự và chống lặp. Vì mạng có thể giao tin sai thứ tự, header còn mang hai bộ đếm: số thứ tự của tin trong chain hiện tại và số tin của chain trước đó. Khi nhận một tin có số thứ tự vượt trước, phía nhận lần lượt sinh và lưu tạm các message key bị bỏ qua (*skipped keys*) trong một giới hạn an toàn để vẫn giải mã được tin đến muộn, đồng thời từ chối ciphertext đã dùng nhằm chống replay. Trong đồ án, giới hạn này là 1000 khóa để cân bằng giữa khả năng chịu lệch thứ tự và rủi ro bị làm cạn bộ nhớ.

Liên hệ hậu lượng tử. Bản thân bánh cóc dùng X25519 nên không kháng lượng tử; tính kháng lượng tử của hội thoại nằm ở *bước bắt tay ban đầu*, nơi root key đầu tiên được gieo từ một bắt tay kiểu PQXDH có trộn thêm shared secret ML-KEM (trình bày ở chương thiết kế). Nói cách khác, ML-KEM bảo vệ bí mật của *điểm khởi đầu*, còn Double Ratchet bảo vệ *tiến trình* của hội thoại theo thời gian.

1.6.3 Messaging Layer Security cho hội thoại nhóm

Nếu áp dụng Double Ratchet riêng cho từng cặp thành viên trong một nhóm n người, mỗi tin phải mã hóa lại $n - 1$ lần và số phiên cần đồng bộ tăng theo $O(n^2)$; việc thêm/xóa thành viên và giữ đúng các thuộc tính bảo mật trở nên rất khó. Messaging Layer Security (MLS, RFC 9420) giải bài toán này bằng một *group state* chung, cho phép làm mới khóa nhóm với chi phí chỉ $O(\log n)$ cho mỗi thao tác.

Cây ratchet và TreeKEM. Trái tim của MLS là một cây nhị phân cân bằng gọi là *ratchet tree*: mỗi thành viên là một lá, mỗi nút (kể cả nút trong) gắn với một cặp khóa bất đối xứng. Bất biến cốt lõi của TreeKEM là: một thành viên biết khóa bí mật của *đúng* những nút nằm trên đường đi trực tiếp từ lá của mình lên gốc (*direct path*), và biết khóa công khai của mọi nút còn lại. Khóa bí mật của nút gốc do đó là một secret mà *toàn bộ* và *chỉ* các thành viên hiện tại mới cùng suy ra được.

Cây được đánh số theo thứ tự duyệt giữa (in-order): với n lá, cây có $2n - 1$ nút, lá ở các chỉ số chẵn còn nút trong ở các chỉ số lẻ; cha và hai con của một nút được xác định thuần túy bằng số học trên chỉ số, nên mọi thành viên cùng nhìn thấy một cấu trúc cây thống nhất mà không phải trao đổi thêm. *Direct path* của một lá là dãy nút từ cha trực tiếp lên gốc; *copath* là dãy các nút anh em của những nút đó (phần cây con ở phía đối diện).

Cập nhật đường đi (path update). Khi một thành viên làm mới khóa của mình (thao tác Update) hoặc khi có thay đổi thành viên, nó sinh một secret lá ngẫu nhiên s_0 rồi dẫn ra một chuỗi *path secret* dọc theo direct path; tại mỗi mức, path secret vừa sinh ra cặp khóa HPKE của nút đó, vừa dẫn tiếp lên mức cha:

$$s_i = \text{DeriveSecret}(s_{i-1}, \text{path}), \quad (pk_i, sk_i) = \text{DeriveKeyPair}(\text{DeriveSecret}(s_i, \text{node})),$$

trong đó $\text{DeriveSecret}(s, L) = \text{ExpandWithLabel}(s, L, \varepsilon, N_h)$ là HKDF-Expand có gắn nhãn L (độ dài đầu ra N_h byte). Để các thành viên khác cập nhật theo, với mỗi nút v trên direct path, người cập nhật mã hóa path secret tương ứng cho *tập độ phân giải* (resolution) của copath bằng HPKE:

$$\text{ct}_v = \text{HPKE.Enc}(pk_{\text{res}(v)}, s_{i(v)}).$$

Một thành viên khác giải mã được ct_v tại đúng điểm hai đường đi gặp nhau (nút tổ tiên chung thấp nhất), rồi tự dẫn tiếp các path secret còn lại lên gốc bằng chính công thức trên. Nhờ chỉ phải mã hóa cho $O(\log n)$ nút thay vì cho từng người, chi phí một lần cập nhật giảm từ tuyến tính xuống còn logarit theo số thành viên.

Lịch khóa và epoch. Mỗi cấu hình nhóm là một *epoch*. Sau một thao tác, secret rút ra từ gốc cây (*commit secret*) được trộn với init secret của epoch trước qua hai bước HKDF để tạo joiner secret rồi epoch secret:

$$\text{joiner}_n = \text{KDF.Extract}(\text{init}_{n-1}, \text{commit}_n),$$

$\text{epoch}_n = \text{ExpandWithLabel}(\text{KDF.Extract}(\text{joiner}_n, \text{psk}), \text{epoch}, \text{GroupContext}_n, N_h)$, trong đó psk là pre-shared key (bằng 0 nếu không dùng) và GroupContext_n buộc khóa vào đúng cấu hình nhóm ở epoch n . Từ epoch_n , mỗi secret chuyên biệt được dẫn ra bằng $\text{DeriveSecret}(\text{epoch}_n, L)$ với một nhãn L riêng, trong đó đáng chú ý là:

- **encryption** - gốc của *secret tree* sinh khóa mã hóa tin ứng dụng;
- **sender data** - bảo vệ metadata định danh người gửi của mỗi tin;
- **confirm** và **membership** - khóa xác thực transcript và tư cách thành viên trong handshake;
- **init** - chính là init_n , được nối làm đầu vào cho key schedule của epoch kế tiếp.

Việc init secret của epoch này nối tiếp vào epoch sau tạo thành một bánh cóc bất đối xứng ở quy mô nhóm: mỗi lần đổi epoch, toàn bộ khóa cũ bị bỏ (forward secrecy) và entropy mới từ commit secret được bơm vào (post-compromise security).

Cây secret và bánh cóc ứng dụng trong một epoch. Riêng việc đổi epoch chỉ cho forward secrecy ở mức thô. Để mỗi *tin* trong cùng một epoch cũng có khóa riêng, secret nhãn **encryption** được dùng làm gốc của một cây nhị phân thứ hai gọi là *secret tree*; mỗi lá ứng với một thành viên và cho ra một chuỗi ratchet đối xứng riêng. Tại thế hệ j , khóa và nonce AEAD cùng secret kế tiếp được dẫn ra bằng HKDF có nhãn:

- Khóa AEAD: $k_j = \text{ExpandWithLabel}(r_j, \text{key}, j, N_k)$;
- Nonce AEAD: $n_j = \text{ExpandWithLabel}(r_j, \text{nonce}, j, N_n)$;
- Secret kế tiếp: $r_{j+1} = \text{ExpandWithLabel}(r_j, \text{secret}, j, N_h)$.

Mỗi tin dùng một cặp (k_j, n_j) riêng rồi r_j tiến tới r_{j+1} và bị xóa, cho forward secrecy theo từng tin ngay trong một epoch. Như vậy MLS đạt cả hai thuộc tính theo hai trục bổ sung nhau: bánh cóc đối xứng trong secret tree lo forward secrecy giữa các tin, còn bánh cóc bất đối xứng qua các epoch (path update) lo post-compromise security ở mức nhóm.

Đề xuất, Commit và Welcome. Thay đổi nhóm được biểu diễn bằng các *proposal*: Add (thêm thành viên qua KeyPackage của họ), Remove (xóa thành viên), Update (làm mới khóa của một thành viên). Một *Commit* gói một tập proposal lại, thực hiện path

update và đẩy nhóm sang epoch mới. Thành viên mới nhận một thông điệp *Welcome* chứa đủ secret (mã hóa riêng cho KeyPackage của họ) để bước vào đúng epoch; thành viên bị Remove không nằm trong tập nhận secret của epoch kế tiếp nên lập tức mất khả năng giải mã.

Tách public state và secret state. MLS phân biệt rõ phần công khai và phần bí mật của group state. Các phần public như GroupInfo, cấu trúc ratchet tree (chỉ gồm khóa công khai), epoch và danh sách credential cho phép kiểm tra tính nối tiếp và hợp lệ của nhóm mà không lộ application secret; các secret như epoch secret và path secret chỉ tồn tại ở client. Nhờ vậy một server không tin cậy vẫn có thể lưu trữ, chuyển tiếp và *kiểm tra ràng buộc membership* dựa trên public state, nhưng không bao giờ giải mã được tin nhắn nhóm.

1.7 Giao thức TLS và vai trò trong kiến trúc bảo mật

Mặc dù mã hóa đầu cuối đã bảo vệ nội dung tin nhắn khỏi server, TLS ở tầng vận chuyển vẫn có ý nghĩa quan trọng trong một kiến trúc bảo mật nhiều lớp. TLS che nội dung gói JSON khỏi người nghe lén trên mạng và giúp xác thực máy chủ ở cấp độ hạ tầng. Tuy nhiên TLS không che hoàn toàn metadata như địa chỉ kết nối, thời điểm truyền và kích thước gói quan sát được trên mạng. Khi kết hợp mã hóa đầu cuối với TLS, hệ thống có hai lớp phòng vệ khác nhau thay vì chỉ phụ thuộc vào một lớp.

Trong đồ án, tầng TLS giữa client và server được thiết lập với chứng chỉ X.509. Toàn bộ các gói JSON được gửi trong kênh TLS này, bao gồm cả những gói đã chứa ciphertext đầu cuối. Nhờ đó, kẻ tấn công trên đường truyền không đọc được nội dung JSON. Dù vậy, server vẫn là điểm cuối của TLS, nên nội dung chat vẫn phải được mã hóa thêm ở tầng ứng dụng bằng E2EE.

1.8 Các công nghệ triển khai

1.8.1 Ngôn ngữ Python và PyQt5

Python là ngôn ngữ được lựa chọn để hiện thực hóa phía client nhờ vào sự kết hợp giữa cú pháp rõ ràng, hệ sinh thái thư viện phong phú và khả năng tích hợp tốt với các thành phần mật mã. Đối với phần giao diện, PyQt5 là framework trưởng thành và ổn định, cung cấp binding đầy đủ của thư viện Qt cho Python, cho phép xây dựng giao diện desktop native trên cả Windows, Linux và macOS từ cùng một mã nguồn.

PyQt5 hỗ trợ kiến trúc signal-slot, rất phù hợp cho các ứng dụng mạng bất đồng bộ, nơi các sự kiện từ luồng mạng cần được truyền về luồng giao diện một cách an toàn. Framework cũng cung cấp nhiều thành phần trực quan sẵn có như QScrollArea, QWidget, QListWidget, QFileDialog và nhiều widget khác để xây dựng message list, danh sách liên lạc, chọn file và các dialog chức năng.

1.8.2 Ngôn ngữ C và thư viện hệ thống

P phía máy chủ được triển khai bằng ngôn ngữ C để đảm bảo hiệu năng cao, mức sử dụng tài nguyên thấp và khả năng kiểm soát bộ nhớ chi tiết. Một máy chủ nhắn tin phải xử lý đồng thời nhiều kết nối mạng, mỗi kết nối có thể truyền tải các gói tin kích thước rất khác nhau từ tin nhắn văn bản ngắn vài byte cho đến file đa phương tiện nhiều megabyte. C cho phép quản lý bộ nhớ động một cách chính xác, tránh được chi phí của garbage collector và tận dụng tối đa các hệ thống lập lịch luồng của hệ điều hành.

Server trong đồ án sử dụng các thư viện hệ thống chuẩn của Linux bao gồm pthread cho mô hình đa luồng, socket BSD cho lập trình mạng, và OpenSSL cho tầng TLS. Ngoài ra, MySQL Connector/C được sử dụng để kết nối tới cơ sở dữ liệu, và một parser JSON nhẹ được viết thủ công để phân tích các gói tin mà không phụ thuộc vào thư viện bên ngoài nặng nề.

1.8.3 Thư viện Open Quantum Safe

Để tích hợp ML-KEM hoặc Kyber vào ứng dụng, đồ án sử dụng thư viện Open Quantum Safe, một dự án mã nguồn mở cung cấp hiện thực hóa các thuật toán mật mã hậu lượng tử đang được chuẩn hóa hoặc trong quá trình đánh giá. Thư viện lõi liboqs được viết bằng C với các tối ưu hóa cho nhiều kiến trúc CPU khác nhau, đồng thời cung cấp binding cho các ngôn ngữ phổ biến bao gồm Python, C++, Java và Go.

Trong đồ án, binding Python oqs-python được sử dụng ở phía client để thực hiện các thao tác KEM cần thiết như sinh khóa, encapsulate và decapsulate. Việc sử dụng một thư viện đã được kiểm định và duy trì tích cực là điều quan trọng, bởi mật mã hậu lượng tử có nhiều chi tiết cài đặt tinh tế mà việc tự viết lại từ đầu có thể dẫn đến các lỗi bảo mật khó phát hiện.

1.8.4 Thư viện cryptography và các thao tác AES-GCM

Đối với mã hóa đối xứng AES-GCM và hàm dẫn xuất khóa PBKDF2, thư viện cryptography của Python được lựa chọn. Đây là thư viện mật mã chính thức được cộng đồng Python Cryptographic Authority phát triển, cung cấp cả giao diện cấp cao để sử dụng và giao diện cấp thấp cho người cần kiểm soát chi tiết. Thư viện được xây dựng trên nền OpenSSL với các ràng buộc CFFI, đảm bảo hiệu năng gần bằng cài đặt C thuần và tự động hưởng lợi từ các tăng tốc phần cứng như AES-NI trên các CPU hỗ trợ.

1.8.5 Cơ sở dữ liệu MySQL

MySQL được chọn làm cơ sở dữ liệu quan hệ do tính phổ biến, tài liệu phong phú và hiệu năng tốt trong các tác vụ ghi-đọc nhiều. Ứng dụng nhắn tin có mô hình dữ liệu tương đối đơn giản với các thực thể như người dùng, bạn bè, hội thoại và tin nhắn, rất phù hợp với mô hình quan hệ. MySQL cũng hỗ trợ kiểu dữ liệu LONGTEXT lên tới 4 GB cho mỗi bản ghi, rất có ích khi lưu trữ tin nhắn chứa file đã được mã hóa và mã hóa base64 với độ dài có thể lên tới vài chục megabyte.

1.8.6 Docker và Docker Compose

Để đơn giản hóa việc triển khai và đảm bảo môi trường chạy nhất quán giữa các máy phát triển khác nhau, đồ án sử dụng Docker làm công cụ container hóa cùng Docker Compose để điều phối nhiều container. Server C và MySQL được đóng gói thành hai container riêng biệt với các volume lưu trữ bền vững cho dữ liệu cơ sở dữ liệu. Với một lệnh docker compose up duy nhất, toàn bộ môi trường máy chủ có thể được khởi động, tạo điều kiện thuận lợi cho việc phát triển, kiểm tra nội bộ và trình diễn.

Chương 2

Phân tích và thiết kế hệ thống

2.1 Khảo sát thực tế

2.1.1 Thực trạng các giao thức nhắn tin mã hóa đầu cuối

Sự phổ biến của các ứng dụng nhắn tin trực tuyến đã kéo theo nhu cầu ngày càng cao về bảo vệ quyền riêng tư và tính bảo mật của dữ liệu trao đổi. Để đáp ứng yêu cầu đó, mã hóa đầu cuối (End-to-End Encryption - E2EE) đã trở thành cơ chế bảo mật cốt lõi trong nhiều nền tảng nhắn tin hiện đại. Với E2EE, nội dung tin nhắn được mã hóa tại thiết bị người gửi và chỉ được giải mã tại thiết bị người nhận, giúp ngăn chặn việc đọc trộm dữ liệu bởi máy chủ trung gian hoặc các bên thứ ba.

Hiện nay, nhiều ứng dụng nhắn tin phổ biến đã triển khai E2EE dưới các hình thức khác nhau. Trong đó, Signal được xem là một trong những nền tảng tiên phong với Signal Protocol, giao thức đã trở thành cơ sở cho cơ chế bảo mật của WhatsApp, Google Messages và nhiều hệ thống liên lạc khác. Các giao thức này không chỉ cung cấp tính bí mật của nội dung trao đổi mà còn hỗ trợ các thuộc tính bảo mật quan trọng như xác thực người dùng, bí mật chuyển tiếp (Forward Secrecy) và bí mật hậu thỏa hiệp (Post-Compromise Security).

Bên cạnh việc bảo vệ nội dung tin nhắn, các giao thức hiện đại còn phải giải quyết nhiều yêu cầu phức tạp hơn như đồng bộ trên nhiều thiết bị, hỗ trợ người dùng ngoại tuyến, quản lý khóa phiên động và đảm bảo hiệu năng khi số lượng người dùng tăng cao. Điều này khiến thiết kế giao thức nhắn tin trở thành một bài toán tổng hợp giữa mật mã học, kiến trúc hệ thống và trải nghiệm người dùng.

2.1.2 Các cơ chế bảo mật được sử dụng trong hệ thống nhắn tin hiện nay

Phần lớn các hệ thống E2EE hiện nay áp dụng mô hình mật mã lai (Hybrid Cryptography), trong đó mật mã khóa công khai được sử dụng để thiết lập khóa phiên và mật mã đối xứng được sử dụng để mã hóa dữ liệu thực tế.

Trong Signal Protocol, quá trình thiết lập khóa ban đầu được thực hiện thông qua Extended Triple Diffie-Hellman (X3DH), cho phép hai người dùng tạo khóa chung ngay cả khi một bên đang ngoại tuyến. Sau khi khóa ban đầu được thiết lập, cơ chế Double Ratchet được sử dụng để liên tục cập nhật khóa phiên sau mỗi lần trao đổi tin nhắn. Nhờ đó, việc lộ khóa ở một thời điểm không làm ảnh hưởng đến toàn bộ lịch sử liên lạc.

Mô hình này đã chứng minh được hiệu quả trong thực tế và hiện được xem là tiêu chuẩn tham chiếu cho việc xây dựng các ứng dụng nhắn tin bảo mật. Tuy nhiên, nền tảng

trao đổi khóa của các giao thức này vẫn chủ yếu dựa trên mật mã đường cong elliptic, vốn được thiết kế cho môi trường tính toán cổ điển.

2.1.3 Những thách thức trong thiết kế giao thức nhắn tin thế hệ mới

Mặc dù các giao thức hiện nay đã đạt được mức độ bảo mật cao, việc thiết kế một hệ thống nhắn tin an toàn vẫn còn nhiều thách thức.

Thứ nhất, hệ thống phải duy trì các thuộc tính bảo mật cốt lõi như tính bí mật, tính toàn vẹn, xác thực, bí mật chuyển tiếp và khả năng phục hồi sau khi khóa bị lộ. Đây là những yêu cầu bắt buộc đối với các ứng dụng nhắn tin hiện đại.

Thứ hai, giao thức cần đảm bảo khả năng hoạt động hiệu quả trong môi trường thực tế với độ trễ thấp, tiêu tốn ít tài nguyên và có khả năng mở rộng cho số lượng lớn người dùng.

Thứ ba, sự phát triển của điện toán lượng tử đặt ra yêu cầu phải xem xét lại các cơ chế trao đổi khóa đang được sử dụng. Mặc dù các giao thức hiện tại vẫn được đánh giá là an toàn trước các phương thức tấn công truyền thống, việc chuẩn bị cho quá trình chuyển đổi sang các thuật toán kháng lượng tử đang trở thành xu hướng nghiên cứu và phát triển quan trọng trong lĩnh vực bảo mật truyền thông.

2.1.4 Xu hướng tích hợp mật mã hậu lượng tử vào hệ thống nhắn tin

Sau khi các tiêu chuẩn mật mã hậu lượng tử được công bố, nhiều tổ chức nghiên cứu và doanh nghiệp công nghệ đã bắt đầu thử nghiệm việc tích hợp các thuật toán hậu lượng tử vào các giao thức truyền thông hiện có. Thay vì thay thế hoàn toàn các cơ chế hiện hữu, phần lớn giải pháp được đề xuất theo hướng hybrid, kết hợp đồng thời thuật toán truyền thống và thuật toán hậu lượng tử trong quá trình trao đổi khóa.

Cách tiếp cận này cho phép hệ thống duy trì khả năng tương thích với các nền tảng hiện tại trong khi từng bước nâng cao khả năng chống chịu trước các mối đe dọa trong tương lai. Tuy nhiên, việc tích hợp mật mã hậu lượng tử cũng làm phát sinh nhiều vấn đề cần nghiên cứu như kích thước khóa lớn hơn, chi phí truyền thông tăng lên và ảnh hưởng đến hiệu năng của giao thức.

Từ thực trạng trên có thể thấy rằng việc xây dựng một giao thức nhắn tin mã hóa đầu cuối kết hợp các cơ chế bảo mật hiện đại với giải pháp trao đổi khóa kháng lượng tử là hướng nghiên cứu có ý nghĩa cả về mặt học thuật lẫn thực tiễn.

2.2 Phân tích yêu cầu hệ thống

2.2.1 Mục tiêu chức năng và phi chức năng

Yêu cầu chức năng

- **Quản lý tài khoản:** đăng ký, đăng nhập, đổi mật khẩu, quản lý hồ sơ cá nhân, thiết lập quyền riêng tư và chính sách cache cục bộ.
- **Quản lý quan hệ bạn bè:** tìm kiếm người dùng, gửi lời mời kết bạn, chấp nhận, từ chối, xóa bạn và chặn người dùng.

- **Nhắn tin trực tiếp E2EE:** thiết lập phiên liên lạc an toàn, gửi và nhận tin nhắn được bảo vệ bằng mã hóa đầu cuối.
- **Quản lý tin nhắn:** hỗ trợ trả lời, chỉnh sửa, ghim, chuyển tiếp, xóa và thả cảm xúc cho tin nhắn.
- **Tùy chọn hội thoại:** tắt thông báo, lưu trữ, ghim hội thoại và đánh dấu chưa đọc.
- **Gửi và nhận tệp tin:** hỗ trợ chia sẻ tệp đính kèm trong các cuộc trò chuyện, preview ảnh.
- **Hội thoại nhóm MLS:** tạo nhóm, quản lý thành viên và trao đổi tin nhắn nhóm bằng giao thức MLS.
- **Kiểm tra danh tính và bảo mật:** xác minh identity key qua safety number, xoay khóa an toàn (rotate identity) và theo dõi trạng thái bảo mật của cuộc trò chuyện và nhóm.
- **Sao lưu và khôi phục E2EE:** xuất và phục hồi dữ liệu mã hóa trên thiết bị khác.

Yêu cầu phi chức năng

- **Giao diện người dùng:** giao diện trực quan, dễ sử dụng đối với người dùng desktop.
- **Hiệu năng:** phản hồi nhanh khi gửi tin nhắn, tải lịch sử và chuyển đổi hội thoại.
- **Bảo mật:** đảm bảo tính bí mật, toàn vẹn và xác thực của dữ liệu trao đổi với các thuộc tính: kháng lượng tử, forward secrecy, server không đọc được plaintext.
- **Khả năng mở rộng:** dễ dàng bổ sung chức năng hoặc nâng cấp hệ thống trong tương lai.
- **Tính ổn định:** hệ thống hoạt động ổn định và có khả năng phục hồi khi xảy ra lỗi.
- **Tính khả dụng:** có health/readiness check và metrics để giám sát trạng thái vận hành.
- **Tương thích đa nền tảng:** client PyQt chạy trên Windows, Linux và macOS từ cùng một mã nguồn.

2.2.2 Đối tượng sử dụng và kịch bản sử dụng chính

Tác nhân	Loại	Vai trò
Người dùng	Chính	Đăng ký, đăng nhập, kết bạn, nhắn tin, tạo group, kiểm tra identity, quản lý backup và quyền riêng tư.
Quản trị vận hành	Phụ	Khởi động Docker, quan sát health, metrics, log, reset dữ liệu demo và kiểm tra trạng thái server.
Auditor	Phụ	Dựng checkpoint từ identity log và cung cấp proof cho client.

Bảng 2.1: Actor và vai trò

2.2.3 Danh sách Use Case mức hệ thống

Ở mức hệ thống, các chức năng của SecChat được chia thành sáu nhóm Use Case, gắn với ba tác nhân đã nêu ở bảng 2.1.

- **Nhóm A: Quản lý tài khoản và danh tính**

Các chức năng khởi tạo tài khoản, xác thực và quản lý danh tính bảo mật của người dùng.

- **Đăng ký (UC01):** tạo tài khoản bằng username, email và mật khẩu; client khởi tạo state E2EE và công bố khóa lần đầu.
- **Đăng nhập và công bố khóa (UC02):** xác thực tài khoản, mở local state và công bố PQXDH bundle cùng MLS KeyPackage X-Wing.
- **Quản lý hồ sơ và quyền riêng tư (UC03):** cập nhật display name, avatar, trạng thái, đổi mật khẩu, thiết lập quyền riêng tư và chính sách cache cục bộ.

- **Nhóm B: Quan hệ người dùng**

Các chức năng thiết lập và kiểm soát quan hệ giữa những người dùng.

- **Kết bạn và chặn (UC04):** tìm kiếm người dùng, gửi và phản hồi lời mời kết bạn, xóa bạn, đồng thời chặn hoặc bỏ chặn người dùng khác.

- **Nhóm C: Nhắn tin trực tiếp**

Các chức năng trao đổi tin nhắn một-một được mã hóa đầu cuối hậu lượng tử.

- **Mở DM và gửi tin (UC05):** lấy bundle của peer, chạy bắt tay kiểu PQXDH, gửi tin đầu tiên S3PQI rồi chuyển sang S3DR theo Double Ratchet.
- **Thao tác tin nhắn (UC06):** các thao tác trên một tin nhắn đã gửi, gồm:
 - * *Trả lời và chuyển tiếp* một tin nhắn.
 - * *Chỉnh sửa và xóa mềm* tin nhắn của chính mình.
 - * *Ghim và thả cảm xúc* cho tin nhắn.
- **Gửi và nhận tệp tin (UC07):** đính kèm và mã hóa tệp trong hội thoại, hỗ trợ preview ảnh.
- **Review identity (UC08):** hiển thị safety number hoặc short code để người dùng so sánh qua kênh ngoài và đánh dấu đã xác minh danh tính của peer.

- **Nhóm D: Hội thoại nhóm hậu lượng tử**

Các chức năng nhóm dựa trên MLS (RFC 9420) với ciphersuite X-Wing hybrid.

- **Tạo group MLS PQ (UC09):** tạo group state, claim KeyPackage X-Wing và gửi các artifact khởi tạo để server validate.
- **Quản lý thành viên và vai trò (UC10):** admin thêm/xóa thành viên, đổi role, rời nhóm và cập nhật thông tin nhóm thông qua quy trình `prepare/apply`.
- **Nhắn tin nhóm và system event (UC11):** gửi tin nhóm S3MLS; các thay đổi membership được hiển thị như system event trong dòng thời gian.

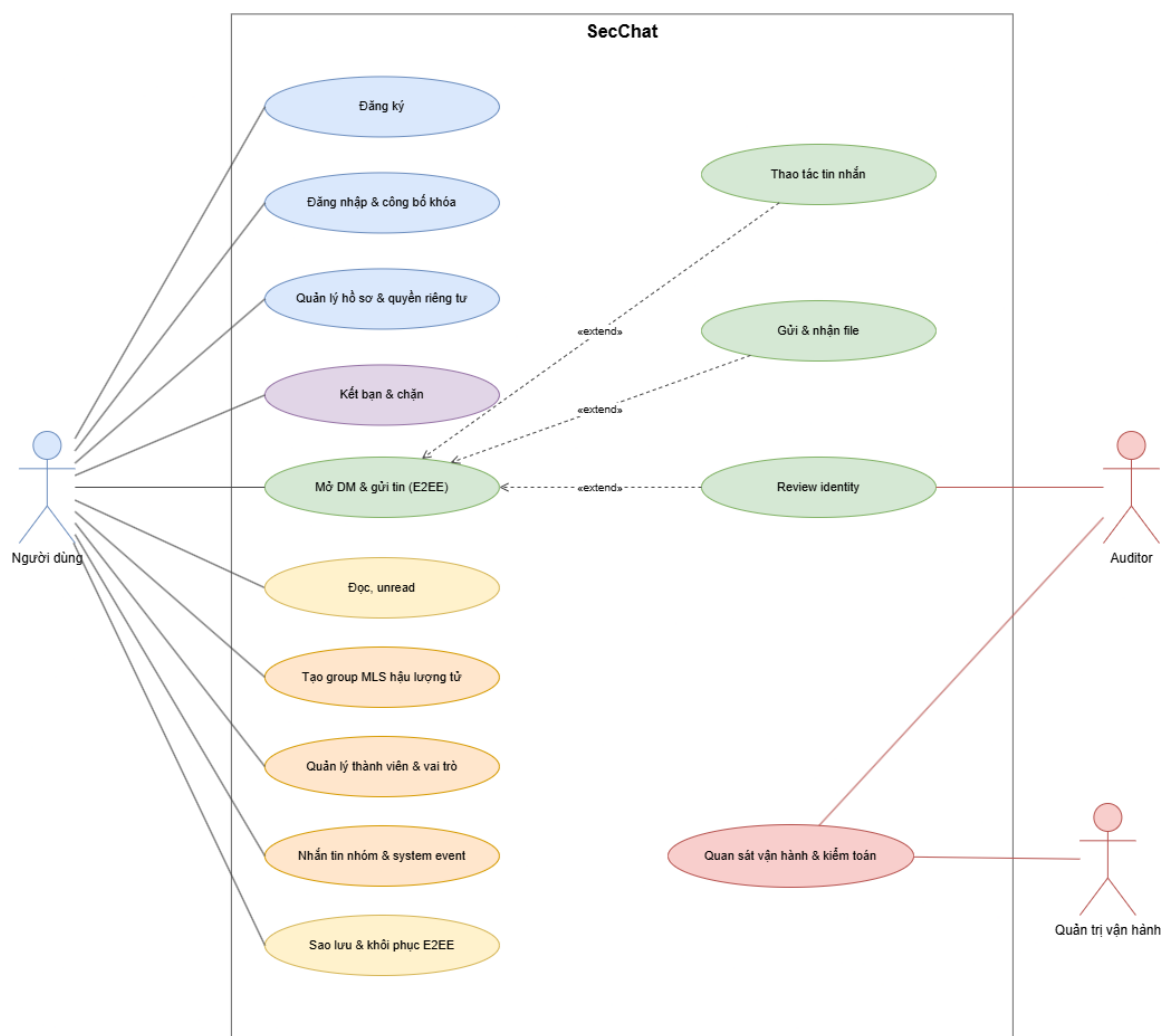
- **Nhóm E: Lịch sử và sao lưu**

Các chức năng đọc lại lịch sử và bảo toàn dữ liệu E2EE khi đổi thiết bị.

- **Đọc, unread(UC12):** mở hội thoại, đánh dấu đã đọc theo `message_id`.

- **Sao lưu và khôi phục E2EE (UC13):** export local state và cache đã mã hóa bằng passphrase riêng; khôi phục và công bố lại bundle trên thiết bị khác.
- **Nhóm F: Vận hành và kiểm toán**
Các chức năng phục vụ vận hành hệ thống và minh bạch danh tính, do các tác nhân phụ thực hiện.
 - **Quan sát vận hành và kiểm toán (UC14):** quản trị vận hành kiểm tra /healthz, /readyz, metrics và log qua control plane và server GUI; auditor dựng checkpoint từ identity_key_log và cấp Merkle proof cho client kiểm tra bundle.

2.2.4 Sơ đồ Usecase tổng quan



Hình 2.1: Sơ đồ use case tổng quan của SecChat

Các tác nhân tham gia hệ thống gồm:

- **Người dùng:** tác nhân chính của hệ thống. Người dùng thực hiện đăng ký, đăng nhập, kết bạn và chặn, nhắn tin trực tiếp và nhắn tin nhóm được mã hóa đầu cuối, thao tác trên tin nhắn, gửi file, kiểm tra danh tính, quản lý hồ sơ, quyền riêng tư và sao lưu hoặc khôi phục dữ liệu E2EE.

- **Auditor:** dịch vụ minh bạch khóa định danh, hoạt động độc lập với chat server. Auditor đọc log định danh, dựng cây Merkle, ký checkpoint và cấp inclusion proof để client kiểm tra khóa định danh của người khác trước khi tin tưởng.
- **Quản trị vận hành:** người triển khai và giám sát hệ thống qua Docker. Tác nhân này kiểm tra health, readiness, metrics và log qua control plane cùng server GUI, nhưng không có khả năng đọc nội dung tin nhắn của người dùng.

2.2.5 Đặc tả chức năng

Phần này tóm tắt toàn bộ use case mức hệ thống ở bảng 2.2, sau đó đặc tả chi tiết cho bốn use case cốt lõi và phức tạp nhất: *đăng nhập và công bố khóa* (UC02), *mở DM và gửi tin* (UC05), *quản lý thành viên group MLS* (UC10) và *sao lưu, khôi phục E2EE* (UC13). Các use case còn lại dùng chung cơ chế client gửi lệnh, server kiểm quyền rồi trả event.

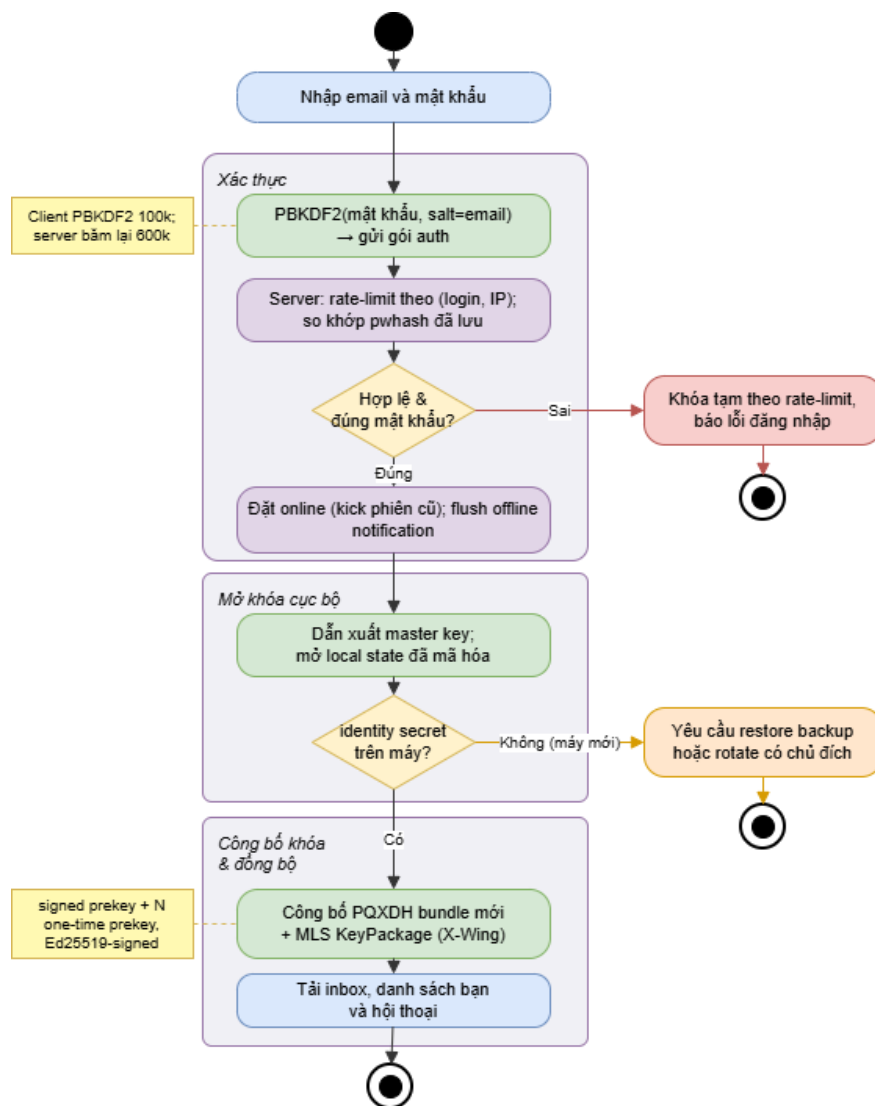
Mã	Use case	Tác nhân	Tóm tắt
UC01	Đăng ký	Người dùng	Tạo tài khoản, khởi tạo state E2EE và công bố khóa lần đầu.
UC02 (*)	Đăng nhập và công bố khóa	Người dùng, Auditor	Xác thực, mở local state, công bố PQXDH bundle và MLS KeyPackage.
UC03	Quản lý hồ sơ và quyền riêng tư	Người dùng	Cập nhật hồ sơ, avatar, đổi mật khẩu, quyền riêng tư và cache.
UC04	Kết bạn và chặn	Người dùng	Tìm, mời, chấp nhận/từ chối kết bạn; chặn/bỏ chặn.
UC05 (*)	Mở DM và gửi tin	Người dùng, Auditor	Bắt tay PQXDH, gửi S3PQI rồi S3DR; server chỉ lưu ciphertext.
UC06	Thao tác tin nhắn	Người dùng	Trả lời, chỉnh sửa, xóa, ghim, thả cảm xúc, chuyển tiếp.
UC07	Gửi và nhận file	Người dùng	Mã hóa và gửi tệp đính kèm trong lớp E2EE.
UC08	Review identity	Người dùng, Auditor	So sánh safety number qua kênh ngoài, đánh dấu đã xác minh.
UC09	Tạo group MLS hậu lượng tử	Người dùng, Server	Tạo group X-Wing, claim KeyPackage; server validate public state.
UC10 (*)	Quản lý thành viên và vai trò	Admin group	prepare/apply: thêm/xóa/đổi role; server validate PublicGroup .
UC11	Nhắn tin nhóm và system event	Thành viên group	Gửi S3MLS ; thay đổi membership hiển thị như system event.
UC12	Đọc, unread	Người dùng	mark_read theo message id.
UC13 (*)	Sao lưu và khôi phục E2EE	Người dùng	Export/restore state mã hóa passphrase; không rollback identity.
UC14	Quan sát vận hành và kiểm toán	Quản trị vận hành, Auditor	Kiểm tra health/metrics/log; auditor cấp Merkle proof.

Bảng 2.2: Tổng hợp các use case mức hệ thống ((*): có đặc tả chi tiết bên dưới)

2.2.5.1 UC02 - Đăng nhập và công bố khóa

Bảng 2.3: Đặc tả Use Case: Đăng nhập và công bố khóa (UC02)

Tên Use Case	Đăng nhập và công bố khóa
Tác nhân	Người dùng (chính), Auditor (phụ)
Tiền điều kiện	• Đã có tài khoản. • Máy hiện tại có local state E2EE, hoặc có backup/identity secret nếu là máy mới.
Luồng sự kiện chính	1. Người dùng nhập email và mật khẩu. 2. Client băm mật khẩu (PBKDF2, salt là email) và gửi gói auth. 3. Server rate-limit theo (login, IP), xác minh PBKDF2 và đặt trạng thái online. 4. Client dẫn xuất master key và mở local state đã mã hóa. 5. Client công bố PQXDH bundle mới cho máy hiện tại và MLS KeyPackage X-Wing. 6. Server flush notification offline; client tải inbox, danh sách bạn và hội thoại.
Luồng sự kiện phát sinh	• Sai mật khẩu hoặc vượt số lần cho phép: bị khóa tạm thời theo rate limit và báo lỗi. • Máy mới thiếu identity secret: client yêu cầu restore backup hoặc rotate có chủ đích; không tự rotate âm thầm.
Hậu điều kiện	• Người dùng online; bundle của máy hiện tại đã được công bố; sẵn sàng gửi và nhận tin.



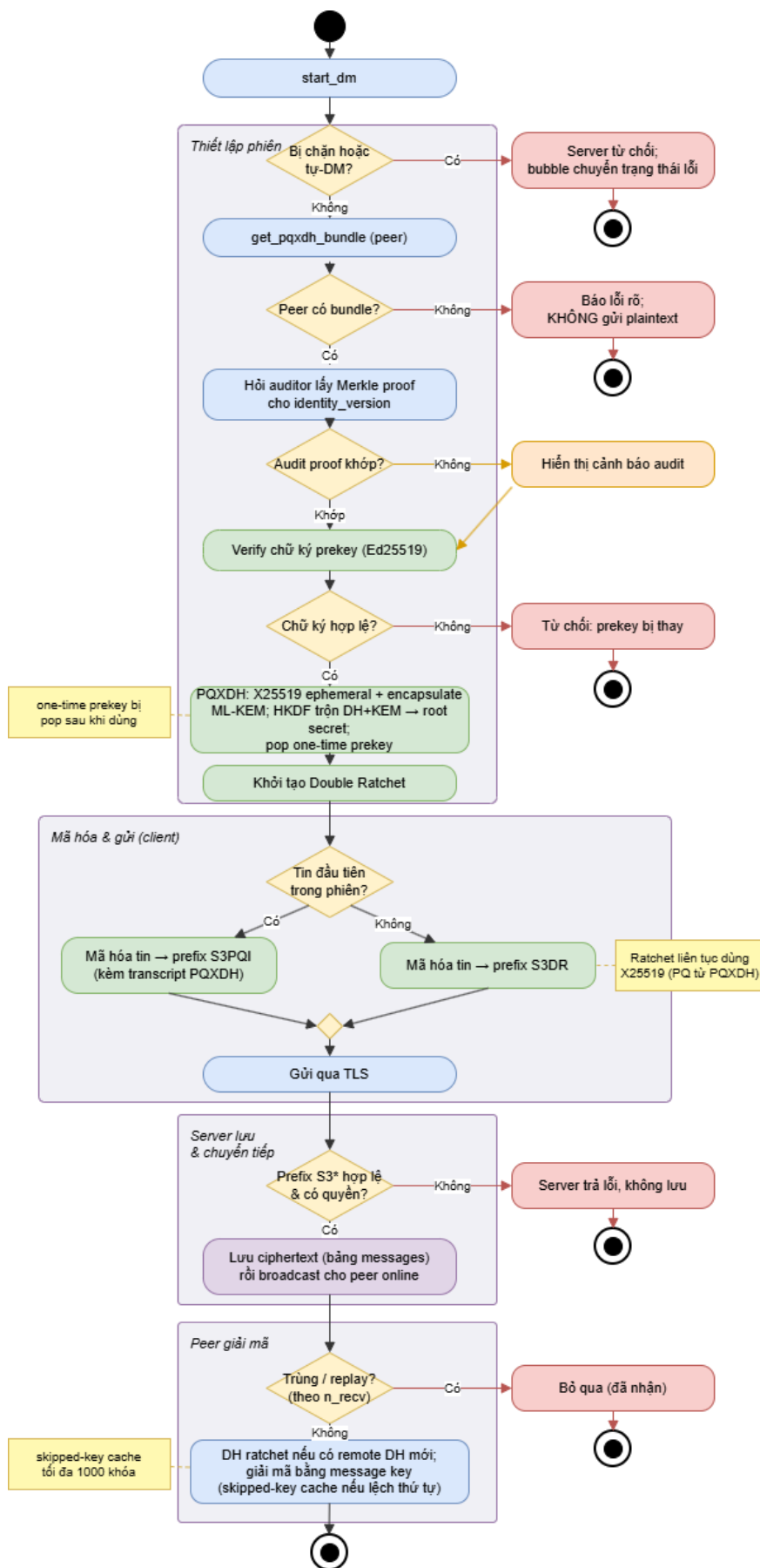
Hình 2.2: Sơ đồ hoạt động: Đăng nhập và công bố khóa (UC02)

2.2.5.2 UC05 - Mở DM và gửi tin

Bảng 2.4: Đặc tả Use Case: Mở DM và gửi tin (UC05)

Tên Use Case	Mở DM và gửi tin
Tác nhân	Người dùng (chính), Auditor (phụ)
Tiền điều kiện	Hai bên là bạn hoặc được phép DM; peer đã công bố bundle.

Luồng sự kiện chính	1. Client gọi <code>start_dm</code>; server kiểm tra quyền và chặn tự-DM. 2. Client gọi <code>get_pqxdh_bundle</code>; server trả bundle của peer kèm one-time prekey cùng generation và <code>identity_version</code>. 3. Client hỏi auditor lấy Merkle proof cho <code>identity_version</code> và kiểm tra tính hợp lệ. 4. Client verify chữ ký prekey, sinh X25519 ephemeral, encapsulate vào ML-KEM của peer, trộn các DH secret và KEM secret bằng HKDF để tạo root secret và khởi tạo Double Ratchet. 5. Client mã hóa tin đầu tiên và gửi với prefix <code>S3PQI</code>; các tin sau dùng <code>S3DR</code>. 6. Server lưu ciphertext và chuyển tiếp cho peer; peer giải mã và tiến ratchet.
Luồng sự kiện phát sinh	• Peer thiếu bundle: client báo lỗi rõ và không gửi plaintext. • Bị chặn: server từ chối; bubble đang gửi chuyển sang trạng thái thất bại. • Audit mismatch: hiển thị cảnh báo nhưng vẫn cho gửi (cảnh báo, không chặn).
Hậu điều kiện	• Tin được lưu dưới dạng ciphertext; trạng thái ratchet tiến lên.

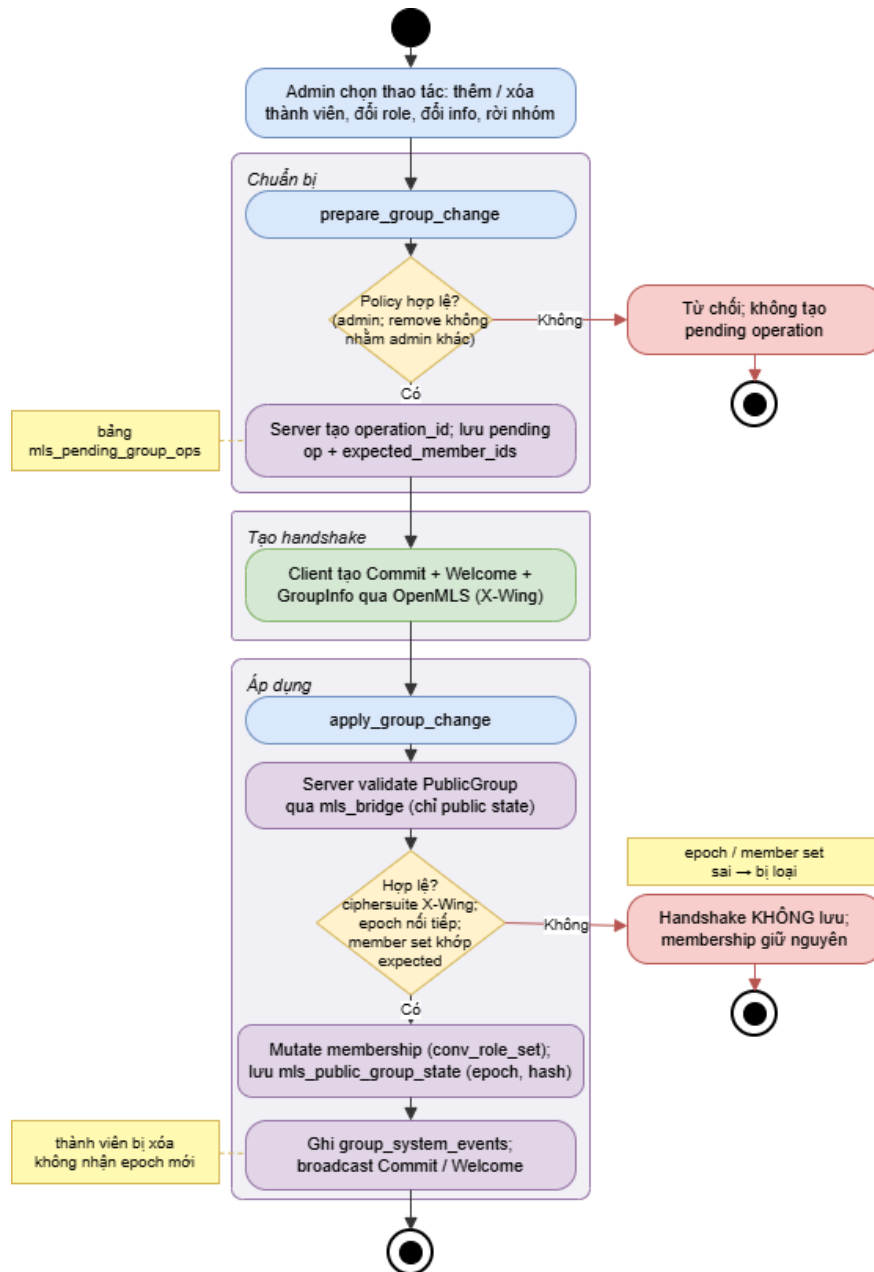


Hình 2.3: Sơ đồ hoạt động: Mở DM và gửi tin (UC05)

2.2.5.3 UC10 - Quản lý thành viên và vai trò

Bảng 2.5: Đặc tả Use Case: Quản lý thành viên và vai trò (UC10)

Tên Use Case	Quản lý thành viên và vai trò
Tác nhân	Admin group (chính), Server, OpenMLS bridge
Tiền điều kiện	• Là admin (thêm/xóa/đổi role/đổi info) hoặc thành viên (rời nhóm).
Luồng sự kiện chính	1. Admin chọn thao tác: thêm/xóa thành viên, đổi role, đổi thông tin nhóm hoặc rời nhóm. 2. <code>prepare_group_change</code>: server kiểm tra policy (quyền admin, friend/block, KeyPackage còn dùng) và tạo operation id. 3. Client tạo Commit, Welcome và GroupInfo qua OpenMLS. 4. <code>apply_group_change</code>: server validate <code>PublicGroup</code>, đối chiếu member set với pending operation và epoch nối tiếp rồi mutate membership và ghi system event.
Luồng sự kiện phát sinh	• Không phải admin hoặc xóa một admin khác: bị từ chối. • Validation thất bại: handshake không được lưu, membership giữ nguyên.
Hậu điều kiện	• Membership và role được cập nhật; system event ghi vào timeline; thành viên bị xóa không nhận epoch mới.



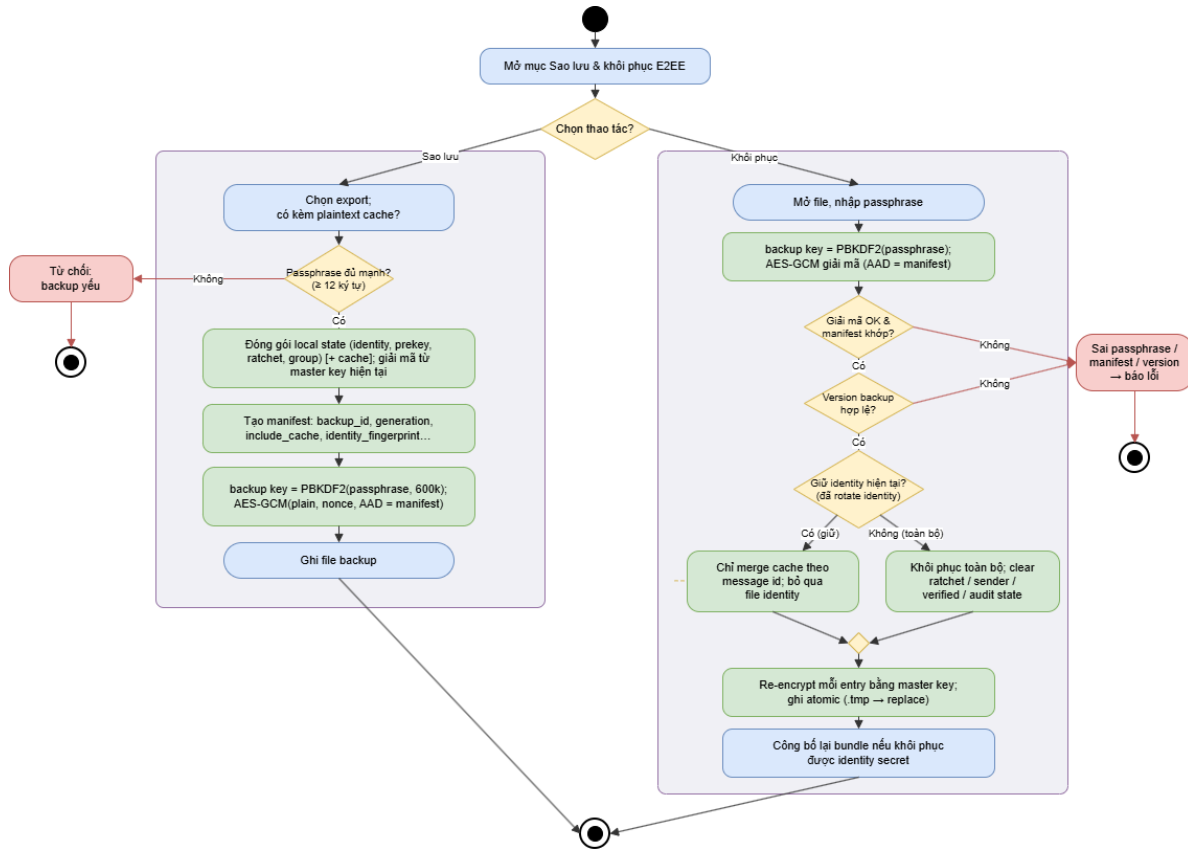
Hình 2.4: Sơ đồ hoạt động: Quản lý thành viên và vai trò (UC10)

2.2.5.4 UC13 - Sao lưu và khôi phục E2EE

Bảng 2.6: Đặc tả Use Case: Sao lưu và khôi phục E2EE (UC13)

Tên Use Case	Sao lưu và khôi phục E2EE
Tác nhân	Người dùng (chính)
Tiền điều kiện	Đã đăng nhập (khi sao lưu); có file backup và passphrase (khi khôi phục).

Luồng sự kiện chính	1. Người dùng chọn export và chọn có kèm plaintext cache hay không. 2. Client đóng gói local state (identity, prekey, ratchet, group) và cache tùy chọn, mã hóa bằng PBKDF2(passphrase) cùng AES-GCM với manifest làm AAD. 3. Khi khôi phục, người dùng mở file, nhập passphrase và client giải mã. 4. Client re-wrap state bằng master key hiện tại, merge cache theo message id và công bố lại bundle nếu khôi phục được identity secret.
Luồng sự kiện phát sinh	• Sai passphrase: báo lỗi không thể giải mã. • Backup dùng passphrase yếu: bị từ chối. • Khôi phục sau khi đã rotate identity: không rollback identity, chỉ merge cache đọc được.
Hậu điều kiện	• Lịch sử đọc được khôi phục cục bộ; server không nhận plaintext.

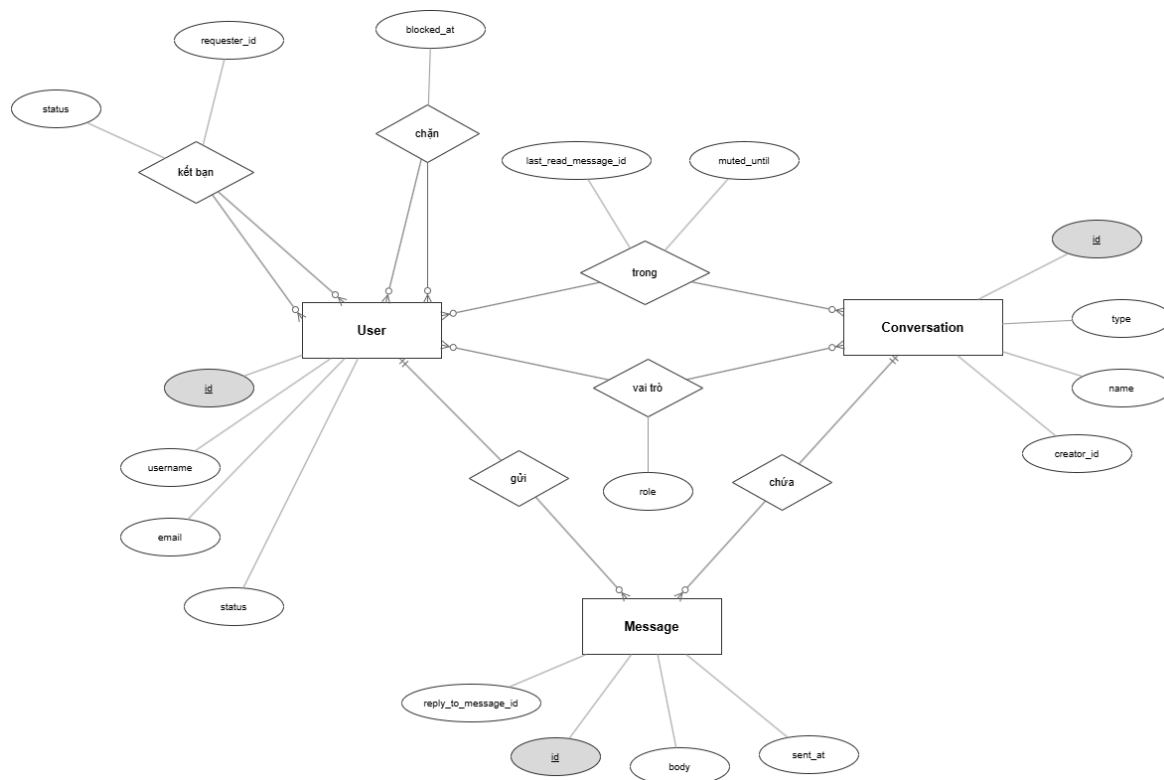


Hình 2.5: Sơ đồ hoạt động: Sao lưu và khôi phục E2EE (UC13)

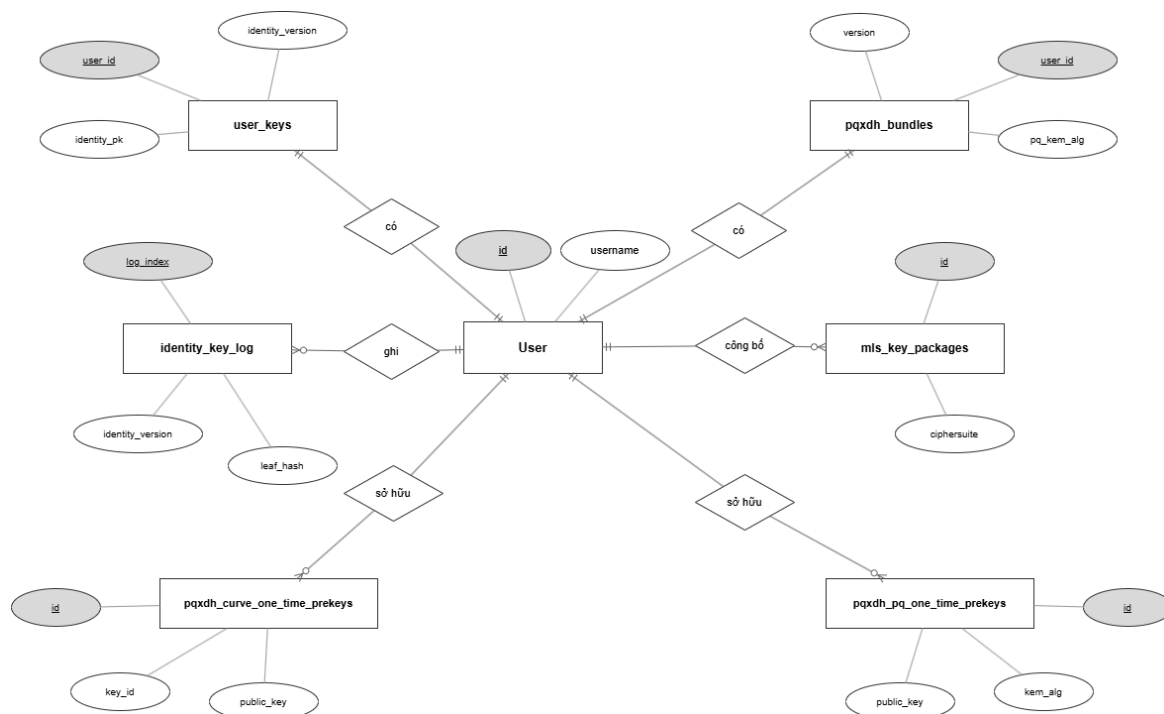
2.3 Thiết kế hệ thống

2.3.1 Thiết kế cơ sở dữ liệu

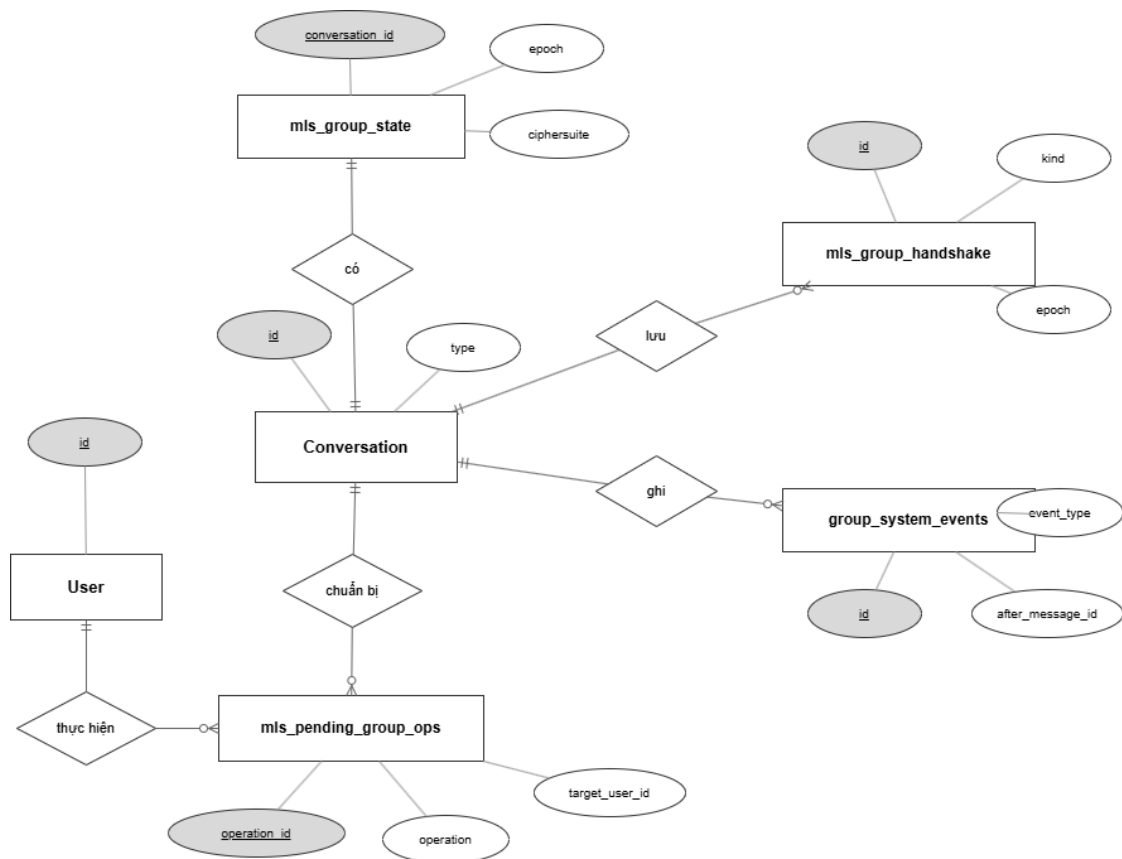
Database của SecChat gồm 20 bảng, không lưu plaintext message, nhưng vẫn lưu metadata cần thiết để hệ thống vận hành. Trường body của bảng **messages** chỉ chứa ciphertext theo định dạng prefix của giao thức (trình bày ở mục 2.5.2). Các bảng **conversation_members**, **mls_group_state** và **group_system_events** là các điểm quan trọng để bảo đảm group state và timeline ổn định. Các sơ đồ ERD trong mục này là mô hình dữ liệu chính ở mức thiết kế logic.



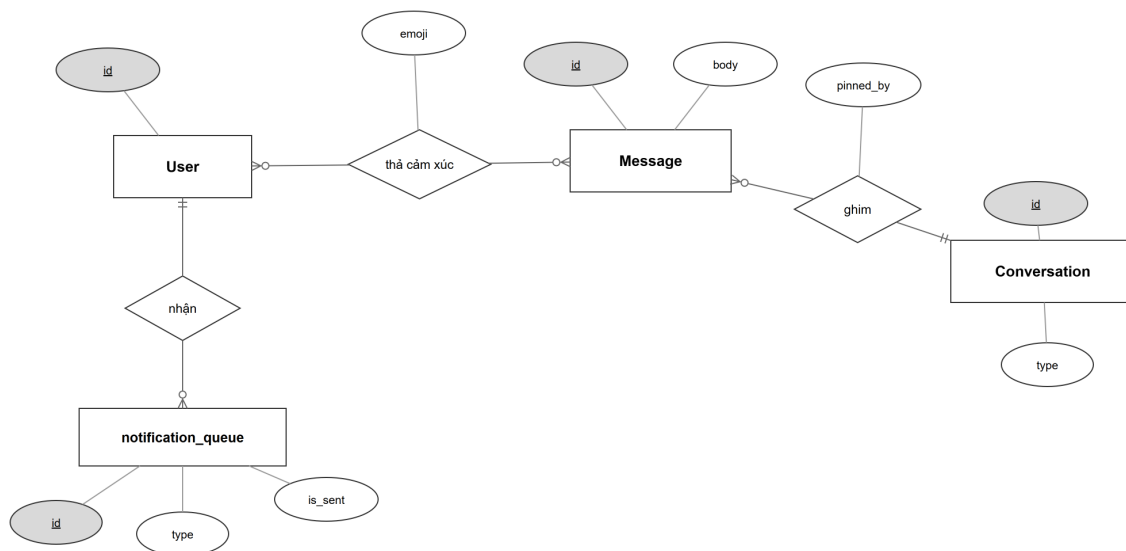
Hình 2.6: ERD phân hệ lõi: người dùng, hội thoại và tin nhắn



Hình 2.7: ERD phân hệ khóa và định danh (PQXDH, MLS KeyPackage, identity log)



Hình 2.8: ERD phân hệ nhóm MLS (group state, handshake, system event, pending op)



Hình 2.9: ERD phân hệ tiện ích tin nhắn và thông báo (reaction, pin, notification)

2.3.2 Mô tả thực thể

Bảng mô tả thực thể dưới đây trình bày các thuộc tính chính của từng bảng dữ liệu. Các trường kỹ thuật lặp lại như mốc tạo/cập nhật, cờ trạng thái phụ hoặc dữ liệu phụ trợ chỉ được nêu khi ảnh hưởng trực tiếp đến luồng nghiệp vụ, E2EE, MLS hoặc quan sát vận hành.

Bảng 2.7: Mô tả các thực thể dữ liệu chính

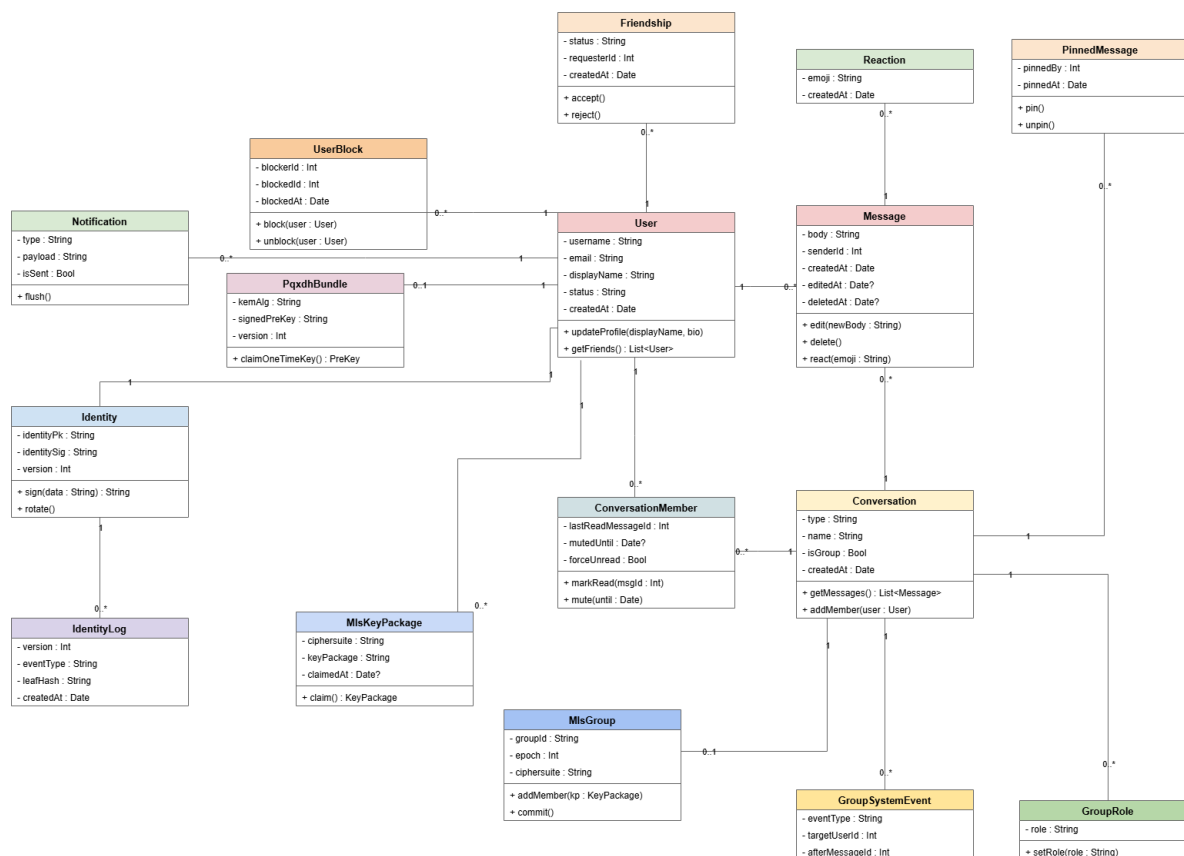
Thực thể	Tên thuộc tính	Kiểu dữ liệu	Mô tả
users	id	INT, PK	Khóa chính tự tăng.
	username	VARCHAR(64), UNIQUE	Tên đăng nhập.
	email	VARCHAR(200)	Email đăng nhập.
	password_hash	VARCHAR(200)	Hash mật khẩu (PBKDF2 salt server), không lưu plaintext.
	display_name	VARCHAR(100)	Tên hiển thị.
	avatar	MEDIUMBLOB	Ảnh đại diện (kèm avatar_mime).
	status	ENUM (online /offline /away)	Trạng thái hiện diện.
	privacy_online, privacy_last_seen	ENUM (every- one /friends /nobody)	Quyền riêng tư hiện diện.
friendships	id	BIGINT, PK	Khóa chính.
	user_id, friend_id	INT, FK	Cặp người dùng trong quan hệ.
	requester_id	INT, FK	Người gửi lời mời.
	status	ENUM (pend- ing /accepted /blocked)	Trạng thái quan hệ.
user_blocks	blocker_id	INT, PK, FK	Người chặn.
	blocked_id	INT, PK, FK	Người bị chặn.
	blocked_at	DATETIME	Thời điểm chặn.
conversations	id	BIGINT, PK	Khóa chính.
	type	ENUM (direct /group)	Loại hội thoại.
	name	VARCHAR(100)	Tên nhóm.
	creator_id	INT, FK	Người tạo nhóm.
	description	TEXT	Mô tả nhóm (kèm avatar).
	disappear_after_secs	INT	Thời gian tin tự hủy (kèm disappear_enabled_at).
	is_disbanded	TINYINT(1)	Cờ nhóm đã giải tán.
conversation_members	id	BIGINT, PK	Khóa chính.
	conversation_id	BIGINT, FK	Hội thoại.
	user_id	INT, FK	Thành viên.
	last_read_message_id	BIGINT	Còn trở đã đọc.
	force_unread	TINYINT	Đánh dấu chưa đọc thủ công.
	muted_until, conv_pinned_at	DATETIME	Tắt thông báo / ghim hội thoại.
group_roles	conversation_id	BIGINT, PK, FK	Nhóm.
	user_id	INT, PK, FK	Thành viên.
	role	ENUM (admin /member)	Vai trò trong nhóm.
messages	id	BIGINT, PK	Khóa chính.
	conversation_id	BIGINT, FK	Hội thoại.
	sender_id	INT, FK	Người gửi.

Thực thể	Tên thuộc tính	Kiểu dữ liệu	Mô tả
	body	LONGTEXT	Ciphertext, prefix S3PQI/S3DR/S3MLS (tệp đính kèm là marker FILE: bên trong plaintext đã giải mã).
	reply_to_message_id	BIGINT	Tin được trả lời.
	edit_target_message_id	BIGINT	Tin gốc của event chỉnh sửa.
	deleted_at	DATETIME(3)	Mốc soft-delete.
message_reactions	id	BIGINT, PK	Khóa chính.
	message_id	BIGINT, FK	Tin được thả cảm xúc.
	user_id	INT, FK	Người thả.
	emoji	VARCHAR(16)	Biểu tượng cảm xúc.
pinned_messages	conversation_id	BIGINT, PK, FK	Hội thoại.
	message_id	BIGINT, PK, FK	Tin được ghim.
	pinned_by	INT, FK	Người ghim.
user_keys	user_id	INT, PK, FK	Chủ sở hữu khóa.
	identity_pk	TEXT	Ed25519 identity public key.
	identity_sig	TEXT	Chữ ký tự xác thực identity.
	identity_sk_enc	TEXT	Identity secret đã mã hóa bằng master key.
	identity_version	INT	Phiên bản identity.
pqxdh_bundles	user_id	INT, PK, FK	Chủ bundle.
	curve_spk, curve_spk_sig	TEXT	Signed X25519 prekey và chữ ký.
	pq_kem_alg	VARCHAR(32)	Thuật toán KEM (ML-KEM/Kyber).
	pq_spk, pq_spk_sig	TEXT	Signed PQ KEM prekey và chữ ký.
	version	INT	Phiên bản bundle.
pqxdh_curve_one_time_prekeys	id	BIGINT, PK	Khóa chính.
	user_id	INT, FK	Chủ prekey.
	key_id	INT	Định danh prekey.
	public_key, signature	TEXT	One-time X25519 prekey và chữ ký.
pqxdh_pq_one_time_prekeys	id	BIGINT, PK	Khóa chính.
	user_id	INT, FK	Chủ prekey.
	kem_alg	VARCHAR(32)	Thuật toán KEM.
	public_key, signature	TEXT	One-time PQ KEM prekey và chữ ký.
identity_key_log	log_index	BIGINT, PK	Chỉ số append-only.
	user_id	INT, FK	Chủ identity.
	identity_version	INT	Phiên bản identity.
	identity_pk	TEXT	Identity key tại version đó.
	event_type	ENUM (initial /rotation)	Loại sự kiện.
	leaf_hash	CHAR(64)	Hash lá Merkle.
mls_key_packages	id	BIGINT, PK	Khóa chính.
	user_id	INT, FK	Chủ KeyPackage.
	ciphersuite	VARCHAR(128)	Ciphersuite X-Wing.
	key_package_b64	LONGTEXT	KeyPackage MLS (base64).

Thực thể	Tên thuộc tính	Kiểu dữ liệu	Mô tả
	secchat_signature	TEXT	Chữ ký SecChat trên KeyPackage.
	claimed_at, claimed_by	DATETIME(3), INT	Trạng thái đã claim (single-use).
mls_group_state	conversation_id	BIGINT, PK, FK	Nhóm.
	group_id_b64	VARCHAR(256)	Group id MLS.
	epoch	BIGINT	Epoch hiện hành.
	ciphersuite	VARCHAR(128)	Ciphersuite X-Wing.
	public_state_b64	LONGTEXT	Public group state đã validate.
	public_state_hash, validated_at	VARCHAR(128), DATETIME(3)	Hash và mốc validate.
mls_pending_group_ops	operation_id	VARCHAR(64), PK	Định danh operation.
	conversation_id	BIGINT, FK	Nhóm.
	actor_id	INT, FK	Người thực hiện.
	operation	VARCHAR(32)	Loại thao tác (add/remove/...).
	target_user_id	INT	Đối tượng tác động.
	status	ENUM (prepared /applied /rejected)	Trạng thái; expires_at giới hạn thời gian.
mls_group_handshake	id	BIGINT, PK	Khóa chính.
	conversation_id	BIGINT, FK	Nhóm.
	kind	ENUM (welcome /commit /group_info /control)	Loại artifact MLS.
	epoch	BIGINT	Epoch của artifact.
	operation_id	VARCHAR(64)	Operation liên quan.
	mls_message_b64	LONGTEXT	Bytes MLS (base64) để client replay.
group_system_events	id	BIGINT, PK	Khóa chính.
	conversation_id	BIGINT, FK	Nhóm.
	event_type	VARCHAR(32)	Loại sự kiện (add/remove/role/...).
	target_user_id, role	INT, VARCHAR(16)	Đối tượng và vai trò.
	operation_id	VARCHAR(64)	Khử trùng lặp với real-time.
	after_message_id	BIGINT	Vị trí trong timeline.
notification_queue	id	BIGINT, PK	Khóa chính.
	user_id	INT, FK	Người nhận.
	type	VARCHAR(64)	Loại notification.
	payload	TEXT	Nội dung sự kiện.
	is_sent	TINYINT	Đã flush hay chưa.

2.3.3 Thiết kế cấu trúc đối tượng

Sơ đồ class diagram trong mục này mô tả các lớp và module chính của hệ thống ở mức thiết kế. Mỗi lớp chỉ nêu các thuộc tính và phương thức tiêu biểu thể hiện trách nhiệm của thành phần đó.



Hình 2.10: Sơ đồ class diagram các lớp chính của hệ thống

1. Class: User

Tên	Loại	Kiểu dữ liệu / Tham số	Mô tả
username	Thuộc tính	String	Tên đăng nhập (duy nhất).
email	Thuộc tính	String	Email đăng nhập.
displayName	Thuộc tính	String	Tên hiển thị.
status	Thuộc tính	String	Trạng thái hiện diện (online/offline).
createdAt	Thuộc tính	Date	Ngày tạo tài khoản.
updateProfile	Phương thức	displayName, bio	Cập nhật hồ sơ.
getFriends	Phương thức	→ List<User>	Lấy danh sách bạn bè.

2. Class: Friendship

Tên	Loại	Kiểu dữ liệu / Tham số	Mô tả
status	Thuộc tính	String	Trạng thái (pending/accepted/blocked).
requesterId	Thuộc tính	Int	Người gửi lời mời.
createdAt	Thuộc tính	Date	Thời điểm tạo.
accept	Phương thức	void	Chấp nhận lời mời.
reject	Phương thức	void	Từ chối lời mời.

3. Class: UserBlock

Tên	Loại	Kiểu dữ liệu / Tham số	Mô tả
blockerId	Thuộc tính	Int	Người chặn.
blockedId	Thuộc tính	Int	Người bị chặn.
blockedAt	Thuộc tính	Date	Thời điểm chặn.
block	Phương thức	user: User	Chặn một người dùng.
unblock	Phương thức	user: User	Bỏ chặn.

4. Class: Notification

Tên	Loại	Kiểu dữ liệu / Tham số	Mô tả
type	Thuộc tính	String	Loại thông báo.
payload	Thuộc tính	String	Nội dung sự kiện.
isSent	Thuộc tính	Bool	Đã đẩy tới client hay chưa.
flush	Phương thức	void	Đẩy thông báo còn tồn khi user online.

5. Class: IdentityLog

Tên	Loại	Kiểu dữ liệu / Tham số	Mô tả
version	Thuộc tính	Int	Phiên bản identity tại sự kiện.
eventType	Thuộc tính	String	Loại sự kiện (initial/rotation).
leafHash	Thuộc tính	String	Hash lá trong cây Merkle minh bạch.
createdAt	Thuộc tính	Date	Thời điểm ghi log.

6. Class: Identity

Tên	Loại	Kiểu dữ liệu / Tham số	Mô tả
identityPk	Thuộc tính	String	Khóa định danh công khai Ed25519.
identitySig	Thuộc tính	String	Chữ ký tự xác thực.
version	Thuộc tính	Int	Phiên bản identity.
sign	Phương thức	data: String → String	Ký prekey/dữ liệu.
rotate	Phương thức	void	Xoay sang identity key mới.

7. Class: PqxdhBundle

Tên	Loại	Kiểu dữ liệu / Tham số	Mô tả
kemAlg	Thuộc tính	String	Thuật toán KEM (ML-KEM-1024).
signedPreKey	Thuộc tính	String	Signed prekey đã ký.
version	Thuộc tính	Int	Phiên bản bundle.
claimOneTimeKey	Phương thức	→ PreKey	Cấp một one-time prekey.

8. Class: MlsKeyPackage

Tên	Loại	Kiểu dữ liệu / Tham số	Mô tả
ciphersuite	Thuộc tính	String	Ciphersuite X-Wing.
keyPackage	Thuộc tính	String	KeyPackage MLS (base64).
claimedAt	Thuộc tính	Date?	Thời điểm bị claim (single-use).
claim	Phương thức	→ KeyPackage	Lấy KeyPackage để thêm vào nhóm.

9. Class: Conversation

Tên	Loại	Kiểu dữ liệu / Tham số	Mô tả
type	Thuộc tính	String	Loại (direct/group).
name	Thuộc tính	String	Tên nhóm.
isGroup	Thuộc tính	Bool	Cờ nhóm/DM.
createdAt	Thuộc tính	Date	Ngày tạo.
getMessages	Phương thức	→ List<Message>	Lấy danh sách tin.
addMember	Phương thức	user: User	Thêm thành viên.

10. Class: ConversationMember

Tên	Loại	Kiểu dữ liệu / Tham số	Mô tả
lastReadMessageId	Thuộc tính	Int	Con trỏ đã đọc.
mutedUntil	Thuộc tính	Date?	Tắt thông báo đến thời điểm.
forceUnread	Thuộc tính	Bool	Đánh dấu chưa đọc thủ công.
markRead	Phương thức	msgId: Int	Đánh dấu đã đọc tới msgId.
mute	Phương thức	until: Date	Tắt thông báo hội thoại.

11. Class: GroupRole

Tên	Loại	Kiểu dữ liệu / Tham số	Mô tả
role	Thuộc tính	String	Vai trò (admin/member).
setRole	Phương thức	role: String	Đặt vai trò thành viên.

12. Class: MlsGroup

Tên	Loại	Kiểu dữ liệu / Tham số	Mô tả
groupId	Thuộc tính	String	Định danh group MLS.
epoch	Thuộc tính	Int	Epoch hiện hành.
ciphersuite	Thuộc tính	String	X-Wing (ML-KEM-768 + X25519).
addMember	Phương thức	kp: KeyPackage	Thêm thành viên, sinh Commit/Welcome.

Tên	Loại	Kiểu dữ liệu / Tham số	Mô tả
commit	Phương thức	void	Áp thay đổi, tiến epoch.

13. Class: Message

Tên	Loại	Kiểu dữ liệu / Tham số	Mô tả
body	Thuộc tính	String	Nội dung đã mã hóa (ciphertext).
senderId	Thuộc tính	Int	Người gửi.
createdAt	Thuộc tính	Date	Thời điểm gửi.
editedAt	Thuộc tính	Date?	Thời điểm sửa (null nếu chưa).
deletedAt	Thuộc tính	Date?	Thời điểm thu hồi (null nếu chưa).
edit	Phương thức	newBody: String	Sửa nội dung.
delete	Phương thức	void	Thu hồi (soft-delete).
react	Phương thức	emoji: String	Thả cảm xúc.

14. Class: Reaction

Tên	Loại	Kiểu dữ liệu / Tham số	Mô tả
emoji	Thuộc tính	String	Biểu tượng cảm xúc.
createdAt	Thuộc tính	Date	Thời điểm thả.

15. Class: PinnedMessage

Tên	Loại	Kiểu dữ liệu / Tham số	Mô tả
pinnedBy	Thuộc tính	Int	Người ghim.
pinnedAt	Thuộc tính	Date	Thời điểm ghim.
pin	Phương thức	void	Ghim tin.
unpin	Phương thức	void	Bỏ ghim.

16. Class: GroupSystemEvent

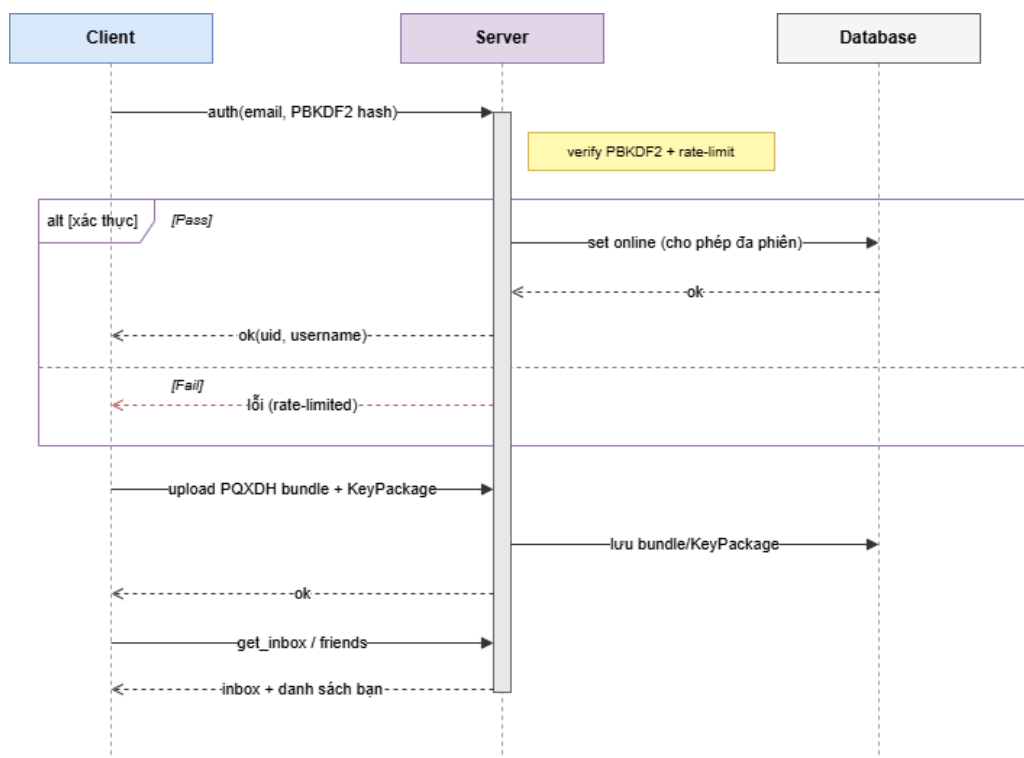
Tên	Loại	Kiểu dữ liệu / Tham số	Mô tả
eventType	Thuộc tính	String	Loại sự kiện (add/remove/role...).
targetUserId	Thuộc tính	Int	Đối tượng tác động.
afterMessageId	Thuộc tính	Int	Vị trí trong dòng thời gian.

2.3.4 Thiết kế luồng xử lý

Mục này đặc tả các luồng nghiệp vụ chính dưới dạng sơ đồ tuần tự, mô tả trình tự trao đổi thông điệp giữa client, server, database, auditor và OpenMLS bridge. Các luồng phụ như sao lưu/khôi phục E2EE và kiểm tra identity transparency đã được trình bày dưới dạng sơ đồ luồng ở mục 2.4.

Quy trình đăng nhập và công bố khóa

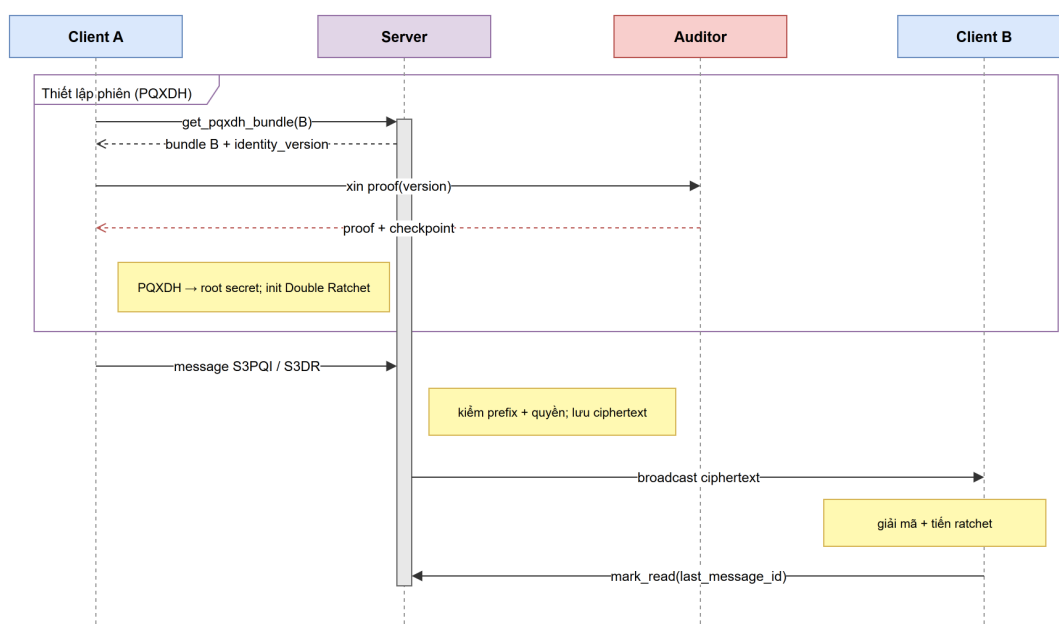
Sau khi xác thực thành công, client mở local state đã mã hóa và công bố lại bundle khóa của máy hiện tại để người khác có thể bắt đầu hội thoại ngay cả khi chủ tài khoản offline.



Hình 2.11: Sơ đồ tuần tự: đăng nhập và công bố khóa

Quy trình gửi tin nhắn DM E2EE

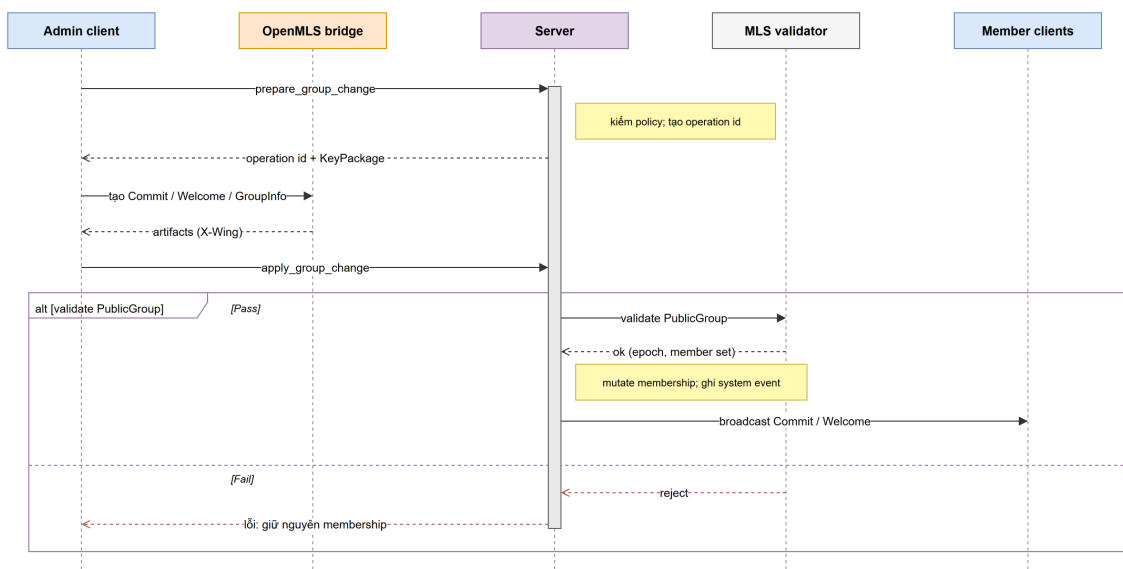
Client lấy bundle của peer, kiểm tra proof identity với auditor, thực hiện bắt tay PQXDH để tạo root secret rồi gửi tin đã mã hóa; server chỉ lưu và chuyển tiếp ciphertext.



Hình 2.12: Sơ đồ tuần tự: gửi tin nhắn DM E2EE

Quy trình thay đổi thành viên group bằng MLS

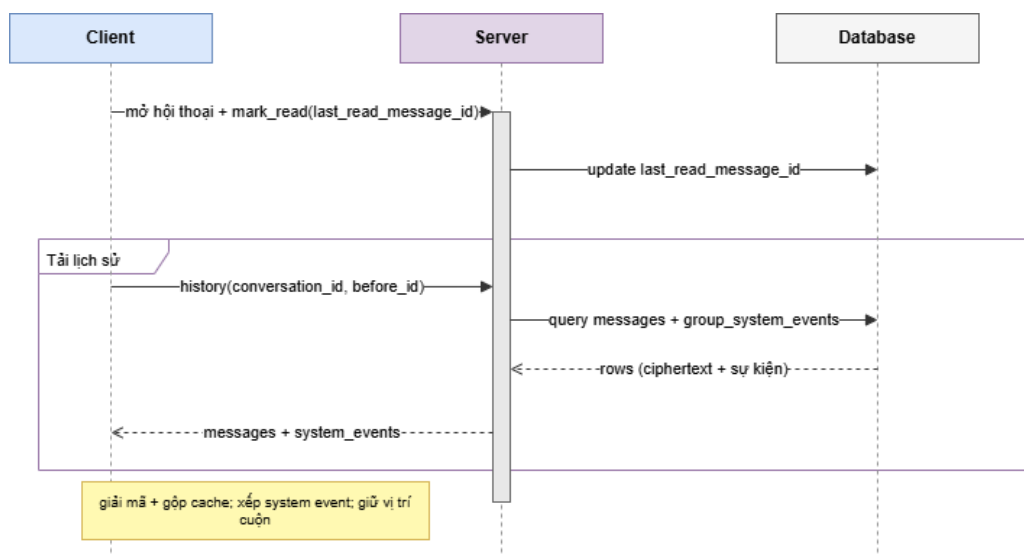
Mọi thay đổi membership đi qua hai bước **prepare** và **apply**; server chỉ áp dụng thay đổi sau khi MLS validator chấp nhận public state dựa trên **PublicGroup**.



Hình 2.13: Sơ đồ tuần tự: thay đổi thành viên group bằng MLS

Quy trình tải lịch sử, read cursor và system event

Khi mở hội thoại, client cập nhật read cursor theo message id và tải lịch sử ciphertext cùng các system event để dựng lại timeline đúng thứ tự sau khi đăng nhập lại hoặc reload.



Hình 2.14: Sơ đồ tuần tự: tải lịch sử, read cursor và system event

2.4 Thiết kế giao thức bảo mật

Yêu cầu thiết kế quan trọng nhất là server không được đọc nội dung tin nhắn người dùng. Tin nhắn phải được mã hóa ở client trước khi gửi lên server. Server có thể lưu, chuyển tiếp, sắp xếp và trả lịch sử, nhưng trường nội dung trong cơ sở dữ liệu phải là ciphertext có prefix hợp lệ của giao thức.

Với group, hệ thống dùng MLS theo RFC 9420 qua OpenMLS để quản lý epoch, secret tree, Welcome và Commit. Ciphersuite group là X-Wing hybrid, kết hợp ML-KEM-768 với X25519 cho confidentiality và dùng Ed25519 cho authentication. Server phải từ chối KeyPackage không đúng ciphersuite, kiểm tra chữ ký SecChat trên KeyPackage, và validate public MLS state trước khi mutate membership.

Khóa bí mật của người dùng không được lưu thô. Client phải mã hóa identity key, prekey, ratchet state, group state và các file trạng thái E2EE bằng master key dẫn xuất từ mật khẩu. Khi người dùng đổi mật khẩu, các file nhạy cảm cục bộ phải được bọc lại bằng master key mới.

Client được tin tưởng để giữ plaintext, master key và trạng thái crypto. Server được tin tưởng để xác thực tài khoản, kiểm tra quyền trong group, validate public MLS state, lưu thứ tự tin nhắn và chuyển tiếp đúng người, nhưng không được tin tưởng để giữ bí mật nội dung tin.

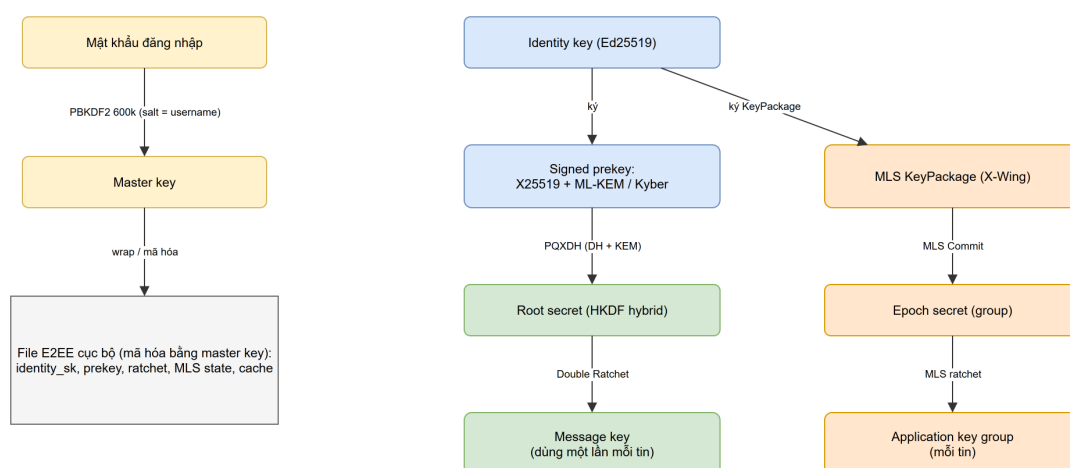
Cơ sở dữ liệu được xem là có thể bị lộ. Vì vậy các secret quan trọng phải được lưu ở dạng mã hóa.

2.4.1 Thiết kế mã hóa đầu cuối

2.4.1.1 Các lớp khóa

SecChat tách khóa theo vai trò thay vì dùng một khóa chung cho toàn bộ hệ thống. Identity key Ed25519 gắn với danh tính bảo mật của tài khoản và dùng để ký prekey. Prekey X25519 và prekey ML-KEM hoặc Kyber dùng cho bắt tay DM kiểu PQXDH. Sau bắt tay, Double Ratchet state tạo message key riêng cho từng tin DM. Với group, OpenMLS giữ MLS group state, epoch secret và application secret; server chỉ thấy public state cần cho kiểm tra membership.

Cách chia này giúp mỗi khóa có vòng đời rõ ràng. Mật khẩu dùng để mở khóa local state, identity key dùng để nhận diện người dùng, prekey dùng để bắt tay phiên, còn message key dùng cho từng tin nhắn.



Hình 2.15: Vòng đời các khóa chính trong SecChat

2.4.1.2 Mật khẩu và master key

Ở tầng đăng nhập, client không gửi mật khẩu thô lên server. Client trước hết băm mật khẩu bằng PBKDF2-HMAC-SHA256 với salt là email đăng nhập. Server tiếp tục

băm giá trị nhận được bằng PBKDF2 với salt ngẫu nhiên riêng trước khi lưu vào bảng `users`. Thiết kế này làm cho server không biết mật khẩu gốc.

Ở tầng bảo vệ file E2EE cục bộ, client dẫn xuất master key từ mật khẩu gốc và username thực tế do server trả về sau đăng nhập. Master key này dùng để mã hóa các file như identity secret key, prekey secret, ratchet state, group state, safety verification và plaintext cache. Khi đổi mật khẩu, client bọc lại các file này bằng master key mới để lịch sử và key cũ không bị mất.

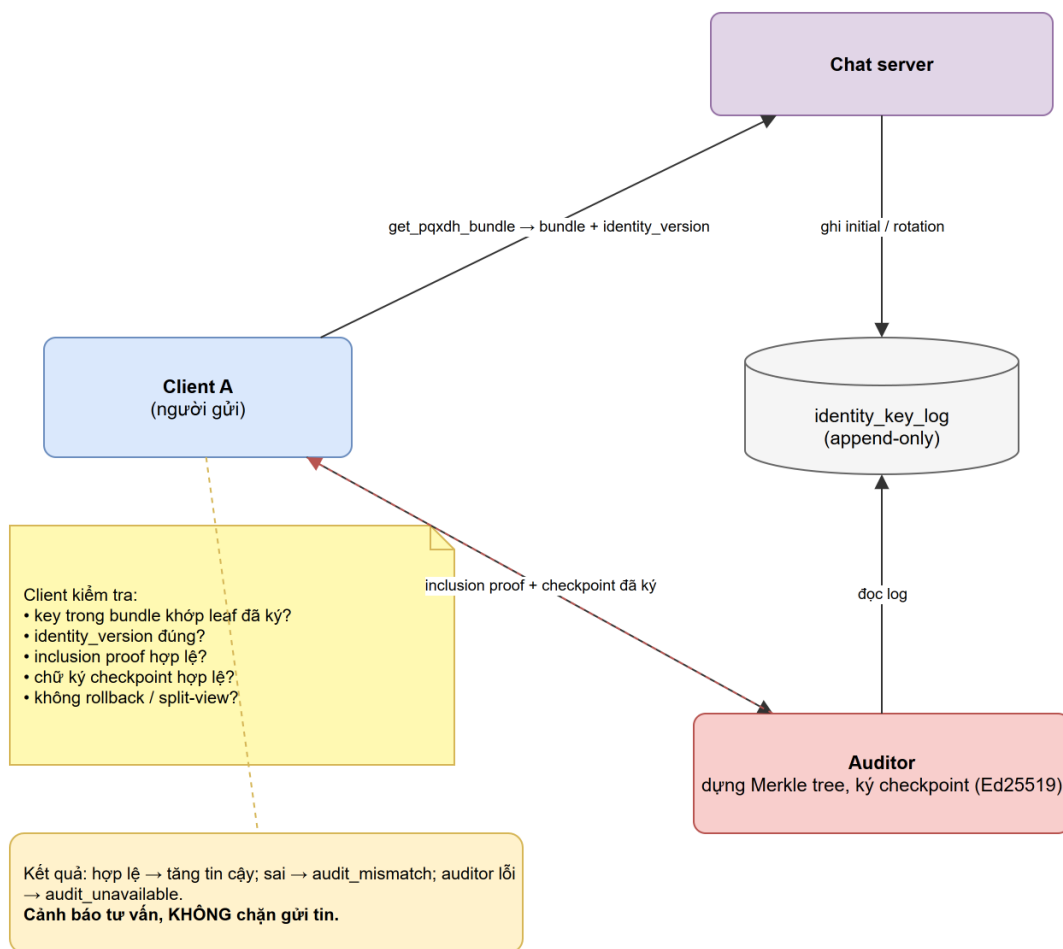
2.4.1.3 Prekey bundle kiểu PQXDH

Mỗi client công bố một bundle khóa cho SecChat. Bundle gồm identity public key Ed25519, signed X25519 prekey, signed ML-KEM hoặc Kyber prekey, cùng các one-time prekey nếu còn. Bắt tay DM ưu tiên ML-KEM-1024 khi thư viện hỗ trợ, nhưng bundle vẫn ghi rõ thuật toán đang dùng để hai phía xử lý đúng. Server lưu bundle trong các bảng như `pqxdh_bundles`, `pqxdh_curve_one_time_prekeys` và `pqxdh_pq_one_time_prekeys`.

Khi cùng một tài khoản đăng nhập trên thiết bị khác, nếu mở được identity secret, client sẽ tạo một PQXDH bundle mới cho thiết bị hiện tại và upload lại trước khi nhận hoặc gửi DM mới. Cách này giữ nguyên safety identity nhưng thay prekey theo máy đang active, tránh tình trạng server trả bundle có public prekey không khớp với secret cục bộ. Nếu server đang có bundle cũ nhưng client hiện tại thiếu private prekey, client vẫn phải publish bundle mới thay vì dựa vào trạng thái cũ.

Khi A muốn bắt đầu DM với B, A gọi `get_pqxdh_bundle`. Server trả bundle của B và chỉ cấp one-time prekey thuộc cùng generation với signed prekey hiện hành. Khi B publish bundle mới, server xóa batch one-time prekey cũ trước khi chèn batch mới; nhờ vậy người gửi không bị nhận signed prekey mới ghép với one-time prekey cũ. Bundle trả về có thêm `identity_version`. Trước khi dùng bundle, client hỏi auditor service để lấy Merkle proof cho identity key và version đó. Kết quả audit được lưu vào security state của hội thoại: proof hợp lệ làm tăng độ tin cậy; proof, key, version hoặc checkpoint sai được ghi là `audit_mismatch` với mã lỗi cụ thể; lỗi vận hành như auditor không truy cập được hoặc HTTP lỗi được ghi là `audit_unavailable`. Client kiểm tra chữ ký prekey bằng identity key của B, sinh X25519 ephemeral, encapsulate vào ML-KEM hoặc Kyber public key của B, rồi trộn các DH secret và KEM secret bằng HKDF. Secret đầu ra dùng để khởi tạo Double Ratchet.

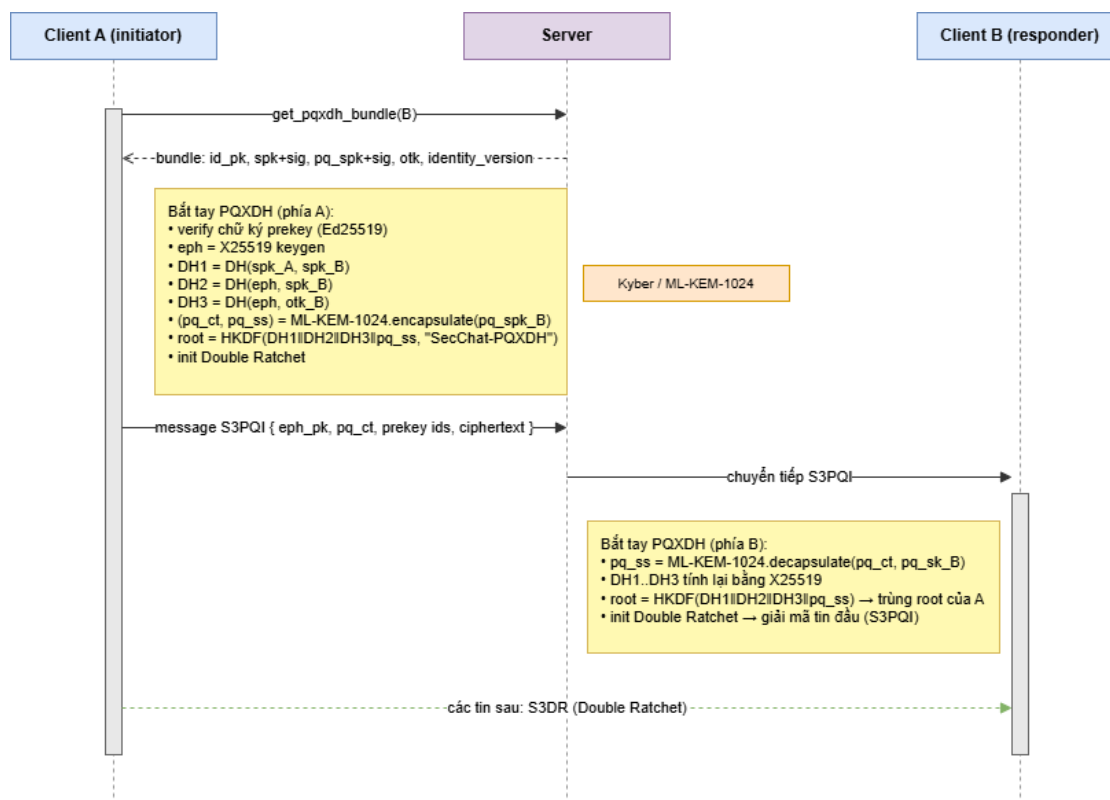
Identity transparency không thay thế safety number. Nó giúp client biết key đang dùng có nằm trong lịch sử identity đã được auditor ký hay không. Nếu safety number đổi, UI đánh dấu hội thoại là key changed và yêu cầu người dùng mở phần review để xem lịch sử identity trước khi tiếp tục gửi tin một cách bình thường. Nếu proof root không khớp, identity version không có trong log, identity key trong bundle khác log, checkpoint rollback, checkpoint split-view hoặc chữ ký checkpoint sai, UI hiển thị cảnh báo audit kèm nguyên nhân cụ thể. Nếu auditor tạm thời không truy cập được, UI cũng báo rõ đây là lỗi khả dụng của auditor, không đánh đồng với lỗi bằng chứng mật mã. Các cảnh báo này nhắc người dùng rằng chưa nên tin danh tính của đối phương nếu chưa so sánh safety number qua kênh ngoài SecChat. Cần phân biệt rõ hai loại: cảnh báo identity ở đây mang tính *tư vấn*, nghĩa là hệ thống hiển thị banner và trạng thái cần review nhưng không tự chặn cứng việc gửi tin, quyết định cuối cùng thuộc về người dùng; còn các lỗi crypto thật như chữ ký bundle sai, thiếu prekey cục bộ hoặc không có group Welcome thì làm việc mã hóa không thể tiếp tục.



Hình 2.16: Luồng identity transparency giữa client, chat server và auditor

Tin nhắn đầu tiên của DM có prefix **S3PQI:**. Prefix này chứa transcript cần thiết để B tính lại cùng root secret và nhận tin đầu tiên. Sau đó các tin DM bình thường dùng prefix **S3DR:**. Toàn bộ trình tự lấy bundle, kiểm tra proof, sinh root secret rồi gửi tin đầu tiên được minh họa ở sơ đồ tuần tự 2.12.

Chi tiết mật mã của bước bắt tay được trình bày ở Hình 2.17: A trộn ba X25519 DH với shared secret **ML-KEM-1024** (Kyber) bằng HKDF để tạo root secret; B decapsulate rồi tính lại đúng root đó. Đây là tính chất hybrid - kẻ tấn công phải phá đồng thời cả X25519 lẫn ML-KEM mới thu được root.

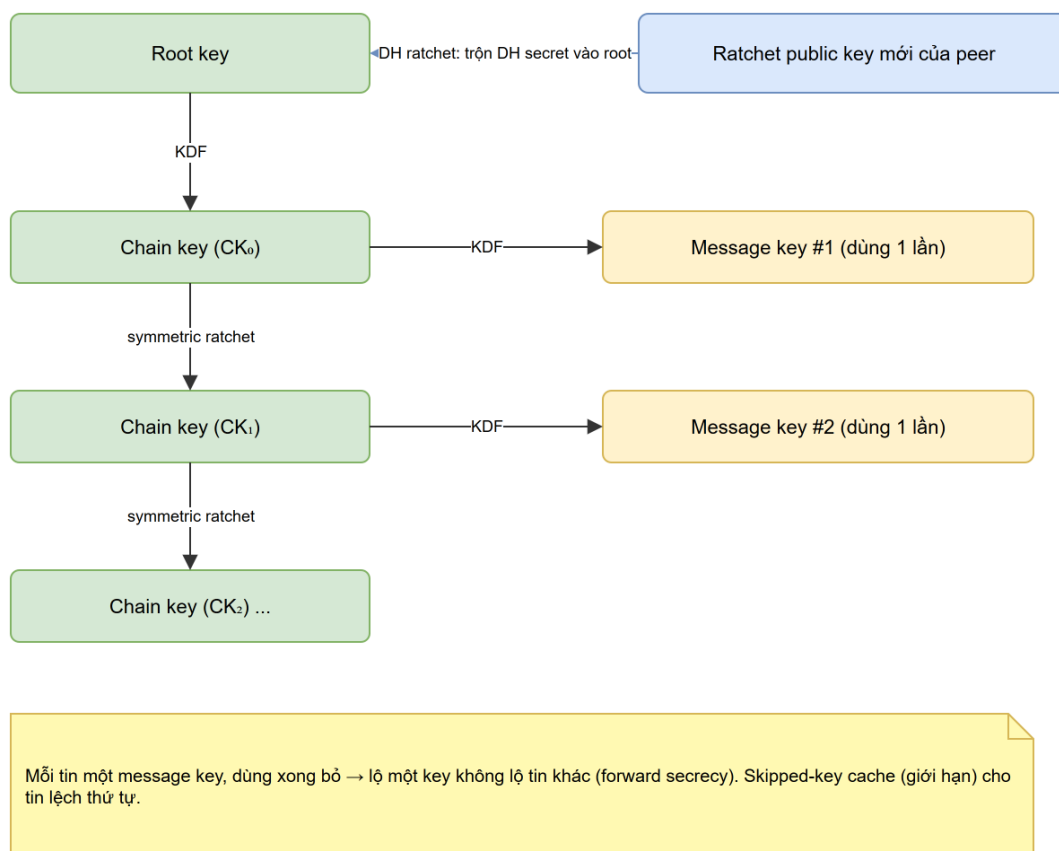


Hình 2.17: Sơ đồ tuần tự: bắt tay PQXDH hybrid (X25519 + ML-KEM-1024/Kyber)

2.4.1.4 Double Ratchet cho DM

Sau khi DM được thiết lập, root key ban đầu lấy từ bắt tay PQXDH, rồi từ đó mỗi tin được mã hóa bằng một message key dùng một lần.

Về định dạng truyền, tin đầu tiên của phiên mang prefix **S3PQI** (gói kèm dữ liệu bắt tay PQXDH), các tin sau mang prefix **S3DR**. Bộ nhớ skipped-key được giới hạn 1000 khóa để chịu được tin offline và tin lệch thứ tự mà không bị làm cạn bộ nhớ. Toàn bộ trạng thái ratchet được mã hóa bằng master key rồi lưu trên đĩa, nên người dùng đăng xuất rồi đăng nhập lại vẫn tiếp tục đúng phiên mà không mất lịch sử.



Hình 2.18: Luồng Double Ratchet cho tin nhắn DM

2.4.1.5 Group MLS hậu lượng tử hybrid

Nhóm chat không dùng Double Ratchet theo từng cặp mà dùng MLS với một group state chung có cấu trúc cây. SecChat hiện thực điều này bằng OpenMLS, chạy MLS theo RFC 9420 với ciphersuite X-Wing:

MLS_256_XWING_CHACHA20POLY1305_SHA256_Ed25519

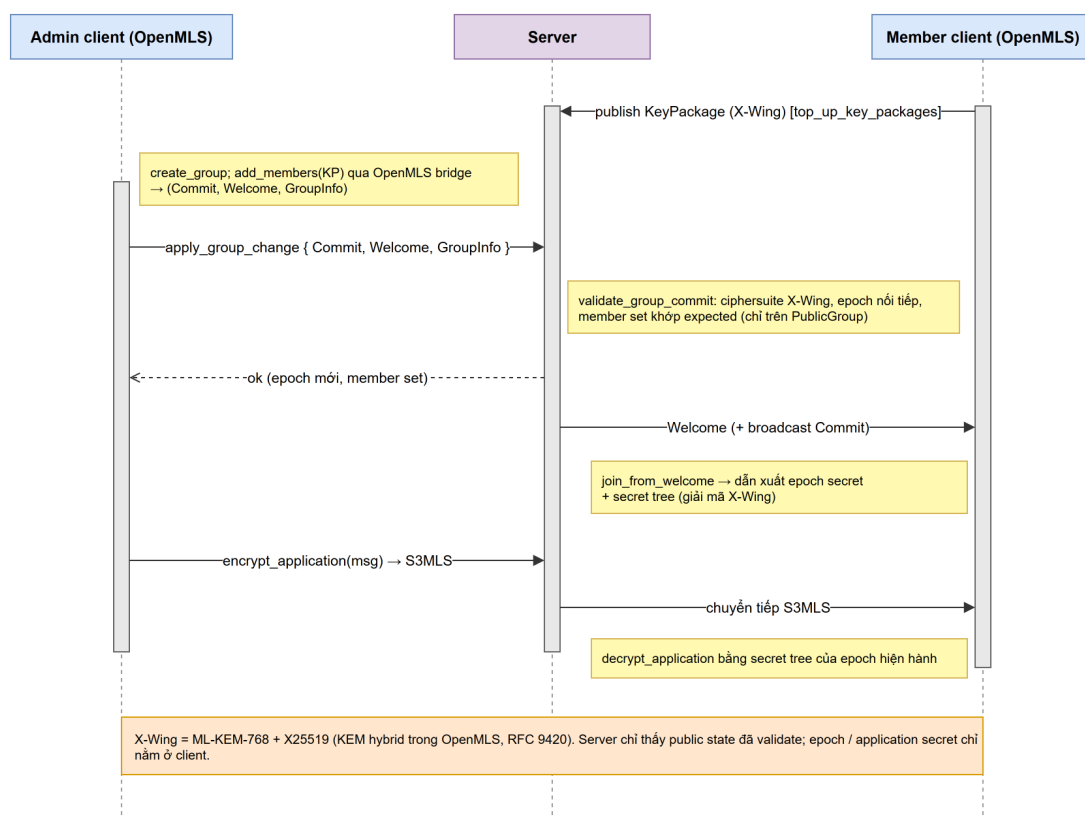
X-Wing là KEM hybrid kết hợp ML-KEM-768 và X25519, cung cấp bảo vệ hậu lượng tử cho confidentiality của group còn phần authentication của MLS sử dụng chữ ký Ed25519 cổ điển.

Mỗi thành viên công bố KeyPackage X-Wing sau khi đăng nhập. KeyPackage được ký thêm bằng SecChat identity key để server và client kiểm tra rằng package thuộc đúng tài khoản và đúng identity version. Server từ chối KeyPackage không đúng ciphersuite hoặc sai chữ ký trước khi lưu vào bảng `mls_key_packages`. Nếu client mở local MLS state cũ không còn khớp ciphersuite X-Wing, state đó bị bỏ và client công bố lại KeyPackage mới.

Mỗi thay đổi membership dùng Commit và GroupInfo do OpenMLS sinh. Luồng thay đổi bắt đầu bằng bước prepare: server kiểm tra policy như quyền admin, friend/block và KeyPackage còn chưa dùng. Client sau đó dùng OpenMLS bridge để tạo Commit, Welcome, GroupInfo và ratchet tree cần thiết, rồi gửi lại ở bước apply. Trước khi lưu handshake hoặc cập nhật bảng thành viên, server gọi validator OpenMLS dựa trên `PublicGroup`. Validator kiểm tra ciphersuite X-Wing, group id của conversation, epoch nối tiếp, credential của actor và member set sau commit có khớp operation đang chờ hay không. Trình tự prepare/apply giữa admin client, OpenMLS bridge, server và MLS validator được minh họa ở sơ đồ tuần tự 2.13.

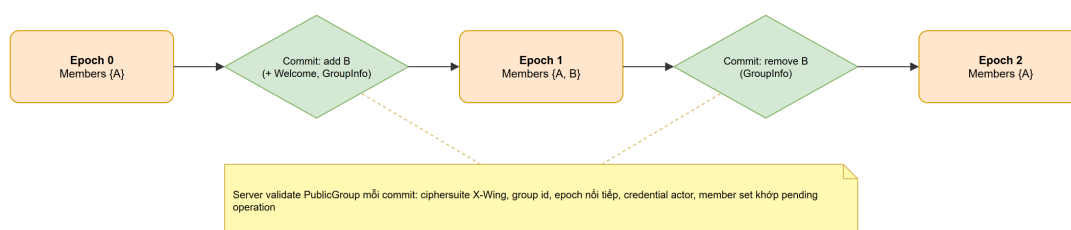
Public MLS state đã validate được lưu trong `mls_group_state`, gồm ciphersuite, public state base64, public state hash, backend validation và thời điểm validate. Server vẫn không giữ epoch secret, không sinh group secret và không giải mã MLS application bytes. Thành viên mới nhận Welcome dành riêng cho KeyPackage của mình và không mặc nhiên đọc được lịch sử trước khi tham gia. Thành viên bị xóa xử lý Commit remove thì group local inactive và không đọc được tin sau epoch mới.

Toàn bộ vòng đời khóa MLS - công bố KeyPackage X-Wing, tạo Commit/Welcome/GroupInfo, server validate `PublicGroup`, thành viên `join_from_welcome` và mã hóa application message (S3MLS) - được mô tả ở Hình 2.19.



Hình 2.19: Sơ đồ tuần tự: vòng đời khóa group MLS X-Wing (KeyPackage → Commit/Welcome → epoch → application message)

Trạng thái verified trong DM không tự áp dụng cho group. Với group, client dựa vào MLS Welcome, Commit, GroupInfo, transcript checkpoint và kiểm tra identity từng thành viên để phát hiện state không nối tiếp hoặc lịch sử bị sắp xếp bất thường.

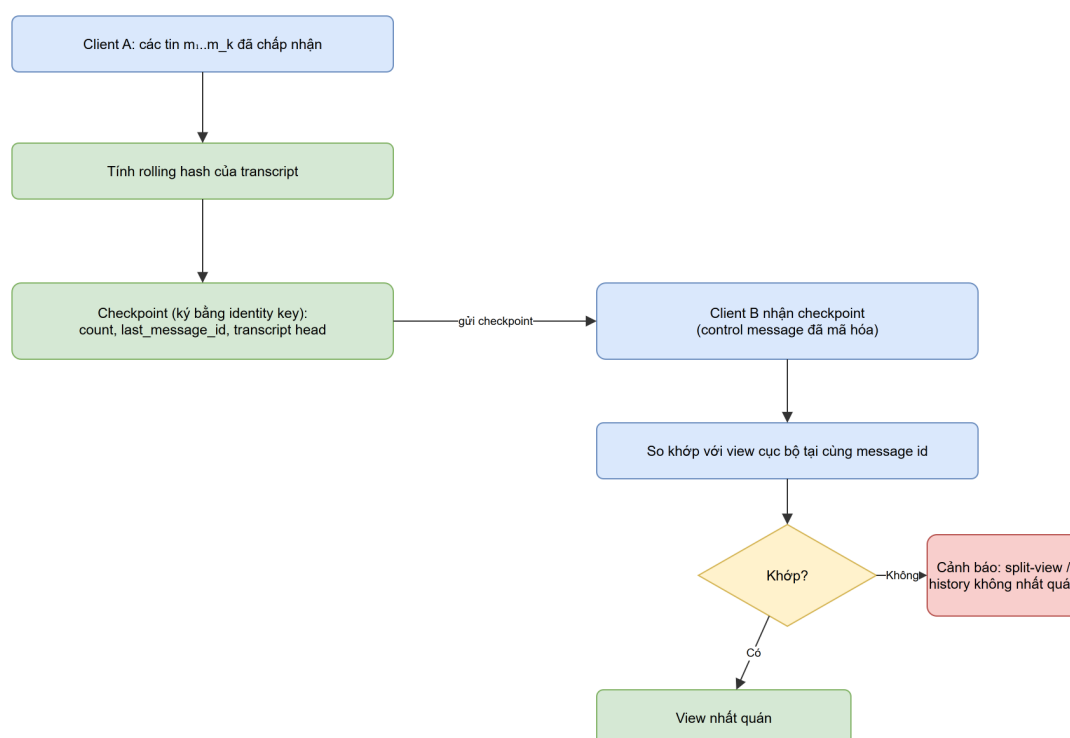


Hình 2.20: Chuỗi MLS Commit dùng để kiểm tra thay đổi membership

Ngoài commit chain cho membership, client group còn có transcript checkpoint. Mỗi client tính hash nối tiếp trên các message đã chấp nhận, ký checkpoint bằng identity key

và gửi checkpoint như một control message được mã hóa. Khi nhận checkpoint của thành viên khác, client so khớp với view cục bộ tại cùng message id. Nếu server bỏ sót message, đảo thứ tự hoặc cho hai client nhìn thấy history khác nhau, checkpoint có thể không khớp và UI sẽ có cơ sở để cảnh báo.

Khi có cảnh báo consistency, giao diện không chỉ ghi một dòng thông báo chung. Header hội thoại hiển thị trạng thái cảnh báo history và có nút Details để người dùng xem nội dung cụ thể của cảnh báo, ví dụ commit chain không nối tiếp, thiếu sự kiện membership hoặc transcript checkpoint không khớp. Cơ chế này không làm server mất khả năng gây từ chối dịch vụ, nhưng giúp client phát hiện và trình bày rõ hơn các trường hợp server trả view không nhất quán.



Hình 2.21: Transcript checkpoint giúp phát hiện group history không nhất quán

2.4.1.6 Local plaintext cache

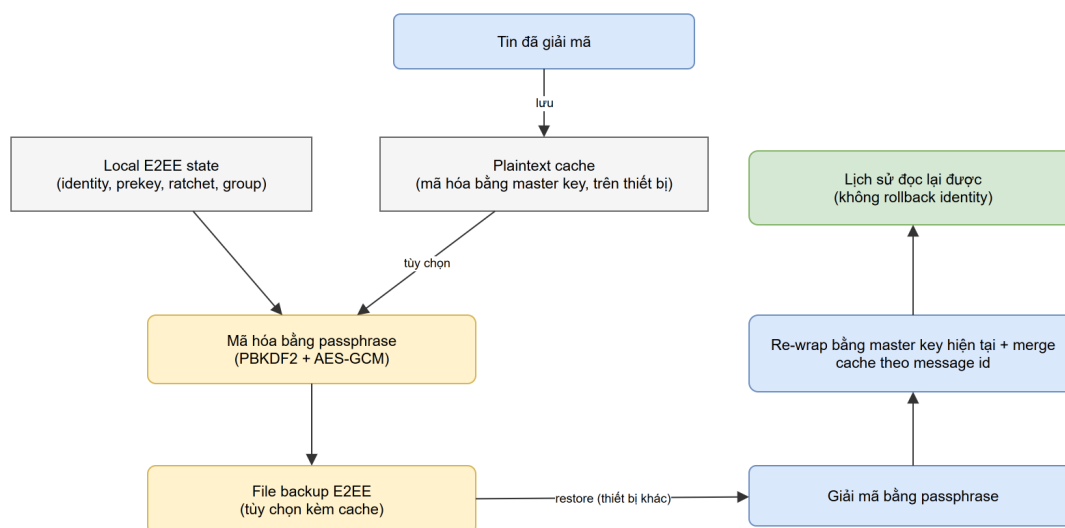
Vì Double Ratchet có tính forward secrecy, không phải mọi tin cũ đều có thể được giải mã lại vô hạn chỉ từ state hiện tại. Để trải nghiệm người dùng không bị mất lịch sử sau khi đăng xuất rồi đăng nhập lại, client lưu một cache plaintext cục bộ đã mã hóa bằng master key.

Đây là một đánh đổi có chủ ý. Cache giúp người dùng đọc lại lịch sử và giúp inbox preview hiển thị nội dung thật. Đổi lại, nếu attacker lấy được máy người dùng và mở được local state, lịch sử đã cache có thể bị đọc. Vì vậy UI có thiết lập bật hoặc tắt cache, thời gian giữ cache, xóa cache thủ công, xóa cache khi logout và tắt cache cho từng conversation nhạy cảm. Backup E2EE cũng phải cho người dùng chọn có đưa cache vào file backup hay không.

Cache được tách theo tài khoản và theo thư mục chạy của client, nên việc đổi sang tài khoản khác trên cùng thiết bị không được xóa cache của tài khoản cũ. Khi tài khoản cũ đăng nhập lại đúng thiết bị đó, các tin đã từng decrypt phải được lấy lại từ cache và hiển thị đúng thứ tự. Nếu một tài khoản mới được thêm vào group sau này, hoặc cùng tài khoản đăng nhập ở thiết bị khác chưa có cache, các ciphertext cũ không giải mã được

sẽ bị ẩn khỏi UI thay vì hiện placeholder lỗi giải mã.

Backup E2EE là một snapshot do người dùng export và giữ bằng passphrase riêng, không phải cơ chế đồng bộ liên tục. Khi restore trên thiết bị mới, client giải mã snapshot, bọc lại bằng master key hiện tại, merge cache theo message id và công bố lại bundle mới dưới cùng identity nếu identity secret được khôi phục. Việc restore như vậy giúp khôi phục lịch sử đã có trong backup mà không làm thay đổi key của người đối thoại.



Hình 2.22: Quan hệ giữa plaintext cache cục bộ và backup E2EE

2.5 Thiết kế giao thức truyền tin

2.5.1 Các loại gói chính

Các gói chính của giao thức được chia theo vai trò:

- Nhóm xác thực gồm `register`, `auth` và `change_password`.
- Nhóm hồ sơ gồm `get_profile`, `update_profile`, `upload_avatar`, `remove_avatar` và các lệnh `privacy`.
- Nhóm hội thoại gồm mở DM, tạo group, gửi message, tải history, lấy inbox và quản lý thành viên.
- Nhóm key gồm công bố identity, rotate identity, công bố PQXDH bundle, lấy PQXDH bundle và công bố MLS KeyPackage.

Các thay đổi membership dùng hai bước `prepare` và `apply`; chi tiết pending operation và validation public MLS state đã trình bày ở mục 2.4.1.5.

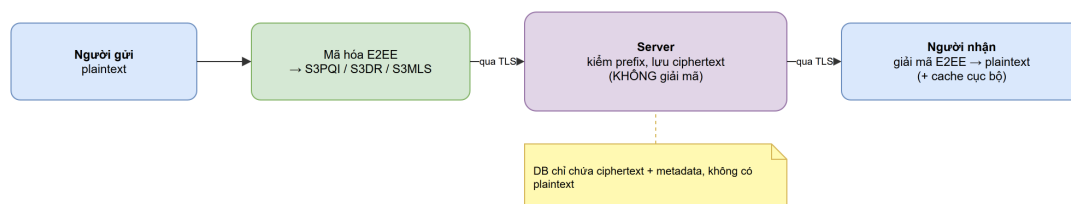
Nhóm message metadata xử lý pin, danh sách tin được ghim, reaction và typing indicator. Typing indicator có thể bị tắt vì đây là metadata nhạy cảm. Trạng thái unread chỉ áp dụng cho chính tài khoản hiện tại; hệ thống không gửi read receipt cho peer.

2.5.2 Định dạng body mã hóa

Server chỉ chấp nhận message body có prefix E2EE hợp lệ. Với SecChat, các prefix chính là:

- S3PQI: cho tin DM đầu tiên chứa transcript PQXDH-style.
- S3DR: cho tin DM sau khi đã vào Double Ratchet.
- S3MLS: cho tin group và encrypted group control message qua MLS.

Những prefix này là điều kiện bắt buộc để server lưu message mới. Nếu body không thuộc một trong ba dạng trên, server trả lỗi thay vì lưu dữ liệu, nhờ đó cơ sở dữ liệu không chứa plaintext message do client gửi nhầm.



Hình 2.23: Data flow của một tin nhắn đã mã hóa đầu cuối

2.5.3 Metadata và quyền riêng tư

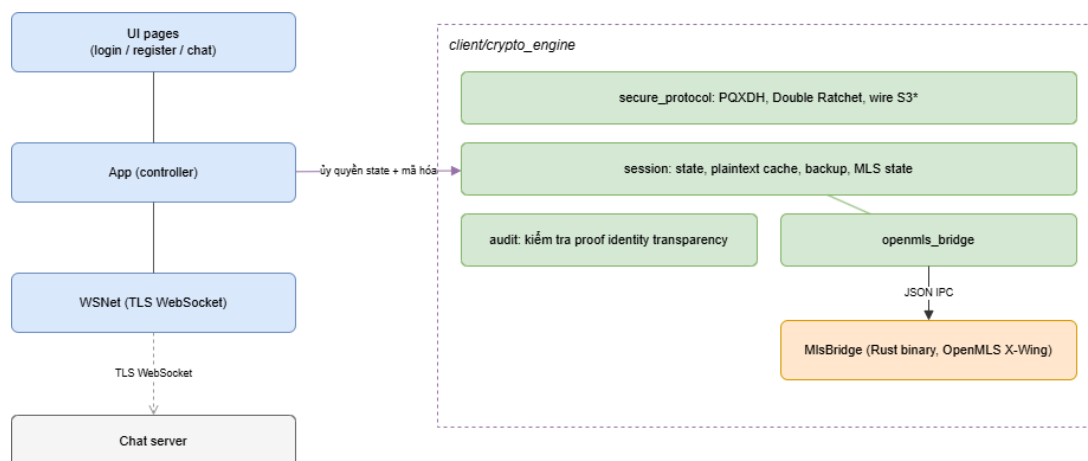
Theo mô hình tin cậy ở mục 2.4, hệ thống chỉ bảo vệ *nội dung* tin nhắn, còn metadata vẫn nằm trong phạm vi tin cậy của server. Server cần metadata để vận hành inbox, group membership, history, pin, reaction và trạng thái unread cục bộ. Đây là giới hạn quan trọng của thiết kế và được đánh giá kỹ hơn ở chương thực nghiệm.

2.6 Thiết kế client

2.6.1 Tổ chức mã nguồn

Client được chia thành các phần chính. `client/pages` chứa các trang giao diện như đăng nhập, đăng ký và chat. `client/network.py` quản lý kết nối WebSocket qua TLS. `client/dialogs.py` chứa các dialog hồ sơ, avatar, đổi mật khẩu và thiết lập phụ. `client/crypto_engine` chứa phần giao thức và trạng thái crypto.

Package `client/crypto_engine` chứa primitive của giao thức như tạo bundle, kiểm tra chữ ký prekey, transcript kiểu PQXDH, Double Ratchet và wrapper OpenMLS. `client/crypto_engine/openmls_bridge.py` gọi bridge Rust dùng ciphersuite X-Wing. `client/crypto_engine/session.py` quản lý state của phiên người dùng, file key mã hóa, cache plaintext mã hóa, trạng thái ratchet, trạng thái MLS group và mã hóa tin gửi đi.



Hình 2.24: Ranh giới giữa UI controller và crypto engine

2.6.2 Vai trò của App

`client/app.py` là controller chính của ứng dụng PyQt. Nó nhận tín hiệu từ UI, gửi lệnh qua WebSocket, nhận event từ server và cập nhật giao diện. Controller này không sở hữu trực tiếp toàn bộ crypto state; các thao tác như lưu identity key, lưu prekey, lưu ratchet state, lưu group state, cache message, rotate identity và encrypt outbound message được ủy quyền cho **CryptoSession**.

Một phần decrypt message nhận vào và replay history vẫn được **App** điều phối cùng **CryptoSession**. Cách tổ chức này giữ UI controller ở vai trò phối hợp sự kiện, còn state nhạy cảm vẫn thuộc crypto engine.

2.6.3 Giao diện chat

UI hỗ trợ các thao tác quen thuộc của ứng dụng chat: danh sách hội thoại, friend list, avatar, profile, đổi mật khẩu, privacy, block, typing indicator, reaction, pin và forward. Read receipt không nằm trong UI; trạng thái read/unread chỉ phục vụ quản lý inbox cục bộ. Các hành động trên từng tin nhắn được gom vào nút ba chấm để tránh làm rối vùng timestamp. Forward hiển thị nhãn nhỏ, edit hiển thị trạng thái đã sửa, reaction hiển thị chip dưới tin nhắn, còn tin được ghim có badge và xuất hiện trong danh sách Pins.

Message view dùng **MessageListView** dựa trên **QScrollArea**, trong đó mỗi message là một row widget có layout riêng. Cách này phù hợp với các thao tác như menu ba chấm, reaction chip, badge pin, drag-drop file và trạng thái gửi.

2.7 Thiết kế server

2.7.1 Kiến trúc phân tầng

Server C được tách thành các lớp rõ ràng. **transport** xử lý socket, WebSocket, danh sách client và HTTP control plane. **protocol** xử lý JSON. **infra** chứa cấu hình, log, crypto helper, metrics, database pool, migration và rate limit. **domain** chứa validation. **repository** chứa truy vấn SQL. **services** chứa logic nghiệp vụ: auth, messaging, groups, keys, friends, profile, presence, blocking, notifications, reaction, pin, disappearing và conversation preferences (cùng dispatcher chat).

Việc tách lớp giúp `main.c` không hình thành một file chứa mọi nghiệp vụ. Khi có lệnh mới, service tương ứng xử lý qua dispatcher riêng, còn repository chịu trách nhiệm truy cập database bằng prepared statement.

2.7.2 Xác thực, rate limit và đổi mật khẩu

Cơ chế băm mật khẩu hai tầng client-server và dẫn xuất master key đã trình bày ở mục 2.4.1.2. Ở góc độ server, điểm cần nhấn mạnh là server chỉ thao tác trên hash nhận từ client nên không bao giờ thấy mật khẩu gốc; khi đổi mật khẩu, server kiểm tra hash cũ rồi lưu hash mới, còn việc bọc lại local E2EE state bằng master key mới do client tự thực hiện. Nếu mật khẩu sai, server trả lỗi để UI báo rõ là email hoặc mật khẩu không đúng.

Rate limit được áp dụng cho luồng xác thực để giảm rủi ro brute-force. Log không nên ghi trực tiếp email hoặc dữ liệu nhạy cảm.

2.7.3 Lưu và chuyển tiếp message

Khi nhận message, server kiểm tra người gửi có quyền trong conversation và body có prefix E2EE hợp lệ (mục 2.5.2), nhưng không giải mã message. Sau khi lưu vào bảng `messages`, server phát event cho các thành viên đang online; nếu người nhận offline, message vẫn nằm trong lịch sử và inbox giúp client tải lại khi người đó đăng nhập. Riêng edit message được lưu theo hướng event-sourced - thêm một row event trở về tin gốc qua `edit_target_message_id` thay vì ghi đè ciphertext cũ - nhờ đó bản gốc không bị mất.

2.7.4 Group và MLS validation

Quy trình hai bước `prepare/apply` cùng các kiểm tra của MLS validator trên `PublicGroup` đã được mô tả ở mục 2.4.1.5. Ở góc độ server, điểm cần nhấn mạnh là server không bao giờ chỉ nhận base64 artifact rồi cập nhật thẳng bảng thành viên: mọi thay đổi membership chỉ được ghi sau khi validation pass, ngược lại handshake không được lưu và membership giữ nguyên. Vì chỉ làm việc trên public state, server có đủ dữ liệu để từ chối Commit giả mạo hoặc nằm ngoài policy nhưng không có application secret để đọc nội dung group. Các artifact MLS đã validate được lưu để thành viên replay Welcome/Commit đúng thứ tự khi online lại hoặc tải lại history.

2.7.5 Vận hành và giám sát

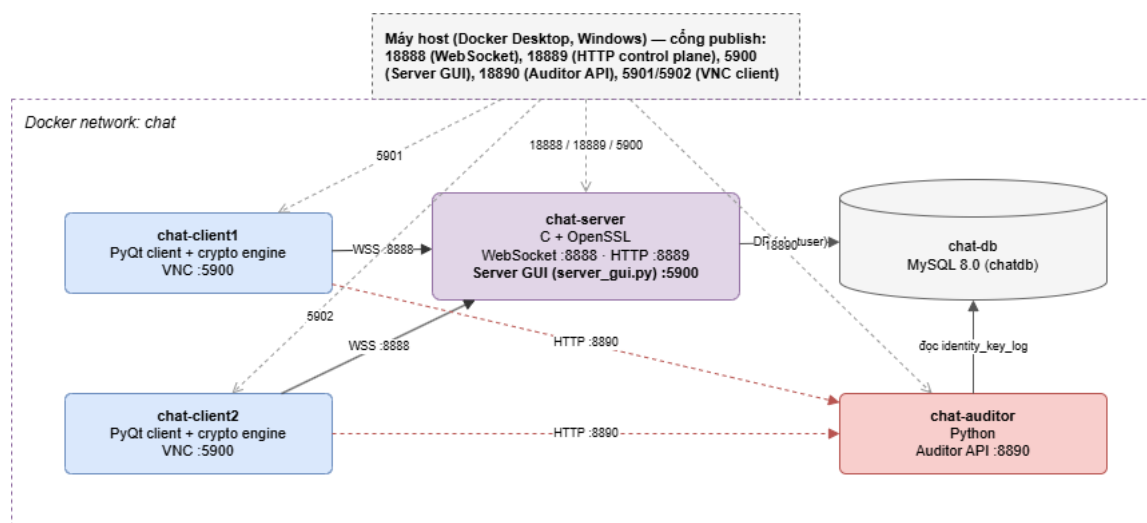
Server có HTTP control plane để trả health, readiness và metrics. Server GUI dùng dữ liệu này để hiển thị trạng thái vận hành qua các tab cần thiết như logs và metrics.

Chương 3

Cài đặt và đánh giá thực nghiệm

3.1 Môi trường cài đặt

SecChat được triển khai trong môi trường Docker Compose để bảo đảm kết quả chạy có thể lặp lại. Môi trường sử dụng Windows 11, Docker Desktop, Python 3, PyQt5, C/OpenSSL cho server, MySQL 8.0 cho lưu trữ, Rust/OpenMLS cho MLS bridge và TigerVNC để quan sát nhiều client đồng thời. Cấu trúc triển khai gồm một container database, một container server, một container auditor và hai container client VNC.



Hình 3.1: Mô hình container khi chạy SecChat

Server WebSocket được expose tại cổng 18888; HTTP control plane gồm /healthz, /readyz và /metrics được expose tại cổng 18889. Hai client VNC dùng cổng 5901 và 5902; server GUI dùng cổng 5900. Trong môi trường demo, các tài khoản 123@gmail.com, 234@gmail.com và 345@gmail.com dùng mật khẩu 123456.

3.2 Cài đặt và khởi động hệ thống

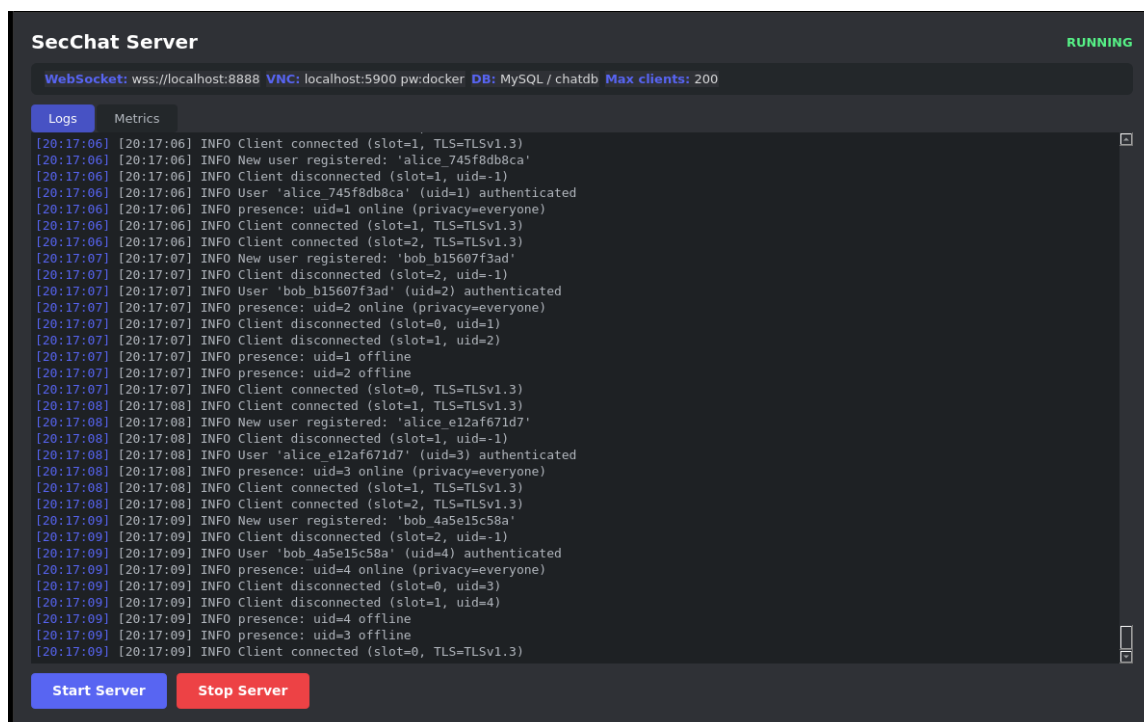
Toàn bộ hệ thống có thể build và khởi động bằng Docker Compose. Lệnh sau tạo image, chạy migration và khởi động các service:

```
1 docker compose up -d --build
```

Khi cần reset dữ liệu để kiểm tra lại từ trạng thái sạch, hệ thống được dừng kèm volume và khởi động lại:

```
1 docker compose down -v
2 docker compose up -d --build
```

Endpoint `/readyz` được dùng để xác nhận server đã kết nối database và sẵn sàng nhận client. Khác với `/healthz`, readiness phản ánh trạng thái phụ thuộc vào MySQL nên phù hợp để kiểm tra trước khi chạy demo hoặc test.

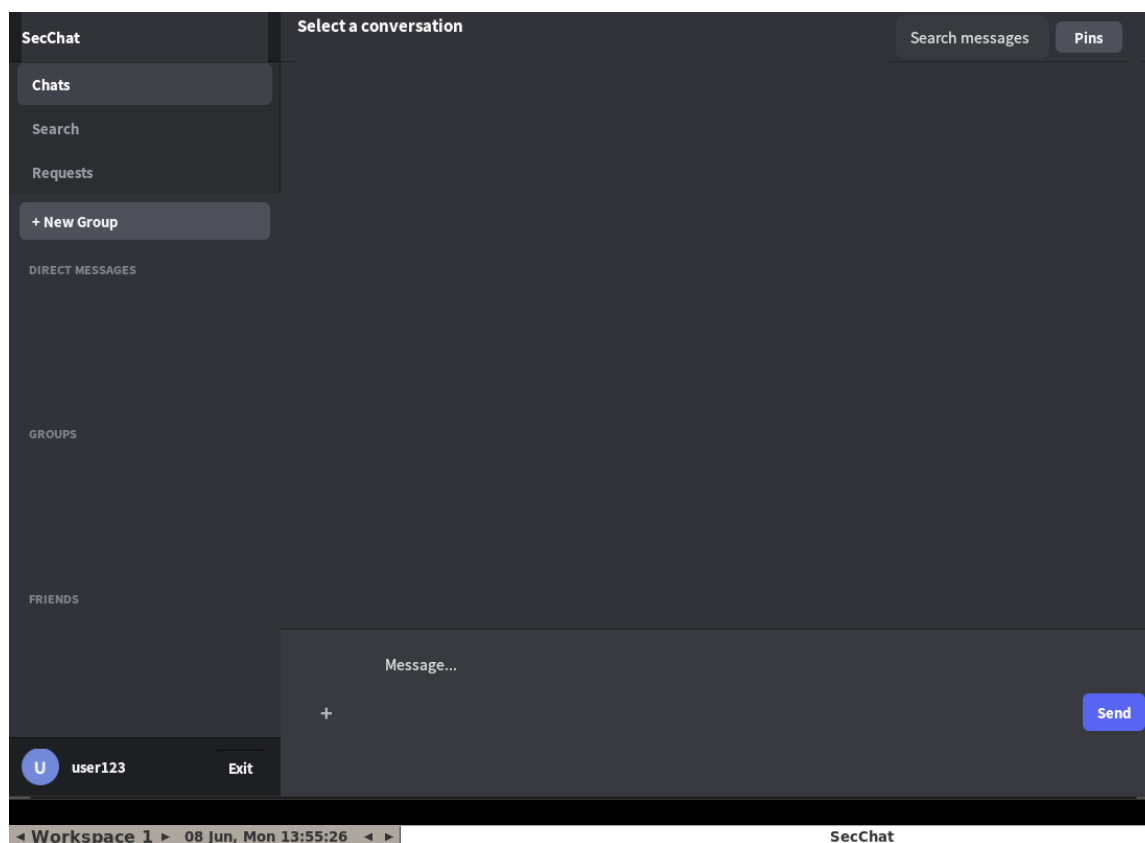


Hình 3.2: Giao diện server khi hệ thống đang chạy

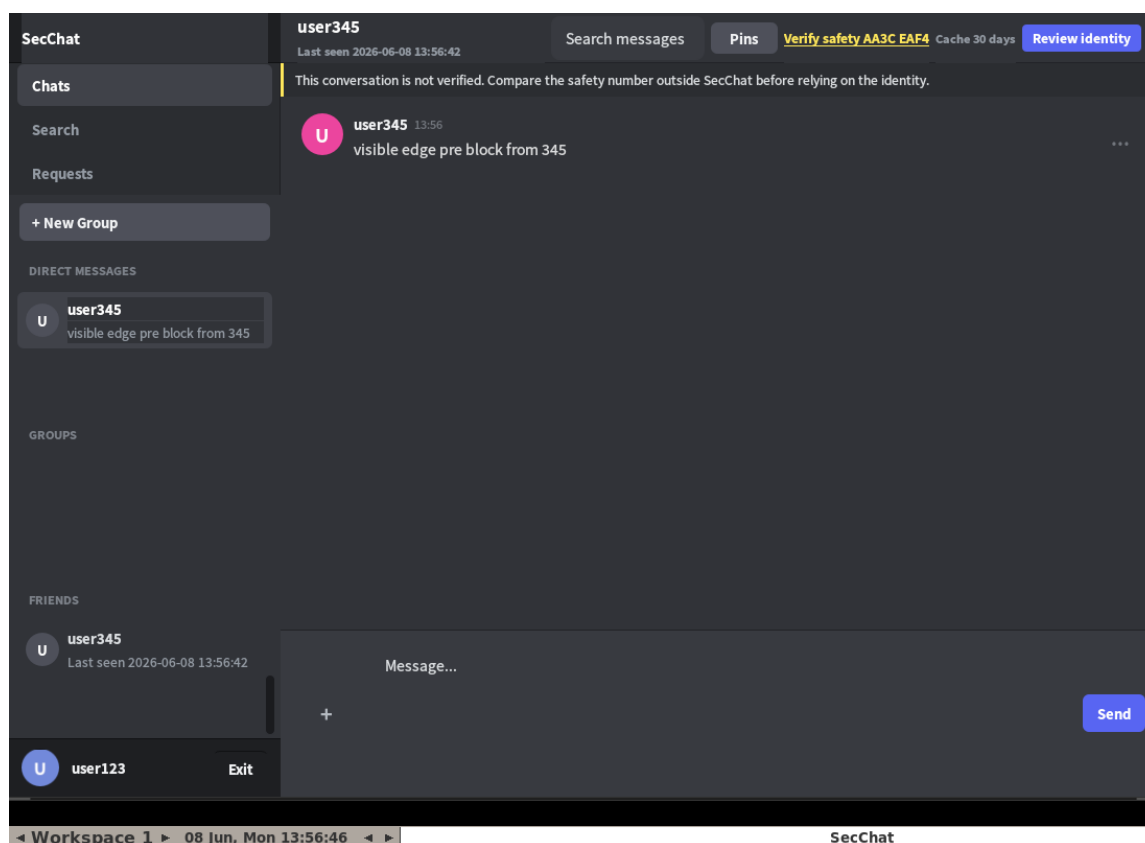
3.3 Kết quả giao diện

3.3.1 Đăng nhập và danh sách hội thoại

Sau khi đăng nhập, client mở local state E2EE, tải danh sách bạn bè, DM, group, request và các badge trạng thái.



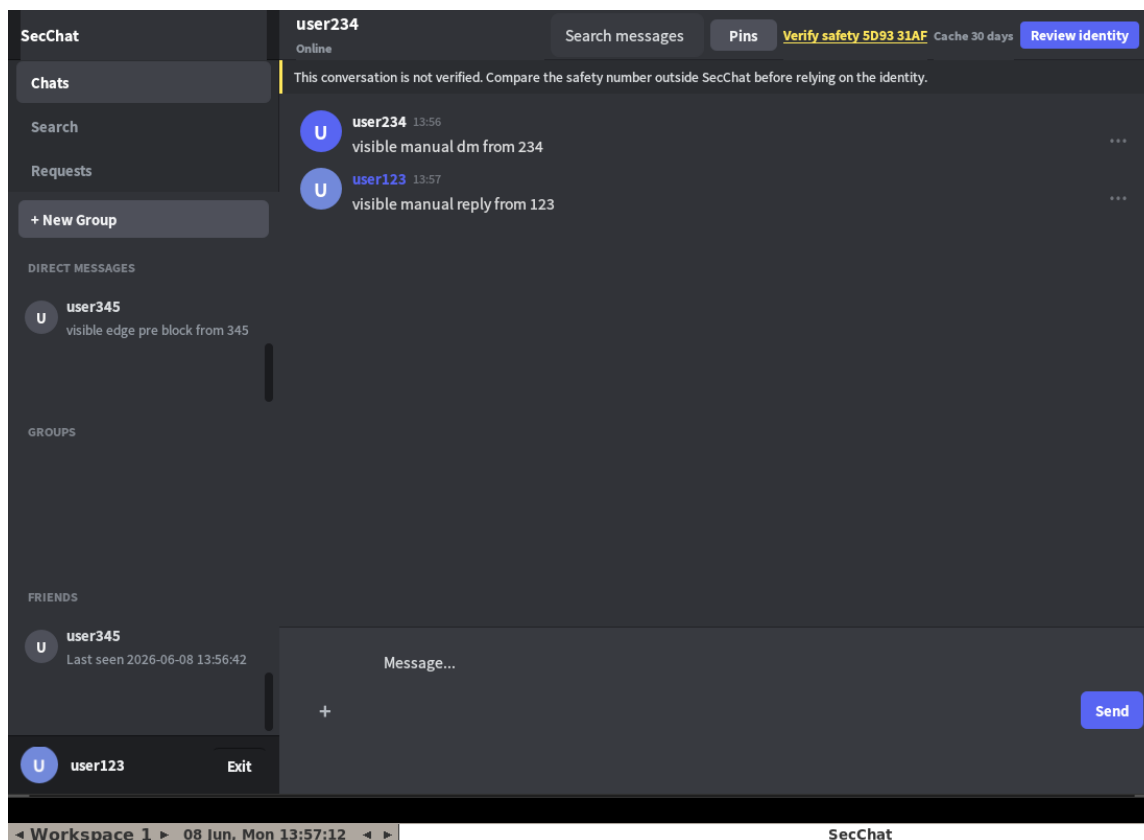
Hình 3.3: Màn hình đăng nhập trong môi trường VNC



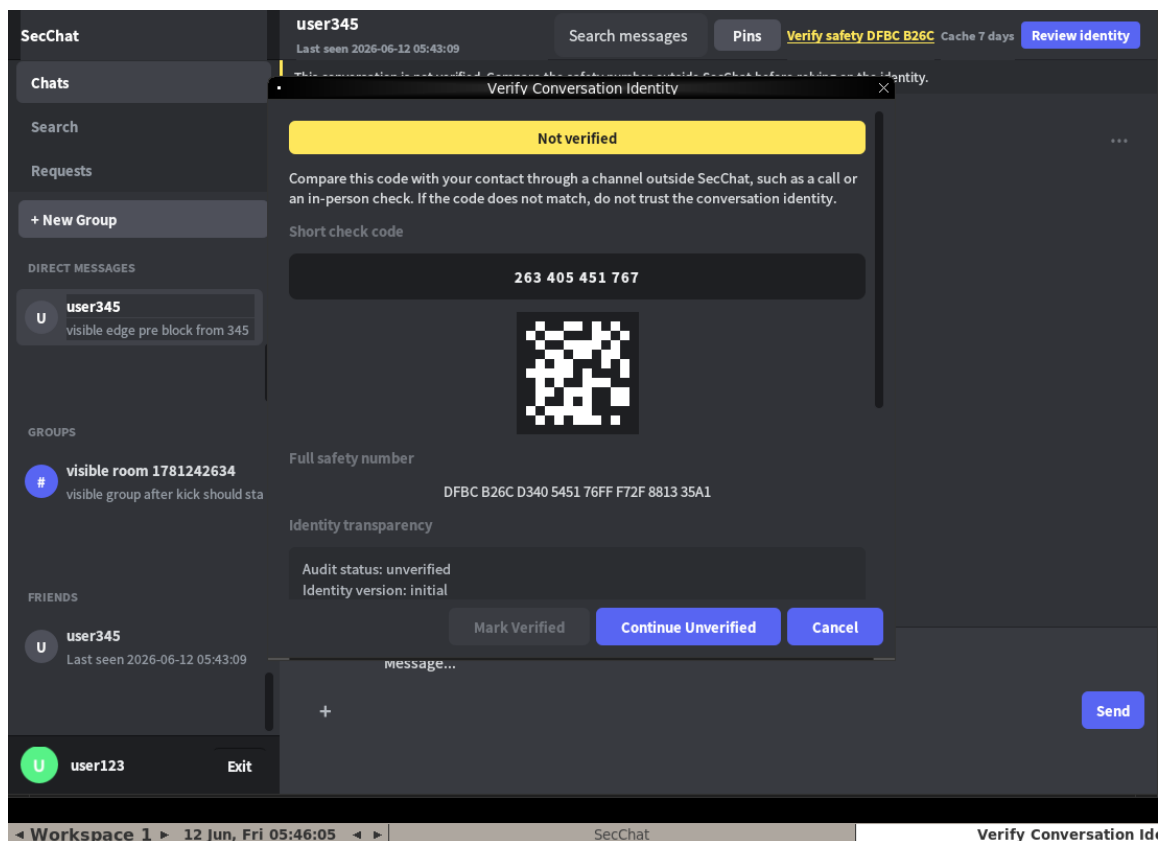
Hình 3.4: Danh sách bạn bè, DM và group sau khi đăng nhập

3.3.2 DM, identity và thao tác tin nhắn

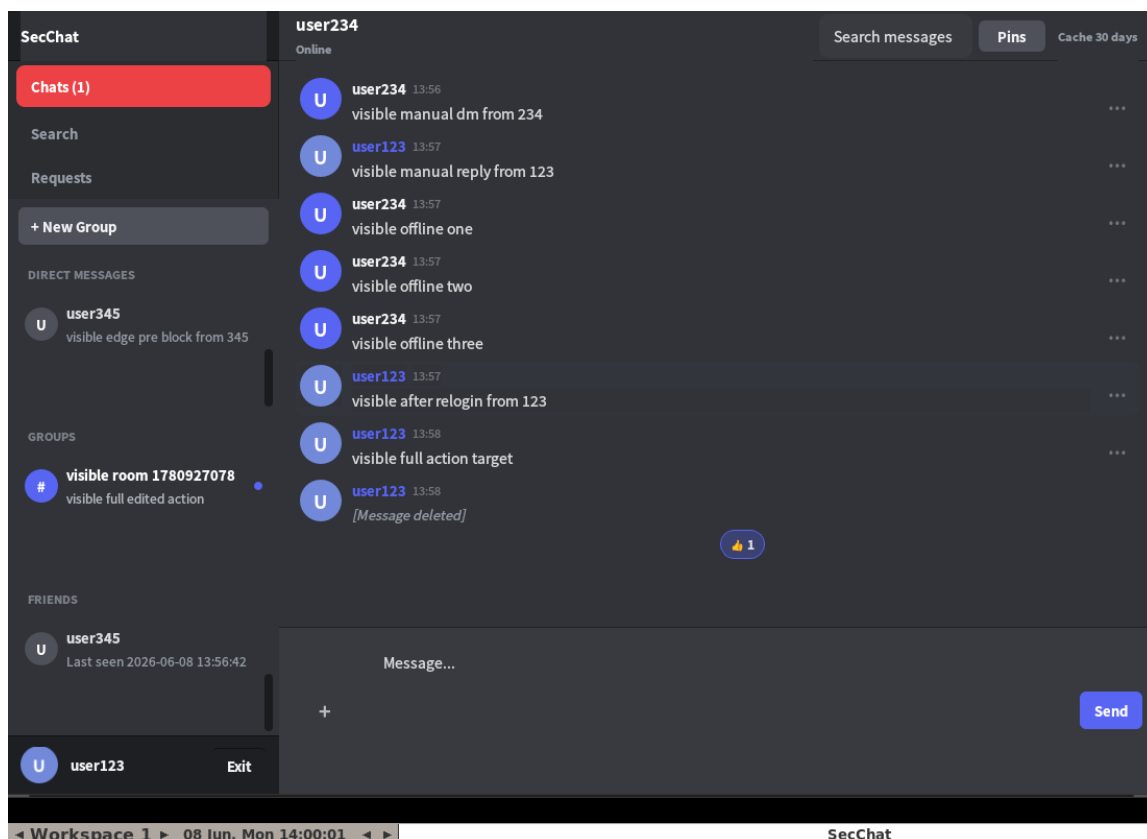
DM hiển thị header bảo mật, trạng thái review identity, message timeline và vùng nhập tin. Khi hai người dùng chưa review identity, giao diện hiển thị hướng dẫn Review identity.



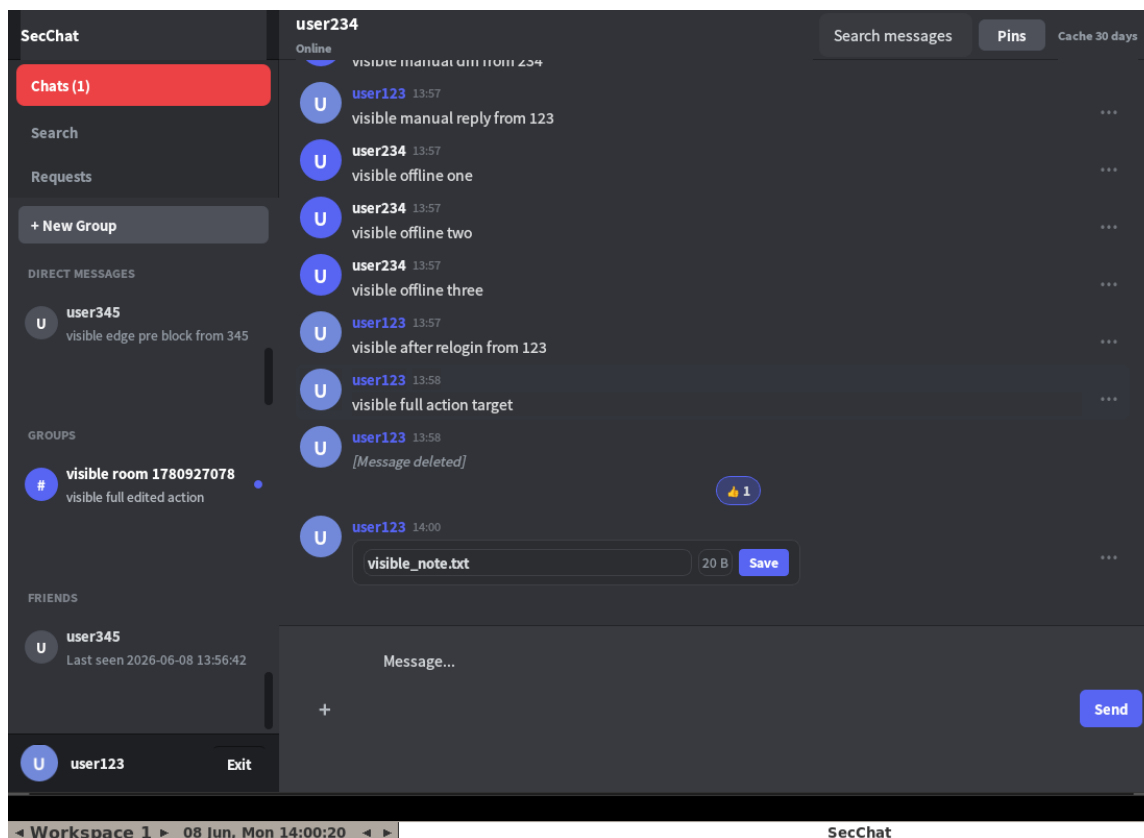
Hình 3.5: Gửi nhận DM hai chiều với nội dung được giải mã ở client



Hình 3.6: Màn hình kiểm tra safety code trong DM



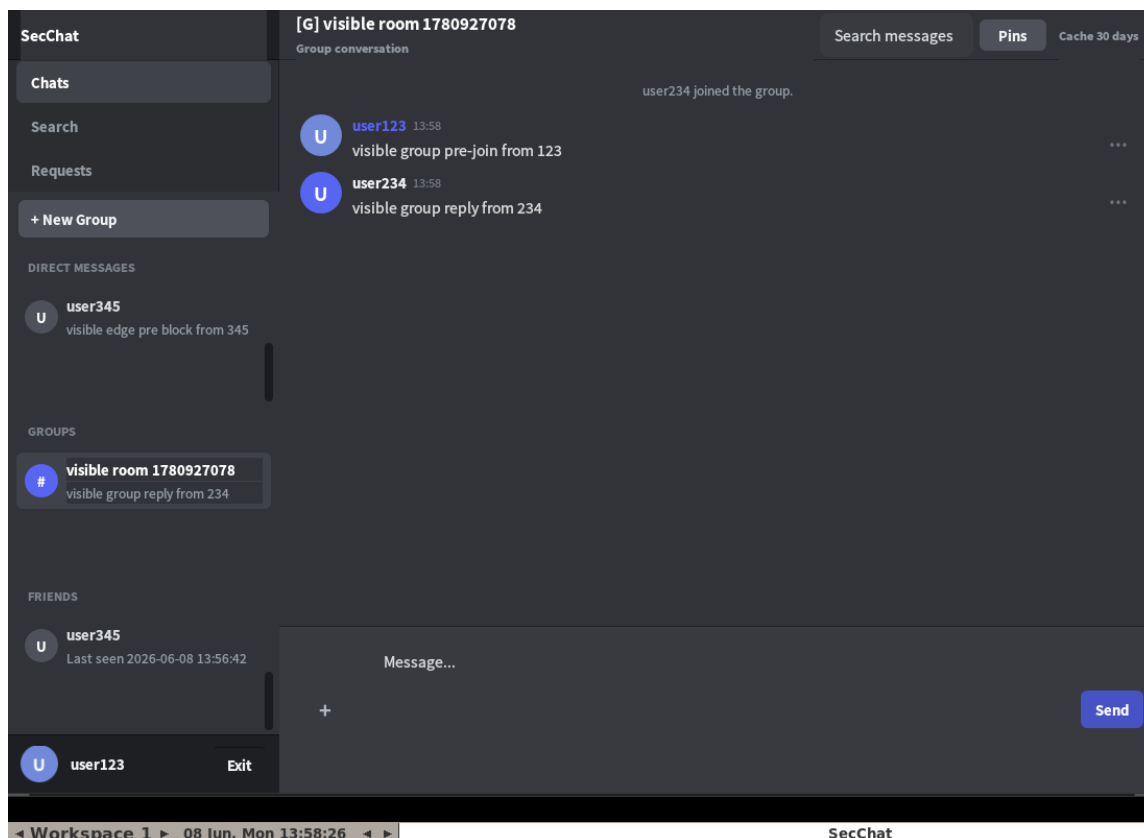
Hình 3.7: Popover thao tác tin nhắn



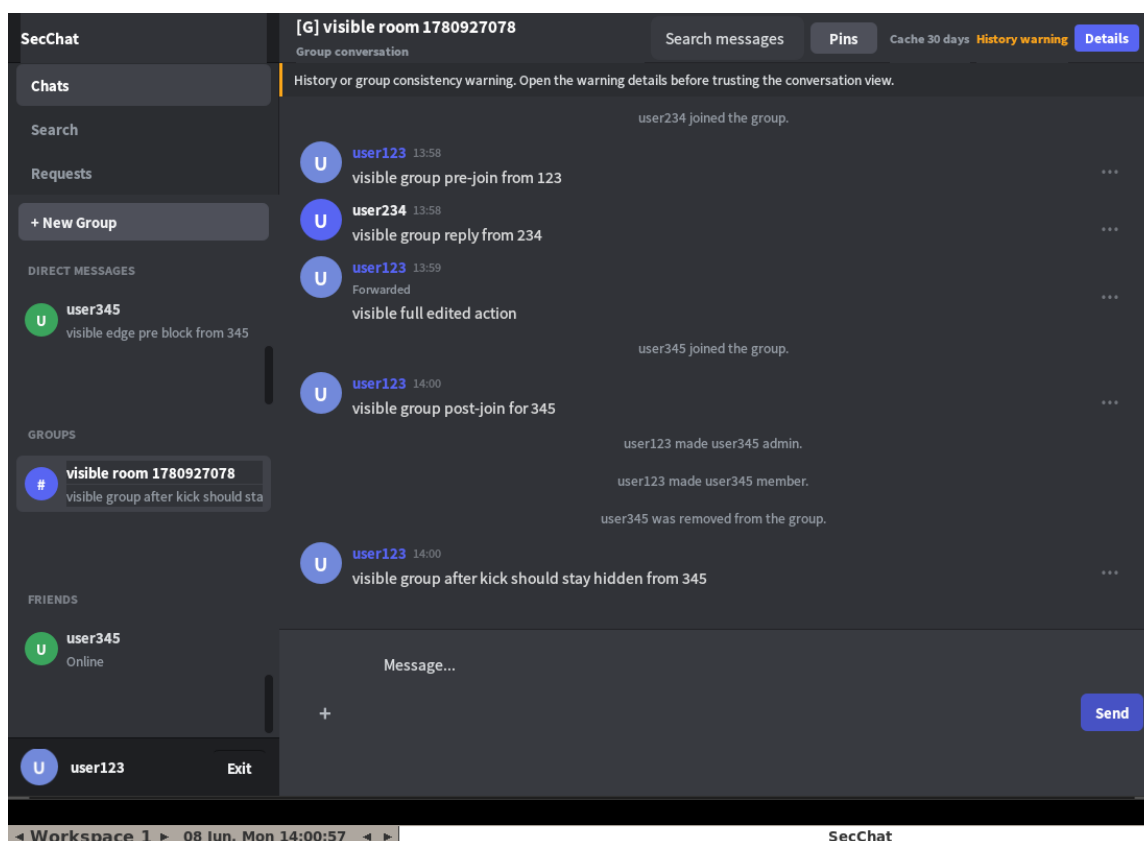
Hình 3.8: Gửi file trong hội thoại

3.3.3 Group chat và quản lý thành viên

Group chat hiển thị trạng thái MLS/PQ ở header. Thay đổi member, role và thông tin group xuất hiện trong timeline như system event ổn định.



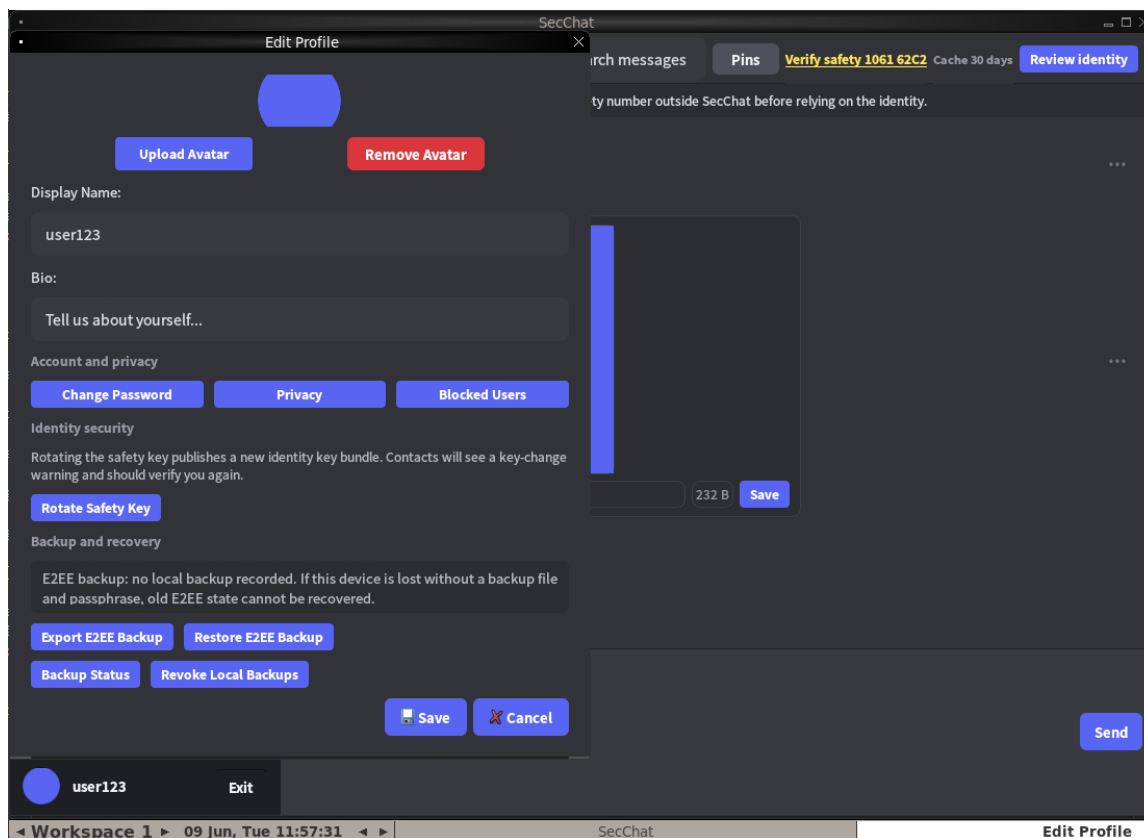
Hình 3.9: Màn hình group chat với trạng thái MLS/PQ



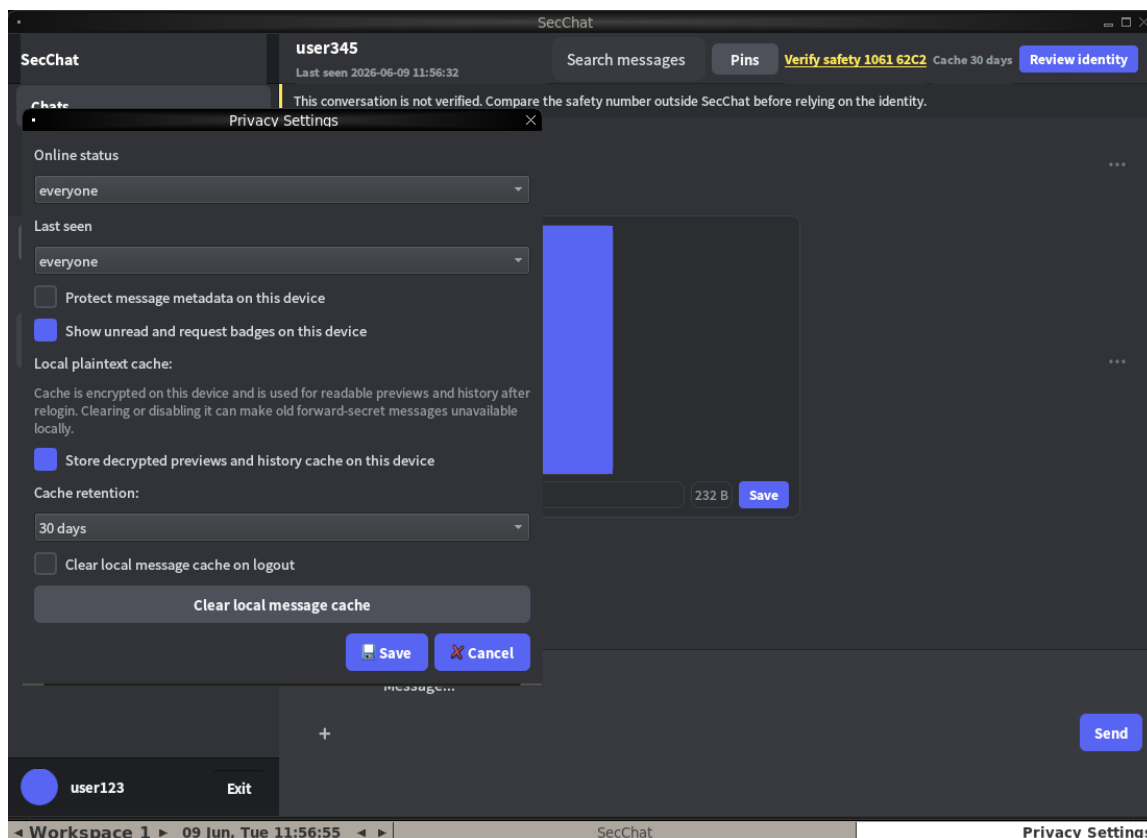
Hình 3.10: Quản lý thành viên, role và system event trong group

3.3.4 Backup, profile và trạng thái cuối

Client cung cấp chức năng quản lý avatar, privacy, cache và backup E2EE trong profile. Backup là file do người dùng giữ, được mã hóa bằng passphrase riêng. Khi restore trên thiết bị khác, client re-wrap local state và publish public material mới mà không đưa plaintext message lên server.



Hình 3.11: Profile, privacy, avatar và backup E2EE



Hình 3.12: Các tùy chọn privacy và cache

3.4 Kiểm thử chức năng

Bộ kiểm thử tự động chạy trực tiếp trên stack Docker thật (server C, MySQL, auditor, hai client) và đối chiếu giao thức với server đang chạy. Mỗi file test chạy trong một tiến trình con riêng để một socket treo ở file này không làm hỏng file kế tiếp. Trình điều phối `tests/run_all_tests.py` gom 19 file thành ba nhóm và in bảng tổng kết. Kết quả: toàn bộ **19/19 file đều PASS, 0 fail** (Bảng 3.1).

Nhóm	Số file	Nội dung kiểm tra	KQ
Tích hợp	4	Domain layer, auth/kết bạn, presence/metrics/search, chặn & disappearing.	4/4
Server contract	13	HTTP control plane, conv prefs, privacy, multi-session, đếm metrics, giới hạn input, notification, avatar, saved messages, rate limit, identity audit, secure protocol (server), fault injection.	13/13
Crypto contract	2	Bắt tay PQXDH & Double Ratchet, crypto session.	2/2
Tổng	19		100%

Bảng 3.1: Tổng hợp kết quả kiểm thử tự động (chạy trên stack Docker)

Ảnh chụp màn hình terminal của một phiên chạy trình điều phối `tests/run_all_tests.py` được trình bày ở Hình 3.13.

```
FILE                                STATUS    TIME
-----
tests/integration/test_domain_layer.py    PASS      1.6s
tests/integration/test_auth_friendship_validation.py PASS      44.6s
tests/integration/test_presence_metrics_search.py PASS      11.0s
tests/integration/test_blocking_disappearing_messages.py PASS      87.7s
tests/test_http_control.py                PASS      16.7s
tests/test_conv_prefs.py                  PASS      51.9s
tests/test_privacy.py                     PASS      32.8s
tests/test_multisession.py                PASS      31.1s
tests/test_metrics_counters.py            PASS      30.8s
tests/test_limits.py                      PASS      25.1s
tests/test_notifications.py               PASS      65.7s
tests/test_avatar.py                      PASS      25.3s
tests/test_saved_messages.py              PASS      30.8s
tests/test_rate_limit.py                  PASS      40.2s
tests/test_identity_audit.py              PASS       1.1s
tests/test_secure_protocol_server.py      PASS     229.5s
tests/test_operational_fault_injection.py PASS       76.2s
tests/test_secure_protocol_crypto.py      PASS       0.8s
tests/test_crypto_session.py              PASS       3.4s
-----
19/19 passed, 0 failed
=====
(run: python tests/run_all_tests.py --category integration --category server_contract --category crypto_contract)
```

Hình 3.13: Kết quả chạy bộ kiểm thử tự động: 19/19 file PASS, 0 fail

Mỗi file lại gồm nhiều test-case con. Ví dụ nhóm kiểm tra tầng dữ liệu (domain) in chi tiết từng khẳng định:

```
$ python tests/run_all_tests.py # chỉ tải test-case con cá»§a nh»³m domain + security
y
SecChat domain unit tests    ... Results: 75 passed, 0 failed
Full server build (domain)   ... Results:  4 passed, 0 failed
Security regression suite    ... Results: 16 passed, 0 failed
Security and metrics suite   ... Results: 17 passed, 0 failed
```

Hình 3.14: Chi tiết test-case con: domain (75), build server (4), an toàn (16), metrics (17) đều PASS.

3.4.1 Kiểm thử thuộc tính an toàn

Vì đây là hệ thống bảo mật, ngoài kiểm thử chức năng còn có các kiểm thử *đối kháng*: cố tình thực hiện thao tác không hợp lệ để xác nhận hệ thống từ chối đúng. Bảng 3.2 liệt kê các thuộc tính an toàn cốt lõi cùng test xác minh và kết quả.

Bảng 3.2: Kiểm thử các thuộc tính an toàn (đối kháng)

Thuộc tính	Cách kiểm	Kỳ vọng	KQ
Server không lưu plaintext	truy vấn bảng <code>messages</code> sau hội thoại	mọi body có prefix <code>S3*</code>	Đạt
Body sai prefix / plaintext bị từ chối	gửi body không hợp lệ (domain validation)	server trả lỗi, không lưu	Đạt
Người ngoài hội thoại không thao tác được	non-member gọi <code>pin_message/history</code>	bị từ chối	Đạt

Thuộc tính	Cách kiểm	Kỳ vọng	KQ
Người bị chặn không gửi/không mở DM	user bị block gửi DM, mở DM mới	bị từ chối	Đạt
Chống brute-force đăng nhập	gửi auth sai liên tục	bị rate limit	Đạt
Edit không ghi đè bản gốc	chỉnh sửa tin	thêm row event trở về gốc	Đạt
Replay ciphertext cũ bị chặn	gửi lại tin có counter cũ	bị từ chối	Đạt
Rotate identity bị phát hiện	đổi identity rồi nhận bundle	audit báo key changed	Đạt
MLS validator từ chối commit sai	nap commit/group state không hợp lệ	bị từ chối	Đạt
Phục hồi khi DB/server gián đoạn	tạm dừng DB/server giữa phiên	client báo lỗi rõ, không hỏng state	Đạt

Bằng chứng trực tiếp cho thuộc tính “server không lưu plaintext”: sau một đợt chạy, truy vấn thẳng bảng `messages` trong MySQL cho thấy mọi trường body còn lưu đều là ciphertext có prefix giao thức (S3DR - Double Ratchet, dạng base64), không có bản rõ:

```
$ mysql -u chatuser -p chatdb --table # truy van thang bang messages sau hoi thoai
+-----+
| total |
+-----+
|      2 |
+-----+
+-----+-----+
| prefix | n |
+-----+-----+
| S3DR:  | 2 |
+-----+-----+
+-----+-----+-----+-----+
| id | conv | body_sample |
+-----+-----+-----+-----+
| 2  | 1    | S3DR:eyJ2IjoyLCJkaCI6InRlc3QiLCJwbiI6MCwibiI |
| 1  | 1    | S3DR:eyJ2IjoyLCJkaCI6InRlc3QiLCJwbiI6MCwibiI |
+-----+-----+-----+-----+
```

Hình 3.15: Bằng chứng server không lưu plaintext: truy vấn bảng `messages`, mọi body là ciphertext prefix S3DR.

3.5 Kiểm toán mã nguồn server

Server viết bằng C nên ngoài kiểm thử chức năng cần kiểm toán an toàn bộ nhớ ở tầng mã nguồn. Các lớp rủi ro cao được tập trung kiểm tra: WebSocket frame parser, JSON command parser, đường lỗi database và MLS validator. Bảng 3.3 tổng hợp phương pháp và kết quả định lượng (chạy trong container server).

Các công cụ dùng đều là công cụ kiểm thử mã C tiêu chuẩn, được giới bảo mật tin dùng: `cppcheck` (phân tích tĩnh), `AddressSanitizer/UndefinedBehaviorSanitizer`

của Clang (phát hiện lỗi bộ nhớ và hành vi không xác định lúc chạy), và **libFuzzer** (fuzzing dẫn hướng theo độ phủ). Bảng 3.3 tổng hợp phương pháp và số liệu định lượng thu được khi chạy trong container server.

Hạng mục	Công cụ / phương pháp	Số liệu thực đo	KQ
Lỗi bộ nhớ / hành vi không xác định	Clang AddressSanitizer + UBSan, chạy test_domain_unit_asan	biên dịch sạch, chạy hết	75/75, 0 lỗi
Phân tích tĩnh	cppcheck (error/warning/performance/portability/style)	36/36 file .c	0 cảnh báo
WebSocket frame parser	libFuzzer fuzz_ws_frame (+ASan+UBSan)	30.000 iter, cov 64, RSS $\approx$ 39 MB	0 crash
JSON command parser	libFuzzer fuzz_json_validation (+ASan+UBSan)	30.000 iter, cov 719, RSS $\approx$ 129 MB	0 crash

Bảng 3.3: Kết quả kiểm toán mã nguồn server

Output thực tế của bốn bước định lượng:

```
$ make cppcheck #phan tich tinh 36 file .c
36/36 files checked 100% done
(0 finding: error / warning / performance / portability / style)

$ make asan_test_domain # clang -fsanitize=address,undefined
=====
Results: 75 passed, 0 failed
=====

$ make fuzz_smoke FUZZ RUNS=30000 # libFuzzer + ASan/UBSan, 2 parser
[json_validation] #30000 DONE cov: 687 ft: 2263 corp: 164/9257b rss: 130Mb
Done 30000 runs in 1 second(s) --> 0 crash artifact
[ws_frame] #30000 DONE cov: 64 ft: 146 corp: 35/694b rss: 39Mb
Done 30000 runs in 0 second(s) --> 0 crash artifact
```

Hình 3.16: Kiểm toán mã C trong container: cppcheck 36/36, ASan/UBSan 75/75, libFuzzer 30k×2 0 lỗi, 0 crash.

3.5.1 Kiểm thử group MLS

Phần MLS được hiện thực bằng OpenMLS - thư viện tham chiếu cho RFC 9420. Lệnh `secchat_mls_bridge selftest` chạy một vòng đầy đủ: tạo group, thêm member, phát hành Welcome, gửi application message và remove member; đồng thời kiểm tra validator chấp nhận trạng thái hợp lệ và từ chối commit giả mạo. Output JSON thực tế xác nhận ciphersuite hybrid X-Wing và các tính chất đã nêu:

```
$ secchat_mls_bridge selftest
{
  "add_commit_bytes": 5609,
  "app_bytes": 168,
  "backend": "openmls",
  "bob_active_after_remove": false,
  "ciphersuite": "MLS_256_XWING_CHACHA20POLY1305_SHA256_Ed25519",
  "ok": true,
  "openmls": "0.8.1",
  "pq_authentication": false,
  "pq_authentication_note": "MLS signatures remain Ed25519 classical authentication",
  "pq_hybrid": true,
  "pq_kem": "X-Wing draft hybrid KEM (ML-KEM-768 + X25519)",
  "remove_commit_bytes": 1618,
  "validator_create_ok": true,
  "validator_rejects_bogus_commit": true,
  "validator_remove_ok": true,
  "welcome_bytes": 5625
}
```

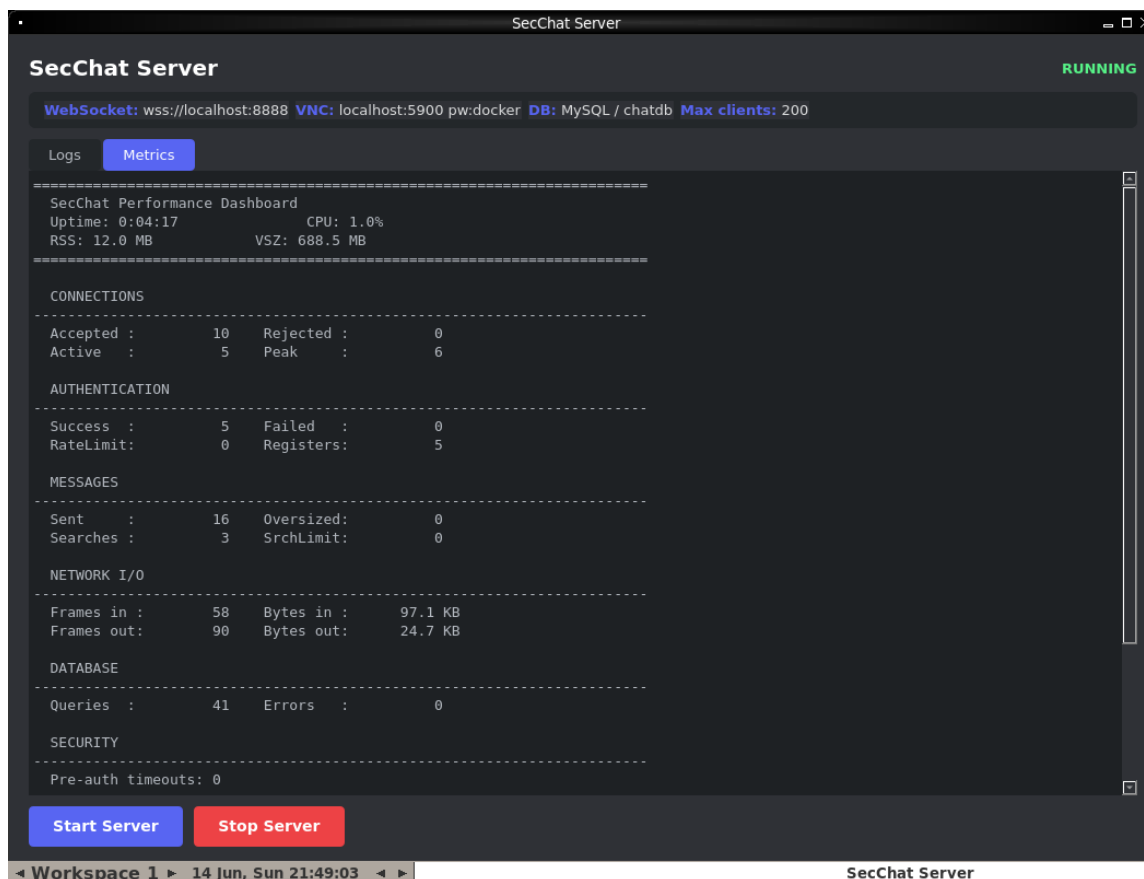
Hình 3.17: Selftest MLS bridge (OpenMLS): ciphersuite X-Wing hybrid; validator chấp nhận state hợp lệ, từ chối commit giả mạo.

3.6 Quan sát vận hành

Để minh họa việc quan sát vận hành trực tiếp, sinh một lượng traffic ở mức bình thường rồi giữ các kết nối mở trong lúc chụp màn hình. Kịch bản tải gồm 5 client đăng ký và đăng nhập; mỗi client mở một kết nối khi đăng ký và một kết nối cho phiên làm việc, nên tổng cộng có 10 lượt kết nối được chấp nhận nhưng tối đa chỉ 6 kết nối tồn tại đồng thời. Năm client này tạo 5 quan hệ bạn bè, mở một cuộc DM hai chiều giữa hai người (mỗi bên gửi 5 tin) kèm một lượt sửa, một lượt xóa và một lượt thả cảm xúc, thêm một cuộc DM một chiều 5 tin, lập một nhóm MLS gồm 3 thành viên, rồi đọc inbox và tìm kiếm người dùng (sinh truy vấn cơ sở dữ liệu). Sau khi sinh xong, 5 kết nối được giữ mở để dashboard hiển thị đồng thời cả *active connections* lẫn các bộ đếm tích lũy. Tab *Metrics* của server GUI tự cập nhật mỗi 2 giây bằng cách đọc `/metrics`; các bộ đếm là tích lũy kể từ khi server khởi động và phiên đo này bắt đầu từ trạng thái đếm bằng 0. Hình 3.18 là ảnh chụp dashboard, còn Bảng 3.4 là số liệu `/metrics` đọc đúng thời điểm chụp.

Chỉ số (phiên theo dõi)	Giá trị thực đo
Phiên xác thực xử lý	5 (ok 5 / fail 0 / rate-limited 0)
Tài khoản đăng ký	5
Message gửi / sửa / xóa	16 / 1 / 1
Tìm kiếm người dùng / bị giới hạn	3 / 0
Nhóm tạo / reaction	1 / 1
Kết nối chấp nhận / đỉnh đồng thời / đang mở	10 / 6 / 5
WebSocket frame nhận / gửi	58 / 90
Truy vấn cơ sở dữ liệu / lỗi	41 / 0
Độ trễ TB auth / gửi tin / DB	218 ms / 26 ms / 11 ms

Bảng 3.4: Số liệu `/metrics` đọc đúng thời điểm chụp màn hình quan sát



Hình 3.18: Ảnh GUI metric của server.

Tổng hợp lại, các yêu cầu chính của sản phẩm đã được kiểm chứng: server không lưu plaintext message, DM dùng bắt tay hậu lượng tử và Double Ratchet, group dùng MLS X-Wing hybrid, server validate public MLS state trước khi thay đổi membership, timeline group giữ đúng thứ tự system event, unread state ổn định theo message id và restore E2EE hoạt động trong kịch bản nhiều thiết bị.

3.7 Đánh giá bảo mật

3.7.1 Nội dung tin nhắn

DM dùng bắt tay PQXDH-style để thiết lập secret ban đầu, sau đó dùng Double Ratchet cho từng message. Group dùng MLS theo RFC 9420 qua OpenMLS cho KeyPackage, Welcome, Commit, GroupInfo và application message. Ciphersuite group là X-Wing hybrid:

MLS_256_XWING_CHACHA20POLY1305_SHA256_Ed25519

Server chỉ lưu ciphertext và public MLS artifact đã validate, không giữ secret giải mã nội dung.

Nếu ciphertext bị sửa trực tiếp trong database hoặc trên đường truyền sau khi ra khỏi TLS, client sẽ không giải mã hợp lệ. Thay vì hiển thị dữ liệu sai, UI phải hiển thị trạng thái không thể decrypt. Đây là hành vi đúng trong hệ thống mã hóa có xác thực.

3.7.2 Kháng lượng tử

SecChat dùng ML-KEM hoặc Kyber trong bước thỏa thuận khóa DM, và dùng X-Wing trong MLS group. Đây là mật mã hậu lượng tử chạy bằng phần mềm. Nó khác QKD, vì QKD cần thiết bị vật lý và kênh lượng tử. Trong SecChat, dữ liệu vẫn đi qua mạng TCP/TLS bình thường.

Cơ chế hybrid trộn secret từ ML-KEM hoặc Kyber với X25519 ở DM, và trộn ML-KEM-768 với X25519 trong X-Wing ở group. Ý nghĩa của hybrid là không đặt toàn bộ niềm tin vào một giả định duy nhất. Nếu máy tính lượng tử trong tương lai phá được X25519 nhưng ML-KEM vẫn an toàn, transcript cũ vẫn có lớp bảo vệ hậu lượng tử. Nếu một giả định hậu lượng tử bị yếu trong tương lai nhưng X25519 vẫn an toàn trước attacker hiện tại, phần cổ điển vẫn đóng góp bảo vệ.

Phần hậu lượng tử của SecChat tập trung vào confidentiality và thiết lập bí mật. Chữ ký dùng để xác thực identity, prekey và credential MLS vẫn là Ed25519 cổ điển. Vì vậy báo cáo không tuyên bố hệ thống có authentication hậu lượng tử đầy đủ.

3.7.3 Forward secrecy và post-compromise security

DM có Double Ratchet state, nên tốt hơn thiết kế dùng một khóa hội thoại cố định. Mỗi message dùng message key được dẫn xuất riêng. Khi ratchet state tiến lên, lộ một key ở thời điểm sau không tự động làm lộ toàn bộ quá khứ.

3.7.4 Xác thực danh tính

Identity key Ed25519 ký các prekey, giúp client phát hiện prekey không khớp với identity được công bố. Safety number giúp hai người dùng so sánh identity của nhau qua kênh khác. Khi một người rotate identity, server ghi sự kiện vào `identity_key_log`, auditor dựng Merkle tree và ký checkpoint. Client kiểm tra inclusion proof khi nhận bundle, lưu lịch sử identity cục bộ và hiển thị trạng thái đã xác minh, cần xác minh, key changed, `audit_mismatch` hoặc `audit_unavailable` trong danh sách hội thoại. Các trạng thái audit có thêm mã lỗi như identity version không có trong log, identity key không khớp, proof root không khớp, checkpoint rollback, checkpoint split-view, chữ ký checkpoint sai hoặc auditor không truy cập được.

Thao tác Mark Verified ghi nhận rằng người dùng đã so sánh safety number hoặc short check code của hội thoại hiện tại qua kênh ngoài SecChat và thấy khớp. Nó không sinh khóa mới và không làm ciphertext mạnh hơn, nhưng là quyết định quan trọng để người dùng xác nhận danh tính của peer sau cảnh báo. Nếu short check code ở hai phía khác nhau, người dùng không nên mark verified vì điều đó cho thấy hai client không đang nhìn cùng một identity state. Khi identity đổi, trạng thái đã xác minh cũ không còn đủ tin cậy và UI chuyển sang cảnh báo key changed hoặc cảnh báo audit có nguyên nhân cụ thể để người dùng review lại. Nếu người dùng restore đúng identity secret trên thiết bị mới, safety identity không đổi; peer không cần thay key, nhưng vẫn có thể cần kiểm tra lại nếu UI hiển thị cảnh báo do lịch sử identity hoặc checkpoint không đủ tin cậy. Nếu người dùng không restore và chọn rotate safety key, đây là thay đổi identity thật sự và các peer phải xem như một safety number mới.

3.7.5 Metadata

E2EE không che được toàn bộ metadata. Server vẫn biết user id, conversation id, thành viên group, thời điểm gửi, kích thước tương đối của message, trạng thái online, pin,

reaction, edit target và một số sự kiện privacy-sensitive nếu người dùng bật chúng.

Client có một số biện pháp giảm metadata như tắt typing indicator, padding plaintext trước khi mã hóa, kiểm tra consistency của history và kiểm tra commit chain của group.

Server cũng validate public MLS state để tránh membership bị mutate bởi Commit không hợp lệ, nhưng việc này không ảnh hưởng membership khỏi server.

Những biện pháp này giúp giảm hoặc phát hiện một số rủi ro, nhưng không làm server trở thành không cần tin cậy. Khi phát hiện history, group commit hoặc transcript checkpoint không nhất quán, UI hiển thị cảnh báo kèm nút Details để người dùng xem nguyên nhân cụ thể. Ẩn metadata mạnh hơn cần các kỹ thuật khác như sealed sender, private information retrieval, mix network hoặc delay traffic. Các kỹ thuật đó nằm ngoài phạm vi của đồ án.

3.7.6 Local plaintext cache

Local plaintext cache giúp người dùng đọc lại lịch sử sau relogin, nhất là khi ratchet state không thể decrypt lại mọi message cũ từ đầu. Cache này được mã hóa bằng master key, nên không phải plaintext thô trên ổ đĩa. Tuy nhiên nó vẫn là dữ liệu nhạy cảm. Nếu thiết bị bị chiếm và attacker có được mật khẩu hoặc master key, lịch sử cache có thể bị đọc.

Hệ thống có chính sách cache rõ: người dùng có thể tắt cache toàn cục, tắt cache cho từng conversation nhạy cảm, đặt thời gian giữ cache, xóa cache thủ công và chọn xóa cache khi logout. Cache được đặt theo tài khoản trong thư mục dữ liệu của client, nên tài khoản mới trên cùng thiết bị không được thấy cache của tài khoản cũ và cũng không được xóa cache đó. Khi export backup E2EE, người dùng chọn có đưa message cache vào file backup hay không. Đây là đánh đổi hợp lý cho trải nghiệm desktop giữa sự tiện dụng và bảo vệ dữ liệu cục bộ.

3.8 Đánh giá kiến trúc phần mềm

3.8.1 Client

Client có ranh giới rõ giữa UI, network và crypto engine. Controller chính không trực tiếp giữ toàn bộ file key, ratchet state và cache; phần lớn state nhạy cảm và thao tác mã hóa outbound nằm trong crypto engine. Điều này làm kiến trúc dễ giải thích hơn: UI gửi yêu cầu, network truyền gói, crypto engine giữ khóa và mã hóa.

Một phần decrypt message nhận vào và replay history cũng nằm trong crypto session, nhưng controller UI vẫn điều phối khá nhiều nhánh xử lý live event, history, pending key và cập nhật UI. Vì vậy crypto engine là thành phần sở hữu state nhạy cảm, còn controller UI là lớp phối hợp sự kiện.

3.8.2 Server

Repository chịu trách nhiệm SQL, services chịu trách nhiệm nghiệp vụ, infra xử lý cấu hình và metrics, transport xử lý socket và HTTP.

Điểm cần chú ý là server viết bằng C. C cho hiệu năng tốt và kiểm soát thấp tầng, nhưng dễ gặp lỗi memory hoặc string nếu không review kỹ. Với hệ thống bảo mật, cần tiếp tục giữ quy tắc dùng prepared statement, kiểm tra độ dài input và tránh build JSON bằng buffer không đủ kích thước.

3.8.3 Cơ sở dữ liệu

Schema bao phủ nhiều nhu cầu của app: user, profile, friendship, block, conversation, member, message, reaction, pin, notification, key bundle, prekey và public MLS state. Điều này đủ cho một ứng dụng chat desktop có nhiều tính năng.

Một điểm cần quản lý là ranh giới giữa dữ liệu phục vụ giao thức chính và dữ liệu phụ trợ cho vận hành. Những bảng lưu identity, bundle, one-time prekey, notification và message metadata phải có vai trò rõ ràng để tránh nhầm lẫn giữa khóa dùng cho E2EE và metadata server cần để điều phối hệ thống.

3.9 Các nguy cơ còn lại

Hệ thống vẫn còn một số giới hạn. Thứ nhất, group protocol dùng OpenMLS theo RFC 9420 cho flow KeyPackage, Welcome, Commit và application message, với X-Wing draft hybrid ML-KEM-768 + X25519. Đây là bảo vệ hậu lượng tử cho confidentiality, không phải authentication hậu lượng tử. Do X-Wing vẫn là draft trong hệ sinh thái MLS, hệ thống cần kế hoạch migration nếu định danh ciphersuite hoặc encoding thay đổi khi chuẩn cuối cùng ổn định.

Thứ hai, metadata chưa được ẩn. Đây là vấn đề khó và cần thiết kế riêng, không thể giải quyết chỉ bằng cách mã hóa body message.

Thứ ba, local cache giúp trải nghiệm tốt hơn nhưng làm tăng lượng dữ liệu nhạy cảm trên thiết bị. Vì vậy bảo vệ thiết bị, mật khẩu mạnh và khóa màn hình vẫn rất quan trọng.

Thứ tư, hệ thống chưa có multi-device đầy đủ. Một tài khoản chủ yếu gắn với một phiên client active tại một thời điểm. Đối thiết bị được hỗ trợ ở mức khôi phục identity, công bố bundle mới và đọc lại lịch sử từ cache hoặc backup; hệ thống chưa có danh sách thiết bị, chưa đồng bộ ratchet state liên tục và chưa có cơ chế xác minh chéo giữa các thiết bị. Có thể cải thiện bằng cách cho mỗi thiết bị thêm identity phụ, prekey riêng và cơ chế thêm thiết bị an toàn.

3.10 Hướng phát triển

3.10.1 Multi-device

Hỗ trợ multi-device thực sự sẽ nâng cao trải nghiệm người dùng, nhưng cũng làm tăng độ phức tạp của giao thức và triển khai. Một số ý tưởng có thể nghiên cứu là thiết kế flow thêm thiết bị an toàn, đồng bộ ratchet state qua MLS hoặc server relay, và xác minh chéo identity giữa các thiết bị để phát hiện xâm nhập.

3.10.2 Nâng cấp bảo vệ metadata

Bảo vệ metadata là hướng khó nhưng có giá trị. Một số ý tưởng có thể nghiên cứu là sealed sender, giảm thông tin trong inbox, gom thời điểm gửi, hoặc thiết kế relay không biết đầy đủ quan hệ người gửi-người nhận. Đây nên được xem là một đề tài riêng, vì nó phức tạp hơn nhiều so với mã hóa nội dung.

3.10.3 Cải thiện UI và nền tảng

Có thể cải thiện tìm kiếm lịch sử, preview media, kéo thả nhiều file, trạng thái gửi thất bại, retry, notification và hỗ trợ mobile. Với mobile hoặc web, cần đặc biệt cẩn thận để không làm yếu mô hình E2EE khi chuyển key và local state sang nền tảng mới.

Kết luận

Qua quá trình tìm hiểu, nghiên cứu, thiết kế, triển khai và báo cáo, đồ án đã được hoàn thiện. Đề tài “Xây dựng giao thức nhắn tin an toàn tích hợp giải pháp mật mã hậu lượng tử” đã đạt được một số kết quả nhất định, cụ thể như sau:

- Tìm hiểu và hệ thống hóa các kiến thức nền tảng liên quan đến ứng dụng nhắn tin, mã hóa đầu cuối và mật mã kháng lượng tử.
- Phân tích yêu cầu và thiết kế kiến trúc cho hệ thống ứng dụng nhắn tin đảm bảo tính bảo mật và riêng tư cho người dùng.
- Xây dựng và triển khai ứng dụng nhắn tin với các chức năng cơ bản.
- Tích hợp cơ chế mã hóa đầu cuối kết hợp với các thuật toán mật mã hậu lượng tử nhằm nâng cao mức độ an toàn trước các mối đe dọa hiện tại và tương lai.
- Thực hiện thử nghiệm và đánh giá một số tính năng cơ bản cũng như hiệu quả hoạt động của hệ thống.

Trong tương lai, đề tài có thể tiếp tục được phát triển theo các hướng sau:

- Mở rộng ứng dụng với nhiều tính năng nâng cao như gọi thoại và gọi video an toàn.
- Nghiên cứu và tích hợp thêm các thuật toán mật mã hậu lượng tử tiên tiến nhằm tăng cường khả năng bảo mật.
- Tối ưu hiệu năng hệ thống, giảm độ trễ và nâng cao khả năng mở rộng khi số lượng người dùng tăng cao.
- Triển khai hệ thống trong môi trường thực tế và đánh giá trên quy mô lớn hơn.

Trong quá trình thực hiện đồ án, mặc dù đã rất cố gắng nhưng do hạn chế về thời gian và kiến thức, đồ án không tránh khỏi những thiếu sót. Em rất mong nhận được sự đóng góp ý kiến và chỉ bảo của các thầy cô và các bạn để đồ án được hoàn thiện hơn.

Em xin chân thành cảm ơn!

Tài liệu tham khảo

- [1] National Institute of Standards and Technology. Module-lattice-based key-encapsulation mechanism standard (fips 203), 2024. [Link](#).
- [2] Joppe Bos, Léo Ducas, Eike Kiltz, Tancrede Lepoint, Vadim Lyubashevsky, John M. Schanck, Peter Schwabe, Gregor Seiler, and Damien Stehlé. Crystals-kyber: a cca-secure module-lattice-based kem, 2018. [Link](#).
- [3] Peter W. Shor. Polynomial-time algorithms for prime factorization and discrete logarithms on a quantum computer, 1997. [Link](#).
- [4] Oded Regev. On lattices, learning with errors, random linear codes, and cryptography, 2005. [Link](#).
- [5] National Institute of Standards and Technology. Post-quantum cryptography standardization, 2024. [Link](#).
- [6] Katriel Cohn-Gordon, Cas Cremers, Benjamin Dowling, Luke Garratt, and Douglas Stebila. A formal security analysis of the signal messaging protocol, 2017. [Link](#).
- [7] Signal Foundation. The pqxdh key agreement protocol, 2023. [Link](#).
- [8] Trevor Perrin and Moxie Marlinspike. The double ratchet algorithm, 2016. [Link](#).
- [9] Open Quantum Safe Project. liboqs: C library for quantum-safe cryptography, 2024. [Link](#).
- [10] IETF. Rfc 9420: The messaging layer security protocol, 2023. [Link](#).
- [11] IETF MLS Working Group. Post-quantum ciphersuites for messaging layer security, 2026. [Link](#).
- [12] OpenMLS. Openmls book: Traits and provider, 2024. [Link](#).